



Data Management I

DTS224

Table of Contents

Module One: Information Management Concept and Introduction to Database Systems	1
Unit 1 — Foundations of Data Storage and Management	2
Unit 2 — Introduction to Database Systems and Architecture	8
Module Two: Introduction to Conceptual Models	22
Unit 1 — Introduction to Conceptual Models: ER Diagram	23
Unit 2 — Enhanced E-R Model	45
Unit 3 — Introduction to Semi-Structured Models (XML Model)	57
Unit 4 — Examples of Conceptual Models	66
Module Three: Logical Design	70
Unit 1 — Introduction to Relational Model	71
Unit 2 — Transforming Conceptual to Logical Design	83
Module Four: Functional Dependencies	97
Unit 1 — Normalization	98
Module Five: Relational Theory and Query Languages	121
Unit 1 — Relational Algebra	122
Unit 2 — Introduction to Structured Query Language and Introduction to Query Processing and Optimization	131
Answers to Self-Assessment Questions	167
Module One	168
Module Two	171
Module Three	174
Module Four	177
Module Five	178

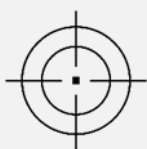
Module One

Information Management Concept and Introduction to Database Systems

Unit 1 — Foundations of Data Storage and Management

INTRODUCTION

In this unit, you will be introduced to the foundational concepts of information storage and management. You will also explore how digital information is captured, represented, stored, retrieved, secured, and analysed. Understanding these processes is vital for building and working with effective databases and information systems. This unit also examines key performance attributes such as scalability, efficiency, and effectiveness, which determine how well a system handles growing data demands and serves users reliably.



LEARNING OUTCOMES

At the end of this unit, you should be able to:

- Explain the purpose and scope of information storage and retrieval in database systems.
- Identify real-world information management applications and their use cases.
- Understand how information is captured and represented in digital form.
- Describe techniques for indexing, searching, and ensuring information privacy.
- Define the importance of data integrity and security.
- Assess a system based on scalability, efficiency, and effectiveness.

1.1.1 Information Storage and Retrieval

Information storage refers to how data is persistently stored using digital structures (e.g., files, tables, documents) on physical or cloud-based storage systems. Retrieval is the process of locating and accessing this stored data using queries or indexing mechanisms.

In database systems:

- Data is typically stored in relations (tables) or documents (in NoSQL systems).
- Structured data is stored with a schema (e.g., relational databases).
- Unstructured or semi-structured data (e.g., logs, XML, JSON) is handled with flexible models.
- Query languages (like SQL) are used to retrieve data efficiently.
- Indexing, normalisation, and metadata improve data retrievability and organisation.

1.1.2 Information Management Applications

Information management systems help users and organisations to manage, access, and safeguard data. These include:

- **Library management systems** (e.g., cataloguing books and lending history)
- **Healthcare information systems** (e.g., patient records, treatment history)
- **Enterprise Resource Planning (ERP)** systems (e.g., HR, accounting)
- **Customer Relationship Management (CRM)** platforms (e.g., client interactions)
- **Content management systems** (e.g., websites, digital asset systems)

These applications are underpinned by databases that enforce consistency, privacy, and availability of data.

1.1.3 Information Capture and Representation

Information capture

Information capture refers to the process of collecting data from real-world sources. This could include:

- Manual entry (e.g., filling forms)
- Sensors/IoT devices

- Scanners/OCR
- APIs from other systems

Data representation

Data representation refers to how captured data is formatted and stored:

- Text, numbers, images, and binary formats
- Use of schemas to define structure
- Use of data types (e.g., INT, VARCHAR, DATE) in relational databases

A database management system must ensure data types are enforced and suitable for intended use.

1.1.4 Analysis and Indexing

Once data is stored, systems must enable fast retrieval and meaningful analysis:

Indexing creates lookup structures to improve query speed (e.g., B-trees, hash indexes).

- Search mechanisms use indexing to locate relevant records quickly.
- Full-text search and natural language processing are used for unstructured data.

Analysis involves summarising and extracting insights using:

Aggregation (SUM, COUNT, AVG)

- Filtering and sorting
- Query optimisation to ensure performance

1.1.5 Information Privacy, Integrity, and Security

Privacy

Ensures sensitive data (e.g., medical records, financial data) is not accessed by unauthorised users. Involves:

- Access control (e.g., role-based permissions)
- Data anonymisation
- Compliance with laws (e.g., GDPR, HIPAA)

Integrity

Guarantees that data is accurate and consistent over its lifecycle:

- Constraints (e.g., NOT NULL, UNIQUE, CHECK).
- Referential integrity via foreign keys.
- Transaction management (ACID properties).

Security

Protects against unauthorised access, alteration, or destruction:

- Authentication and encryption.
- Firewalls and database hardening.
- Regular backups and disaster recovery planning.

1.1.6 Scalability, Efficiency, and Effectiveness

Scalability

The ability of a system to handle increasing volumes of data or users. Achieved via:

- Vertical scaling (stronger hardware)
- Horizontal scaling (replication, sharding)

Efficiency

How well the system uses resources (CPU, memory, storage) to process operations.
Relates to:

- Query performance
- Indexing strategies
- Storage optimisation

Effectiveness:

Measures how well the system meets users' needs:

- Accuracy of search results
- System reliability and uptime
- User satisfaction

All three factors must be considered when designing or selecting an information system.

1.1.7 Conclusion

In this unit, you have explored the essential components of information storage and management. From the initial stages of data capture and representation to indexing, retrieval, and security, each process ensures that digital information is accessible,

secure, and usable. You also learned how well-designed systems are evaluated based on scalability, efficiency, and effectiveness, which are key to building robust database applications.

Summary

In this unit, you have learnt that information is stored digitally using various models and accessed via retrieval techniques; management applications like CRMs, ERPs, and library systems depend on database systems; data must be captured accurately and represented appropriately to support storage and retrieval; indexing enhances performance while privacy, integrity, and security protect data from misuse or corruption; and scalability, efficiency, and effectiveness are key performance indicators of any information system.

Self-Assessment Questions

1. What is the purpose of indexing in a database, and how does it improve performance?
2. Explain the differences between data privacy, data integrity, and data security.
3. List and describe three real-world applications of information management systems.

Tutor-Marked Assignment

1. Define the term "information storage and retrieval." Explain its relevance in managing a large-scale data system
2. Identify any two security features implemented in modern database systems and explain their role.
3. Using a hospital management system as an example, describe how information is captured, stored, and retrieved, ensuring both security and efficiency.
4. Discuss what is meant by scalability and explain one strategy used to achieve it in enterprise databases

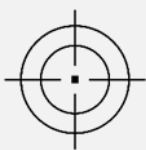
References/Further Readings

- Ramakrishnan, R., Gehrke, J.: Database Management Systems. USA: McGraw-Hill Companies (2000)
- Elmasri, R., & Navathe, S. (2000). Fundamentals of database systems (3rd ed.). Reading, Mass.: Addison-Wesley.
- Hoffer, J. A., Prescott, M. B., & McFadden, F. R. (2005). Modern database management. Upper Saddle River, N.J: Pearson/ Prentice Hall.
- Connolly, T. M., & Begg, C. E. (2002). Database systems: A practical approach to design, implementation, and management. Harlow, England: Addison-Wesley
- <http://pages.cs.wisc.edu/~dbbook/openAccess/thirdEdition/solutions/ans3ed-oddonly.pdf>

Unit 2 — Introduction to Database Systems and Architecture

INTRODUCTION

In this unit, you will learn the fundamental concepts of database systems, including what a database is, its key components, and how it supports modern information systems. You will explore important concepts such as data abstraction and data independence, which simplify database interaction and enhance flexibility. The unit also introduces the Database Management System (DBMS). Finally, you will understand the importance of database security, including mechanisms for protecting data from unauthorised access, corruption, and loss.



LEARNING OUTCOMES

At the end of this unit, you should be able to:

- Define what a database is and identify its key components.
- Explain the concepts of data abstraction and data independence in database design.
- Describe the role and functions of a DBMS.
- Differentiate between various types of database systems (e.g., hierarchical, relational, NoSQL).
- Understand the principles of database security and how data is protected.

1.2.1 Database

- A database is an extensive collection of data organised especially for rapid search and retrieval.
- Database is a shared collection of logically related data (and a description of this data), designed to meet the information needs of an organisation.
- System catalogue (metadata) describes data to enable program–data independence.

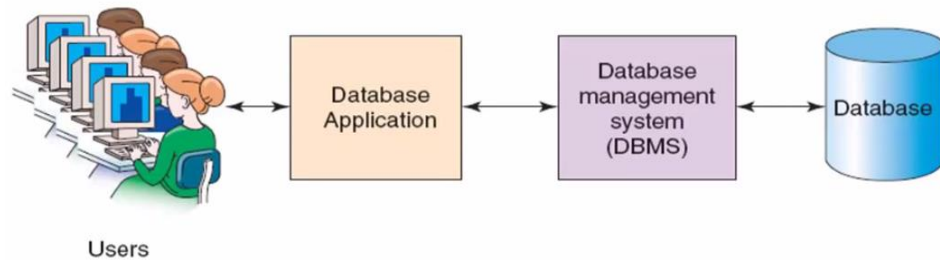


Figure 1.1: Components of a database system

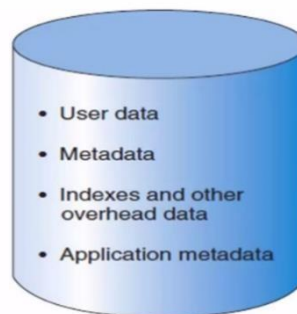


Figure 1.2: Components of the database

Applications of database technology

Storage and retrieval of “traditional” numeric and alphanumeric data

- Keeping track of number of products in stock

Storage and retrieval of multimedia data (pictures, video, sound,...)

- A popular video database for audio visual data

Storage and retrieval of Web content (HTML,PDF, images, XML)

- Google: caching to retrieve websites which are currently offline

Storage of large datasets for analysis (data warehouses)

- Collection and analysis of customer purchasing for shop floor optimization.

1.2.2 Definition of Terms

- A data model is a collection of concepts used to describe data.
- A schema is a description of a particular collection of data, using a given data model, e.g the schema for a student database
- Student {matricNo, name, Address, programme, age}
- Database instance: The information in a database at a particular point in time is called an instance of the database.
- Database Management System (DBMS): A software system that enables users to define, create, and maintain the database and that provides controlled access to this database.

1.2.3 Functions of Database Management System (DBMS)

Enforce rules

- Integrity issues

Provide security

- Authorization services

Services to promote data independence

Perform data backup- and recovery

• Components of a DBMS

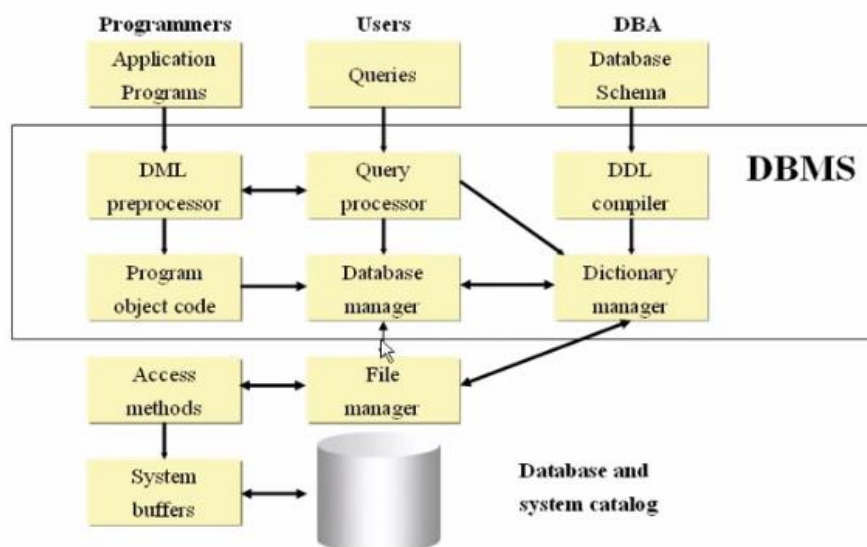


Figure 1.3: Components of DBMS

1.2.4 Data Definition Language (DDL)

- It is a descriptive language that allows DBAs or users to describe and name the entities required for the application, as well as the relationships that may exist between different entities.
- DDL statements are compiled, resulting in a set of tables stored in a special file called a data dictionary or data directory (also called system catalogue, catalogue tables, information schema in some contexts).
- The data directory contains metadata (data about data).

1.2.5 Data Manipulation Language (DML)

- Data Manipulation is a language that provides a set of operations to support the basic data manipulation operations on the data held in the database.
- A DML is a language which enables users to access and manipulate data.
- The goal is to provide efficient human interaction with the system.
- A query language is a portion of DML involving information retrieval only.
- The terms DML and query language are often used synonymously.
 - retrieval of information from the database.
 - insertion of new information into the database.
 - deletion of information in the database.
 - modification of information in the database

1.2.6 Database Applications

A database application is a set of one or more computer programs or websites that serve as an intermediary between the user and the DBMS.

1.2.7 Database Manager

The database manager is a program module which provides the interface between the low-level data stored in the database and the application programs and queries submitted to the system.

- Databases typically require lots of storage space (gigabytes).
- This must be stored on disks.
- Data is moved between disk and main memory (MM) as needed.
- The goal of the database system is to simplify and facilitate access to data

Interaction with the file manager:

- Storing raw data on disk using the file system usually provided by a conventional operating system.

The database manager must translate DML statements into low-level file system commands

- (for storing, retrieving and updating data in the database).

Integrity enforcement:

- Checking that updates in the database do not violate consistency constraints (e.g. no bank account balance below \$25)

Security enforcement:

- Ensuring that users only have access to information they are permitted to see

Backup and recovery:

- Detecting failures due to power failure, disk crash, software errors, etc., and restoring the database to its state before the failure

Concurrency control:

- Preserving data consistency when there are concurrent users.

1.2.8 Database Administrator

The database administrator is a person who has central control over data and programs that access that data.

Duties of the database administrator include:

- Schema definition:
 - the creation of the original database schema.
 - This involves writing a set of definitions in a DDL (data storage and definition language), compiled by the DDL compiler into a set of tables stored in the data dictionary.
- Granting of authorisation for data access:
 - granting different types of authorisation for data access to various users
- Integrity constraint specification:

- generating integrity constraints. These are consulted by the database manager module whenever updates occur
- Storage structure and access method definition:
 - writing a set of definitions translated by the data storage and definition language compiler
- Schema and physical organisation modification:
 - writing a set of definitions used by the DDL compiler to generate modifications to appropriate internal system tables (e.g. data dictionary).

This is done rarely, but sometimes the database schema or physical organisation must be modified.

1.2.9 Database Users



1. Naive users

They interact with the database-driven application programs without having knowledge about the back-end processes: e.g. a user interacting with the ATM.

2. Application programmers

These are computer professionals interacting with the system through DML calls embedded in a program written in a host language (e.g. C, PL/1, Pascal).

- These programs are called application programs.
- The DML pre-compiler converts DML calls (prefaced by a special character like \$, #, etc.) to standard procedure calls in a host language.
- These programs are called application programs.
- The DML pre-compiler converts DML calls (prefaced by a special character like \$, #, etc.) to standard procedure calls in a host language.
- The host language compiler then generates the object code.
- Some special types of programming languages combine Pascal-like control structures with control structures for the manipulation of a database.

- These are sometimes called fourth-generation languages.
- They often include features to help generate forms and display data

3. Sophisticated users

They interact with the system without writing programs.

- These users form requests by writing queries in a database query language.
- These are submitted to a query processor that breaks a DML statement down into instructions for the database manager module

1.2.10 Database Abstraction

Data abstraction

is the Masking of the unwanted data from users, and this happens at different levels in the database.

- These programs are called application programs.
- The DML pre-compiler converts DML calls (prefaced by a special character like \$, #, etc.) to standard procedure calls in a host language.
- Database is composed of data and records.
- What we see in rows and columns is that the actual data is quite different when it reaches the memory.
- But when they are stored in the memory like disks or tapes, they are stored in the form of bits.
- But the user only needs a little information he may want to work with about the data stored in the memory; others are not necessary.

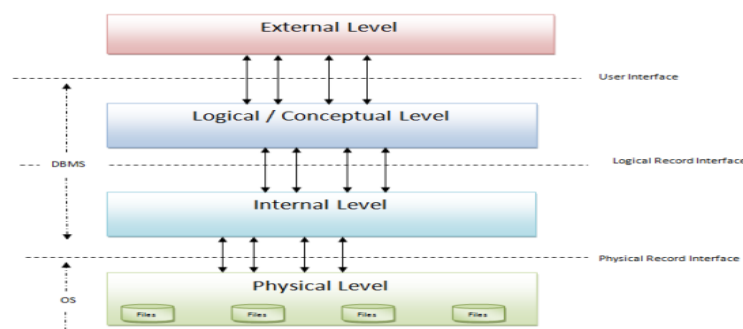


Figure 1.4: Data Abstraction Levels

Levels of Abstraction

▪ **Levels of Abstraction: External level**

- This is the highest level in data abstraction.
- At this level, users see the data in the form of rows and columns.
- This level illustrates to the users how the data is stored in terms of tables and relations.
- Users view full or partial data based on the business requirement.
- The users will have different views here, based on their levels of access rights.
- For example, a student will not have access to see the lecturer's salary details, and an employee will not have access to see other employees' details, unless they are a manager.
- At this level, one can access the data from the database and perform some calculations based on the data.
- For example, calculate the tax from the salary of an employee, calculate the CGPA of a Student, calculate the age of a person from his Date of Birth, etc. These users can be real users or any programs.
- Any changes/ computations done at this level will not affect other levels of data.
- That means, if we retrieve the few columns of the STUDENT table, it will not change the whole table, or if we calculate the CGPA of a Student, it will not change/update the table.
- This level of data is based on the levels below, but it will not alter the data at those lower levels

▪ **Levels of Abstraction: Logical/ Conceptual level**

- This is the next level of abstraction.
- It describes the actual data stored in the database in the form of tables and relates them through mapping.
- This level does not have any information on what a user views at the external level.
- This level has all the data in the database.
- Any changes made in this level will not affect the external or physical levels of data.
- That is, any changes to the table structure or the relation will not modify the data that the user is viewing at the external view or the storage at the physical level.

- For example, suppose we have added a new column 'skills', which will not modify the external view data on which the user was viewing the Ages of the students.
- Similarly, it will have space allocated for 'Skills' in the physical memory, but it will not modify the space or address of Date of Birth (using which Age will be derived) in the memory.
- Hence, external and physical independence is achieved.

▪ **Levels of Abstraction: Internal level**

- This is one of the intermediary levels.
- In most cases, this level is not mentioned, and usually, it is said that we have three levels of data abstraction.
- This level depends on the DBMS software.
- This level is how the database is seen from the DBMS.
- We can even combine the logical level and this level.

▪ **Levels of Abstraction: Physical level**

- This is the lowest level in data abstraction.
- This level describes how the data is actually stored in the physical memory, like magnetic tapes, hard disks, etc.
- In this level, the file organisation methods like hashing, sequential, and B+ tree come into the picture.
- At this level, the developer would know the requirement, size and accessing frequency of the records clearly. So, designing this level will not be too complex for him.

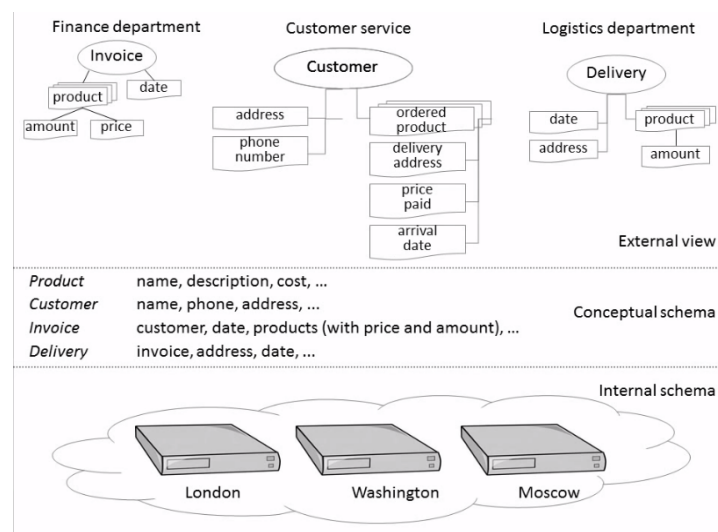


Figure 1.5: Data Abstraction Levels Example

1.2.11 Data Independence

- Data Independence is the ability to modify the schema definition at one level without affecting the schema definition at the next higher level.
- It is the type of data transparency that matters for a centralised DBMS.
- It refers to the immunity of user applications to changes made in the definition and organisation of data.
- The data independence and operation independence together give the feature of data abstraction.

Types

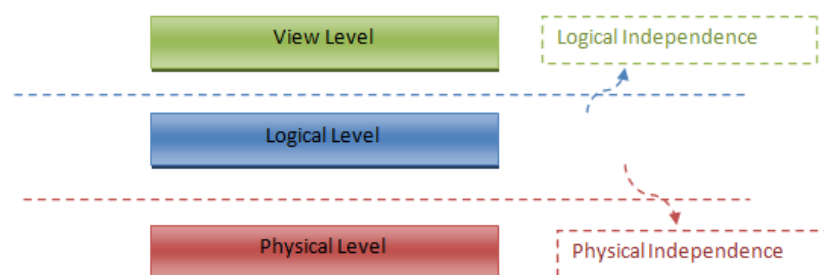


Figure 1.6: Data Independence

- **Physical Level of Data Independence**
 - Physical data independence is the ability to modify the physical schema without causing application programs to be rewritten.
 - Modifications at this level are occasionally necessary to improve performance; thus, this level deals with hiding the details of the storage structure from user applications.
 - It means the physical storage/level is changeable without affecting the conceptual or external view of the data.
 - The new changes are absorbed using mapping techniques.
- **Logical Level of Data Independence**
 - Logical data independence is the ability to modify the logical schema without causing the application program to be rewritten.
 - Logical data independence is more difficult to achieve than physical data independence, since application programs are heavily dependent on the logical structure of the data that they access.
 - Modifications at the logical level are necessary whenever the logical structure of the database is altered (for example, when money-market accounts are added to the banking system).

- For instance, if new columns are added or removed from a table, then the user view and programs should not change.
- For example, consider two users A & B. Both are selecting the fields "EmployeeNumber" and "EmployeeName".
- If user B adds a new column (e.g. salary) to his table, it will not affect the external view for user A, though the internal schema of the database has been changed for both users A & B.

Advantage of data independence

- The conceptual schema can be changed at one level without affecting the data at another level.
- The structure of a database can be changed without affecting the data required by users and programs. This feature was not available in the file-oriented approach

1.2.12 Types of Database

1. Personal Database System

- Support one application
- Have only a few tables
- Are simple in design
- Involve only one computer
- Support one user at a time

2. Enterprise-Level database systems

- Support several users simultaneously
- Support more than one application
- Involve multiple computers
- Are complex in design
- Have many tables
- Have many databases

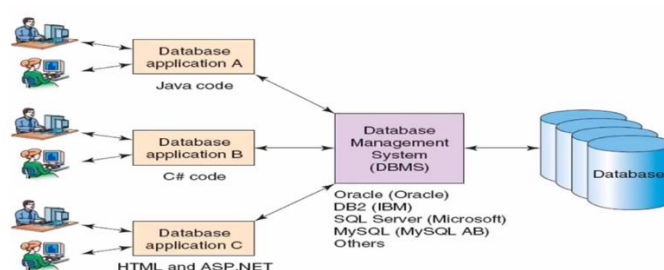


Figure 1.7: Enterprise-Level database systems

1.2.13 Database Security

Database security refers to the protective measures implemented to prevent unauthorised access, misuse, or breaches in a database system.

- It ensures data confidentiality, integrity, and availability.

Key Aspects of Database Security

Access Control:

- Restricts who can view or modify data using authentication (usernames/passwords, multi-factor authentication) and authorization (role-based access control).

Encryption:

- Protects sensitive data by converting it into an unreadable format using encryption algorithms (AES, RSA).

Auditing & Logging:

- Tracks database activities to detect suspicious behavior and maintain accountability.

Backup & Recovery:

- Regular backups ensure data is not lost due to hardware failures, cyberattacks, or human errors.

Firewalls & Network Security:

- Protects databases from external threats by implementing firewalls, intrusion detection systems, and secure connections.

SQL Injection Prevention:

- Ensures queries are sanitized to prevent attackers from manipulating SQL commands.

1.2.14 Conclusion

In this unit, you have been introduced to the foundational concepts of database systems and architecture. You learned what a database is, explored its core components, and understood the role and responsibilities of a Database Management System (DBMS). The unit also covered essential concepts such as data abstraction and

data independence, which are critical for simplifying user interaction with data and maintaining system flexibility. Furthermore, you studied various types of databases, the responsibilities of a database administrator, and how database security ensures data confidentiality, integrity, and availability. Altogether, these topics provide a comprehensive understanding of how modern database systems are structured, accessed, and managed.

Self-Assessment Questions

1. Define a database and list at least three of its key components.
2. What is data abstraction, and what are the three levels involved in it?
3. Differentiate between physical data independence and logical data independence with examples.
4. What are the primary functions of a Database Management System (DBMS)?
5. Describe two types of database systems and their typical use cases.
6. Explain the role of the database administrator and list three key responsibilities.
7. What is the purpose of a Data Manipulation Language (DML)? List four common operations

Tutor-Marked Assignment

1. What does the term data independence mean, and why is it an important goal?
2. Perform a search on the Internet of relational DBMS vendors. Pick two competing products and discuss whether each is scalable and if it can be used for a personal, workshop, departmental, enterprise, or Internet database.
3. One of the biggest challenges of building e-commerce sites has been the ability to deliver merchandise ordered by customers over the web quickly. Why do you think companies turned to database solutions to help them improve their supply chain management and expedite the filling and delivery of orders?
4. Explain the difference between user views, a conceptual schema, and an internal schema as different perspectives of the same database

References/Further Readings

Ramakrishnan, R., Gehrke, J.: Database Management Systems. USA: McGraw-Hill Companies (2000)

Elmasri, R., & Navathe, S. (2000). Fundamentals of database systems (3rd ed.). Reading, Mass.: Addison-Wesley.

Hoffer, J. A., Prescott, M. B., & McFadden, F. R. (2005). Modern database management. Upper Saddle River, N.J: Pearson/ Prentice Hall.

Connolly, T. M., & Begg, C. E. (2002). Database systems: A practical approach to design, implementation, and management. Harlow, England: Addison-Wesley

<http://pages.cs.wisc.edu/~dbbook/openAccess/thirdEdition/solutions/ans3ed-oddonly.pdf>

<https://gyires.inf.unideb.hu/GyBITT/03/>

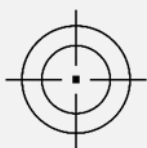
Module Two

Introduction to Conceptual Models

Unit 1 — Introduction to Conceptual Models: ER Diagram

INTRODUCTION

In this unit, you will learn about the fundamentals of data modelling, the use of conceptual models (Entity-Relationship Model), and how business rules shape data structures. You will explore how to model entities and attributes, define relationships among entities, and understand the role of cardinality constraints in capturing real-world associations. All of these concepts are essential and useful for designing a well-structured, efficient database system.



LEARNING OUTCOMES

At the end of this unit, you should be able to:

- Understand the basic concept and purpose of data and conceptual modelling.
- Explain the structure and use of the Entity-Relationship (ER) model.
- Identify and define business rules that influence data models.
- Model entities and their attributes in an ER diagram.
- Model relationships between entities using appropriate notation.
- Understand and apply cardinality constraints to describe relationship rules accurately.

2.1.1 Data Modelling

A conceptual model cannot be adequately understood without introducing its basics (data modelling). Data modelling is the documentation of rules and policies that determine how an organisation's data is governed.

The Role of the Database Analyst in Data Modelling

In information system development, a database analyst performs the following duties:

- Identify and understand those rules that govern data
- Represent those rules so that they can be unambiguously understood by information system developers and users
- Implement those rules in database technology. It is a branch of mathematics that deals with the verification of the truth/falsity of mathematical statements/assertions

It is the study of valid arguments, inference (induction + deduction), proofs, proof techniques, proving theorems, paradoxes, and fallacies

"Logic will get you from A to B. Imagination will take you everywhere." A. Einstein

Importance of Data Modelling- System Developers' View

1. The characteristics of data captured during data modelling are crucial in the design of databases, programs, and other system components.
2. Data, rather than processes, are the most complex aspect of many modern information systems and hence require a central role in structuring system requirements.
3. Data tends to be more stable than the business processes that use that data

Benefits of Data Modelling

Data modelling:

- Facilitates communication between the designer, the application programmer, and the end user.
- Fosters understanding of the organisation (rules) for which the data model is being developed.
- Saves maintenance or development costs.
- Results in improved data quality

- Reduces the high costs of moving and translating data from one system to another

2.1.2 Conceptual Model

In general, conceptual models are high-level representations of an organisation's data and the relationships between different data elements.

The Entity Relationship Model

- The E-R model is a detailed conceptual representation of entities in the business environment, the relationships (or associations) among them, and the attributes (or properties) of both entities and their relationships.
- An E-R model is usually expressed as an entity-relationship diagram (E-R diagram, or ERD).
- The E-R model is a detailed conceptual representation of entities in the business environment, the relationships among those entities, and the attributes of both the entities and their relationships

The Entity-Relationship Diagram - ERD

An ERD is a graphical representation of an entity-relationship model.

It is a representation of the structure and constraints of a database that is independent of the database management system.

Sample E-R Diagram

Based on the E-R diagram in the figure:

- Several suppliers supply and ship different items to Pine Valley Furniture.
- The items are assembled into products that are sold to customers who order the products.
- Each customer order may include one or more lines corresponding to the products appearing on that order.

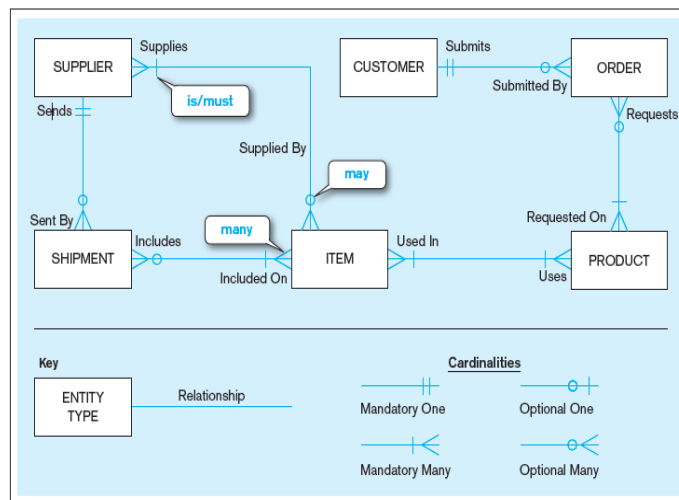


Figure 2.1: E-R Diagram

Meta-Data of Entities

In conceptual modelling, entity metadata refers to descriptive details such as entity names, definitions, attributes, and identifiers that clarify the structure and meaning of each entity in an ER diagram.

- It is essential to define each entity as metadata clearly. The metadata of the E-R diagram in Figure 2.1 above is shown in Table 2.1 below.

CUSTOMER	A person or an organization that has ordered or might order products. <i>Example:</i> L. L. Fish Furniture.
PRODUCT	A type of furniture made by Pine Valley Furniture that may be ordered by customers. Note that a product is not a specific bookcase, because individual bookcases do not need to be tracked. <i>Example:</i> A 6-foot, 5-shelf, oak bookcase called O600.
ORDER	The transaction associated with the sale of one or more products to a customer and identified by a transaction number from sales or accounting. <i>Example:</i> The event of L. L. Fish buying one product O600 and four products O623 on September 10, 2015.
ITEM	A type of component that goes into making one or more products and can be supplied by one or more suppliers. <i>Example:</i> A 4-inch ball-bearing caster called I-27-4375.
SUPPLIER	Another company that may provide items to Pine Valley Furniture. <i>Example:</i> Sure Fasteners, Inc.
SHIPMENT	The transaction associated with items received in the same package by Pine Valley Furniture from a supplier. All items in a shipment appear on one bill-of-lading document. <i>Example:</i> The receipt of 300 I-27-4375 and 200 I-27-4380 items from Sure Fasteners, Inc., on September 9, 2015.

Table 2.1: Meta-Data of E-R Diagram

Data Names

In an organisational data model, data objects must be named and defined before they can be used unambiguously.

General guidelines about naming any data object should:

- Relate to business,
- Not technical characteristics
- Be meaningful

- Be unique
- Be repeatable
- Follow a standard syntax
- Be composed of words taken from an approved list

2.1.3 Business Rule

Overview of Business Rule

- Business rules are the foundation of data models and are derived from policies, procedures, events, functions, and other business objects, and they state constraints on the organisation.
- Business rules represent the language and fundamental structure of an or

Business rules formalize the understanding of the organisation by organisation owners, managers, and leaders with that of information systems architects.

Business rules are important in data modelling because they govern how data are handled and stored.

Business Rule

- It is intended to assert business structure or to control or influence the behaviour of the business.
- Business rules and policies govern the creation, updating, and removal of data in an information processing and storage system.
- Hence, they must be described along with the data to which they are related.
- Business rules and policies are not universal.
- A business rule is also known as an Integrity constraint in the database context.

Business rules appear (possibly implicitly) in descriptions of business functions, events, policies, units, stakeholders, and other objects

Gathering Business Rules?

These descriptions can be found in interview notes from individual and group information systems requirements collection sessions, organisational documents (e.g., personnel manuals, policies, contracts, marketing brochures, and technical instructions), and other sources.

- Rules are identified by asking questions about the who, what, when, where, why, and how of the organisation.
- A data analyst has to be persistent in clarifying initial statements of rules because initial statements may be vague or imprecise.
- Precise rules are formulated from an iterative inquiry process. A truth set of a predicate is the set of all values of the predicate that makes the predicate true.

You should be prepared to ask such questions as:

1. Is this always true?
2. Are there special circumstances when an alternative occurs?
3. Are there distinct kinds of that person?
4. Is there only one of those, or are there many? and
5. Is there a need to keep a history of those, or is the current data all that is useful?

Characteristics of a Good Business Rule

A good business rule is:

1. Declarative
 - statement of policy, what a process validates.
2. Precise
 - have only one interpretation
3. Atomic
 - no part of the rule can stand on its own
4. Consistent
 - Not conflicting
5. Expressible
 - Natural language, understanding
6. Distinct
 - not be redundant but can refer to other rules
7. Business-oriented

- State business terms

2.1.4 Modelling Entities and Attributes

E-R Model Notation

There is no industry-standard notation

- This course combines most of the desirable features of the different notations that are commonly used in E-R drawing tools to model accurately most situations that are encountered in practice.
- Additional notation for enhanced entity-relationship models can also be introduced.

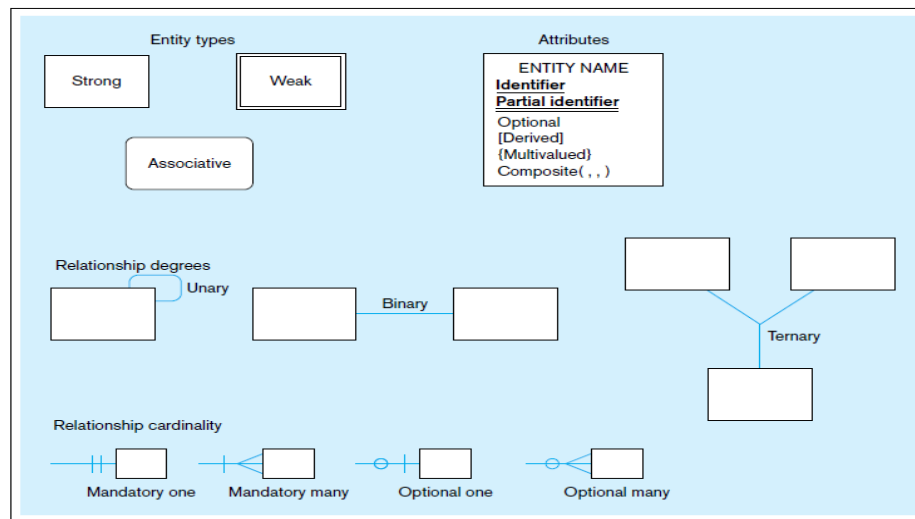


Figure 2.2: Basic E-R Notations

Entity

An entity is a person, place, object, event, or concept in the user environment about which the organisation wishes to maintain data.

- An entity has a noun name.
- Examples of an entity could be:
 - Person: EMPLOYEE, STUDENT, PATIENT
 - Place: STORE, WAREHOUSE, STATE
 - Object: MACHINE, BUILDING, AUTOMOBILE
 - Event: SALE, REGISTRATION, RENEWAL
 - Concept: ACCOUNT, COURSE, WORK CENTER

Entity Type Versus Entity Instance

An entity type is a collection of entities that share common properties or characteristics. Each entity type in an E-R model is given a name.

An entity instance is a single occurrence of an entity type. An entity type is described just once (using metadata) in a database, whereas many instances of that entity type may be represented by data stored in the database.

There is one EMPLOYEE entity type in most organisations. Still, there may be hundreds (or even thousands) of instances of this entity type stored in the database, as seen in Figure 2.3 below.

Entity type: EMPLOYEE			
Attributes	Attribute Data Type	Example Instance	Example Instance
Employee Number	CHAR (10)	642-17-8360	534-10-1971
Name	CHAR (25)	Michelle Brady	David Johnson
Address	CHAR (30)	100 Pacific Avenue	450 Redwood Drive
City	CHAR (20)	San Francisco	Redwood City
State	CHAR (2)	CA	CA
Zip Code	CHAR (9)	98173	97142
Date Hired	DATE	03-21-1992	08-16-1994
Birth Date	DATE	06-19-1968	09-04-1975

Figure 2.3: Entity Type Versus Entity Instance

Entity Type Versus Entity Instance

▪ **Strong Entity**

- A strong entity type is one that exists independently of other entity types.
- Examples include *STUDENT*, *EMPLOYEE*, *AUTOMOBILE*, and *COURSE*.
- Instances of a strong entity type always have a unique characteristic (*called an identifier*)—
- that is, an attribute or a combination of attributes that uniquely distinguishes each occurrence of that entity.
- *It can also be referred to as an independent entity.*

▪ **Weak Entity**

- A weak entity type is an entity type whose existence depends on some other entity type.
- *It is also referred to as a dependent entity.*
- A weak entity type has no business meaning in an E-R diagram without the entity on which it depends
- The entity types on which the weak entity type depends are called the *identifying owner* (or simply *owner for short*).
- A weak entity type does not typically have its own identifier.

- Generally, on an E-R diagram, a weak entity type has an attribute that serves as a *partial identifier*.

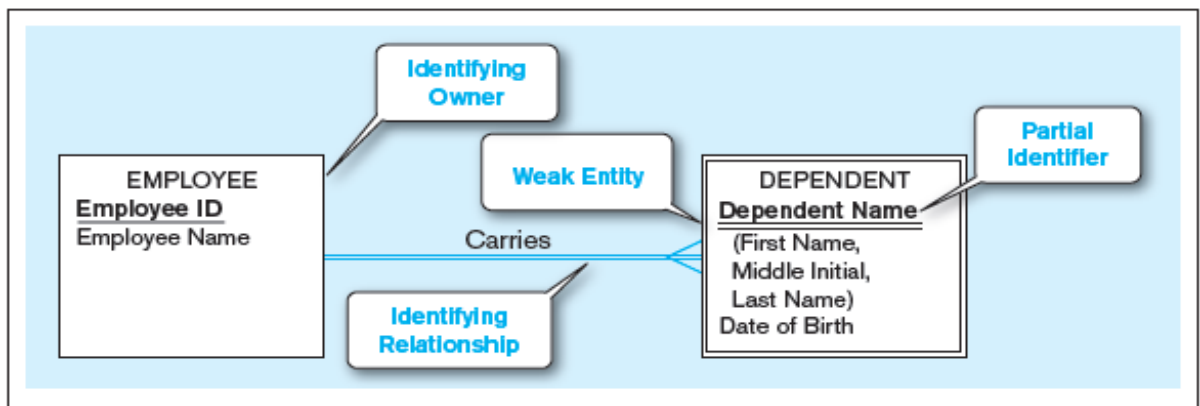


Figure 2.4: Strong, Weak Entity and Identifying Relationship Example

Attributes

An attribute is a property or characteristic of an entity type that is of interest to the organisation.

- An attribute has a noun name.
Examples of some typical entity types and their associated attributes:
- Student ID, Student Name, Home Address, Phone Number, Major
- AUTOMOBILE Vehicle ID, Colour, Weight, Horsepower
- Employee ID, Employee Name, Payroll, Address, Skill.

Required Versus Optional Attributes

- An attribute that must be present for each entity instance is called a required attribute.
- An attribute that may not have a value is called an optional attribute.
- Figure 2.5 shows two STUDENT entities (instances) with their respective attribute values.
- The only optional attribute for STUDENT is Major#

Entity type: STUDENT				
Attributes	Attribute Data Type	Required or Optional	Example Instance	Example Instance
Student ID	CHAR (10)	Required	876-24-8217	822-24-4456
Student Name	CHAR (40)	Required	Michael Grant	Melissa Kraft
Home Address	CHAR (30)	Required	314 Baker St.	1422 Heft Ave
Home City	CHAR (20)	Required	Centerville	Miami
Home State	CHAR (2)	Required	OH	FL
Home Zip Code	CHAR (9)	Required	45459	33321
Major	CHAR (3)	Optional	MIS	

Figure 2.5: Required and Optional Attribute(s)

Simple Versus Composite Attributes

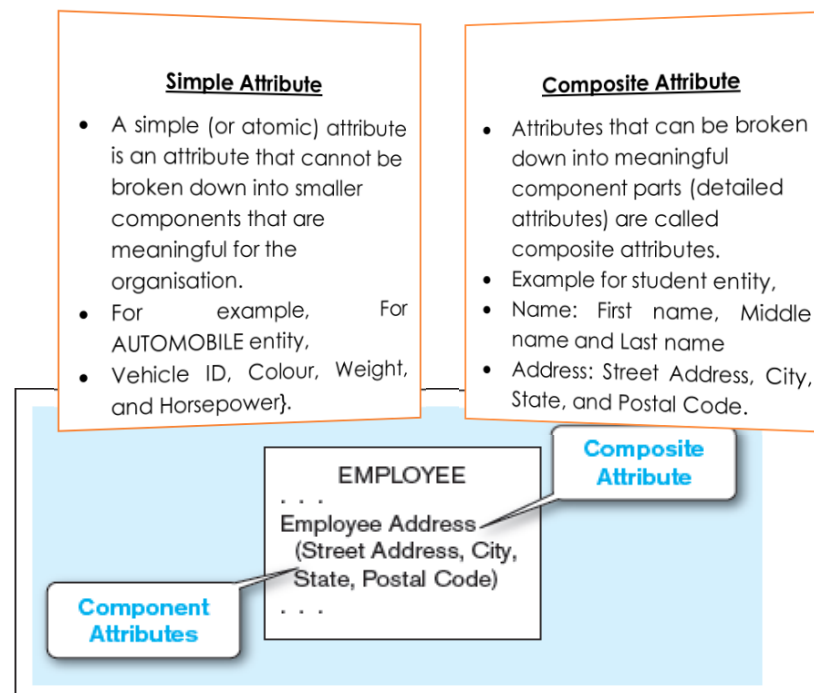


Figure 2.6: Composite Attributes

Multi-Valued Attributes

Identifiers consist of letters and digits, in any order, except that the first character must be a letter.

- A multivalued attribute is an attribute that may take on more than one value for a given entity (or relationship) instance.
- For example, the EMPLOYEE entity type in Figure 2.7 has an attribute named Skill, whose values record the skill (or skills) for that employee.

- Some employees may have more than one skill, such as a PHP Programmer and C++ Programmer
- *We can indicate a multivalued attribute with curly brackets around the attribute.*

Derived Attributes

- A derived attribute is an attribute whose values can be calculated from related attribute values (plus possibly data not in the database, such as today's date, the current time, or a security code provided by a system user).
- A derived attribute is denoted by using square brackets around the attribute name.

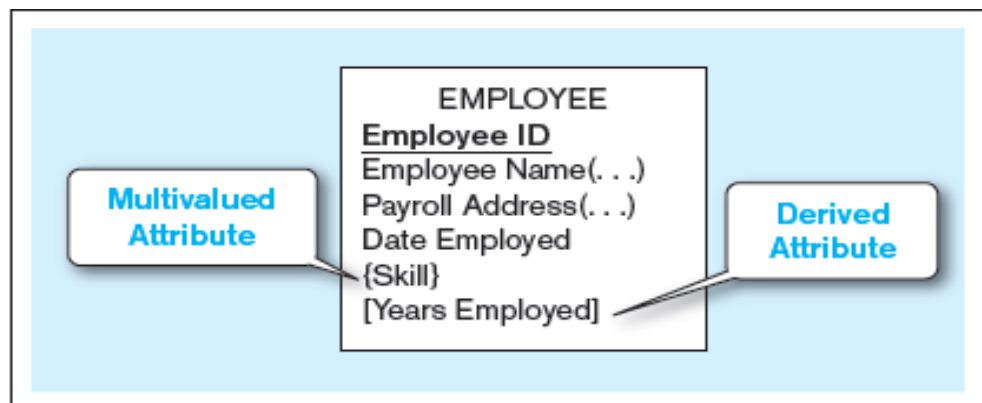


Figure 2.7: Multivalued and Derived Attributes

Identifier Attributes

- An identifier is an attribute (or combination of attributes) whose value distinguishes individual instances of an entity type.
- No two instances of an entity type may have the same value for the identifier attribute.
- For some entity types, there is no single (or atomic) attribute that can serve as the identifier (i.e., that will ensure uniqueness).
- However, two (or more) attributes used in combination may serve as the identifier.
- A composite identifier is an identifier that consists of a composite attribute.

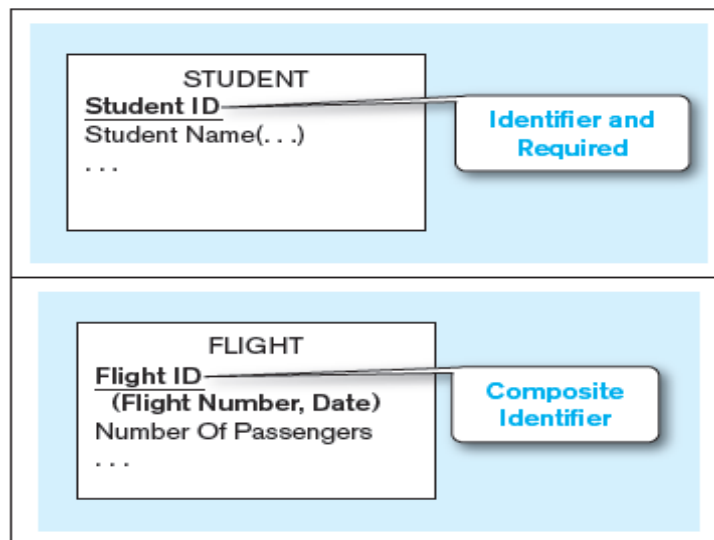


Figure 2.8: Simple & Composite Identifier Attributes

Relationships

- A relationship is an association representing an interaction among the instances of one or more entity types that is of interest to the organisation.
 - A relationship has a verb phrase name.
- Relationships and their characteristics (degree and cardinality) represent business rules, and usually represent the most complex business rules shown in an ERD.

Relationship Type

- A Relationship type is a meaningful association between (or among) entity types.
- A relationship type is denoted by a line labelled with the name of the relationship.
- Short, descriptive verb phrase that is meaningful to the user is used to name a relationship.

Relationship Instance

- A relationship instance is an association between (or among) entity instances, where each relationship instance associates exactly one entity instance from each participating entity type.

Relationship Type and Instances Example

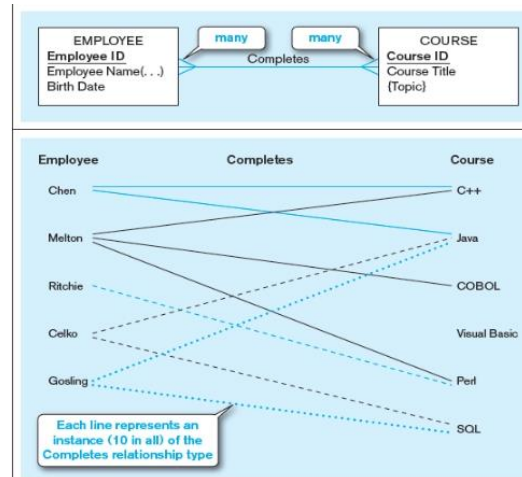


Figure 2.9: Relationship Type and Instances

- To track courses that have been completed by particular employees, we define a relationship called Completes between the two entity types
- This is a many-to-many relationship
- The employee Melton has completed three courses (C++, COBOL, and Perl).
- The SQL course has been completed by two employees (Celko and Gosling), and
- The Visual Basic course has not been completed by anyone.

Attribute on Relationship

- Attributes may be associated with a many-to-many or one-to-one relationship,
- For example, suppose the organisation wishes to record the date (month and year) when an employee completes each course.
- This attribute is named Date Completed
- It is interesting to note that an attribute cannot be associated with a one-to-many relationship.

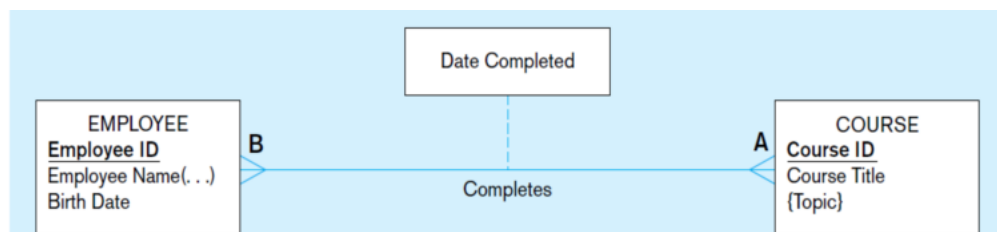


Figure 2.10: Attribute on Relationship

Associative Entity

- An associative entity is an entity type that associates the instances of one or more entity types and contains attributes that are peculiar to the relationship between those entity instances.
- The associative entity CERTIFICATE is represented with the rectangle with rounded corners, as shown below.

(b) An associative entity (CERTIFICATE)



Figure 2.11: Associative Entity

Degree of Relationships

- The degree of a relationship is the number of entity types that participate in that relationship.
- The three most common relationship degrees in E-R models are unary (degree 1), binary (degree 2), and ternary (degree 3).
- Higher-degree relationships are possible, but they are rarely encountered in practice, so we restrict our discussion to these three cases.

Degree of Relationships: Unary Relationship

- A unary relationship is a relationship between the instances of a single entity type.
- *Unary relationships are also called recursive relationships.*

(a) Unary relationships

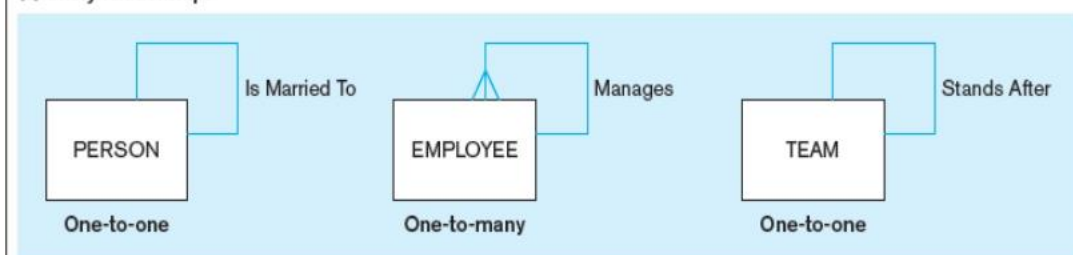


Figure 2.12: Unary Relationship

Degree of Relationships: Binary Relationships

- A binary relationship is a relationship between the instances of two entity types and is the most common type of relationship encountered in data modelling.

(b) Binary relationships

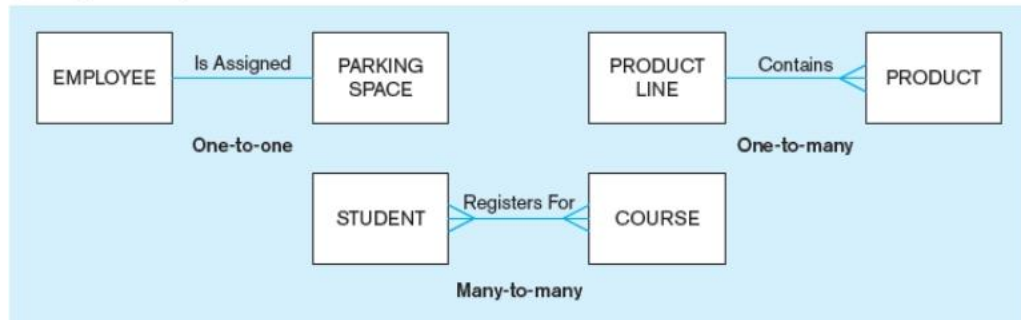


Figure 2.13: Binary Relationships

Degree of Relationships: Ternary Relationship

- A ternary relationship is a simultaneous relationship among the instances of three entity types.

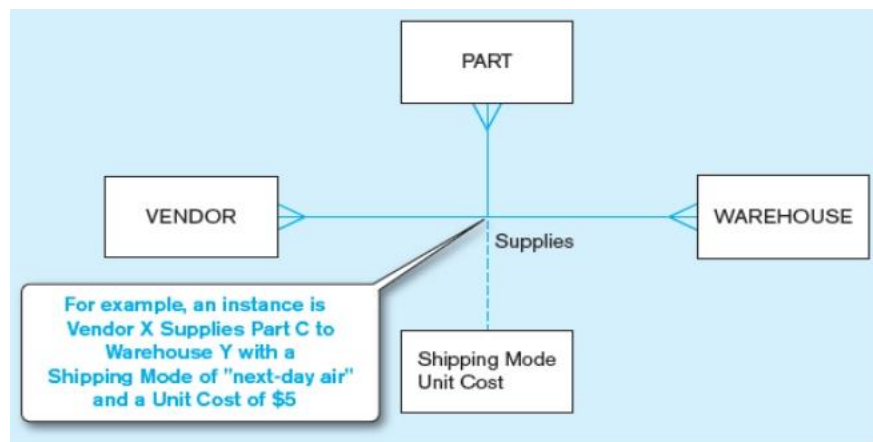


Figure 2.14: Ternary Relationship

2.1.5 Cardinality Constraints

- A rule that specifies the number of instances of one entity that can (or must) be associated with each instance of another entity.
- A cardinality constraint specifies the number of instances of entity B that can (or must) be associated with each instance of entity A. Predicates represent properties or relations among objects. A predicate $P(x)$ assigns a value true or false to each x depend

Minimum and Maximum Cardinality

Minimum Cardinality

- The minimum number of instances of one entity that may be associated with each instance of another entity.

- When the minimum number of participants is zero, we say that the entity type participation is optional in the relationship; otherwise, it is mandatory.

Maximum Cardinality

- The maximum number of instances of one entity that may be associated with each instance of another entity.
- It is often indicated by the “crow’s foot” symbol.

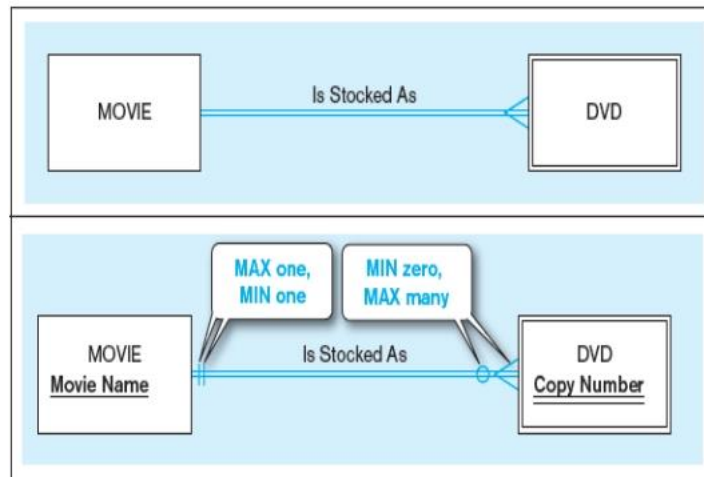


Figure 2.15: Cardinality Constraint

In Figure 2.15 above:

The relationship is bidirectional; a cardinality notation appears next to the MOVIE entity.

Each DVD of a movie must be a copy of exactly one movie.

Participation in a relationship may be optional or mandatory for the entities involved.

If the minimum cardinality is zero, participation is optional; if the minimum cardinality is one, participation is mandatory.

Example on Relationship and Cardinality

1. PATIENT has Recorded PATIENT HISTORY:

- Each patient has one or more patient histories. (The initial patient visit is always recorded as an instance of PATIENT HISTORY.)
- Each instance of PATIENT HISTORY “belongs to” exactly one PATIENT as seen below

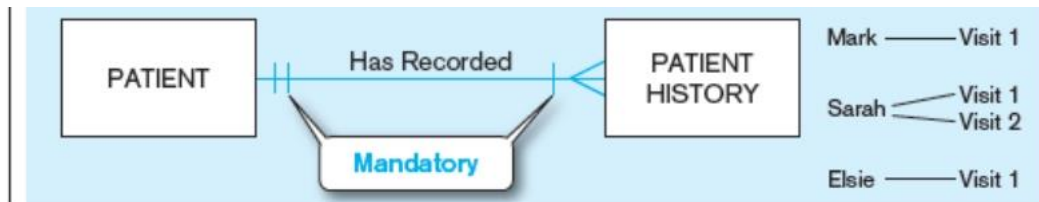


Figure 2.16: PATIENT Has Recorded PATIENT HISTORY

2. EMPLOYEE is Assigned to PROJECT:

- Each PROJECT has at least one EMPLOYEE assigned to it. (Some projects have more than one.)
- Each EMPLOYEE may or (optionally) may not be assigned to any existing PROJECT (e.g., employee Pete) or may be assigned to one or more PROJECTS.

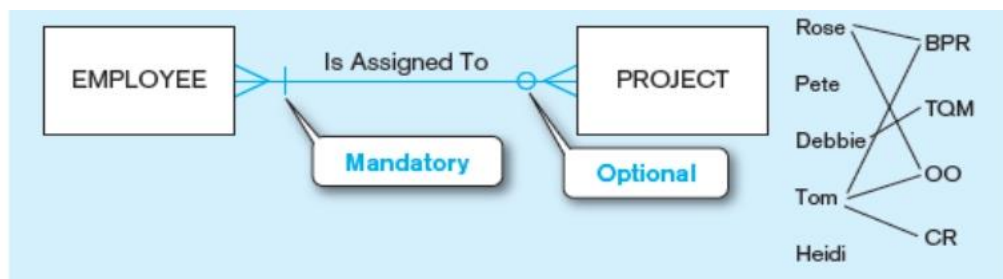


Figure 2.17: EMPLOYEE Is Assigned To PROJECT

3. PERSON is Married to PERSON:

- This is an optional zero or one cardinality in both directions, because a person may or may not be married at a given point in time.

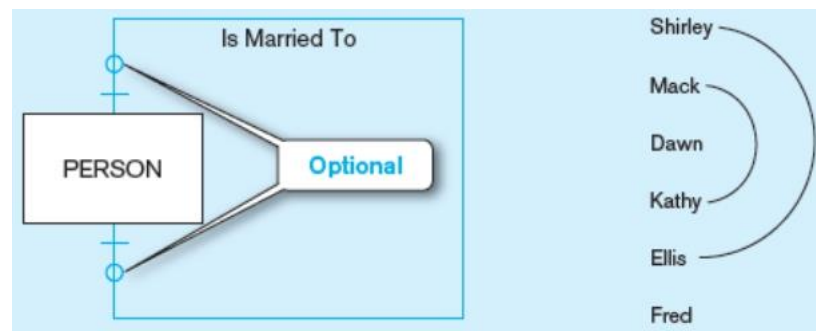


Figure 2.18: PERSON Is Married to PERSON

2.1.6 Example of a Business Rule for the ERD

- A SUPPLIER may supply many ITEMS (by “may supply,” we mean the supplier may not supply any items). Each ITEM is supplied by any number of SUPPLIERS (by “is supplied,” we mean that the ITEM must be supplied by at least one supplier).

- Each ITEM must be used in the assembly of at least one PRODUCT and may be used in many products. Conversely, each PRODUCT must use at least one ITEM.
- A SUPPLIER may send many SHIPMENTS. However, each shipment must be sent by exactly one SUPPLIER. Notice that sends and supplies are separate concepts. A SUPPLIER may be able to supply an item but may not yet have sent any Shipments of that item.
- A SHIPMENT must include at least one ITEM. An ITEM may be included on several SHIPMENTS.
- A CUSTOMER may submit any number of ORDERS. However, each ORDER must be submitted by exactly one CUSTOMER. Given that a CUSTOMER may not have submitted any ORDERS, some CUSTOMERS may be potential, inactive, or otherwise have no related ORDERS.
- An ORDER must request one (or more) PRODUCTS. A given PRODUCT may not be requested on any ORDER or may be requested on one or more orders

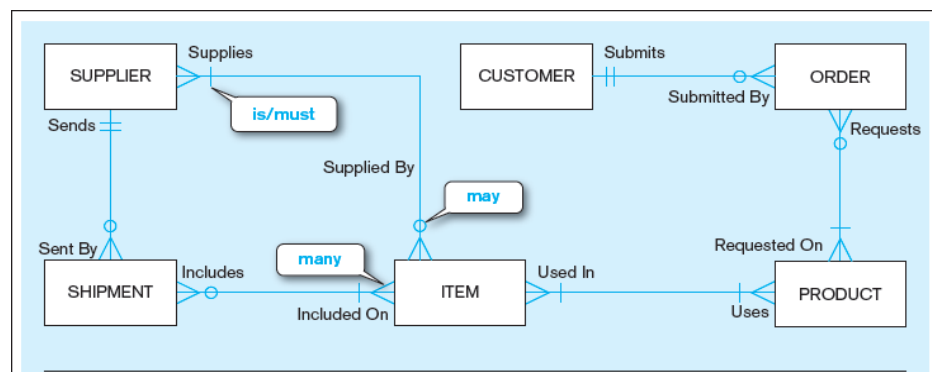


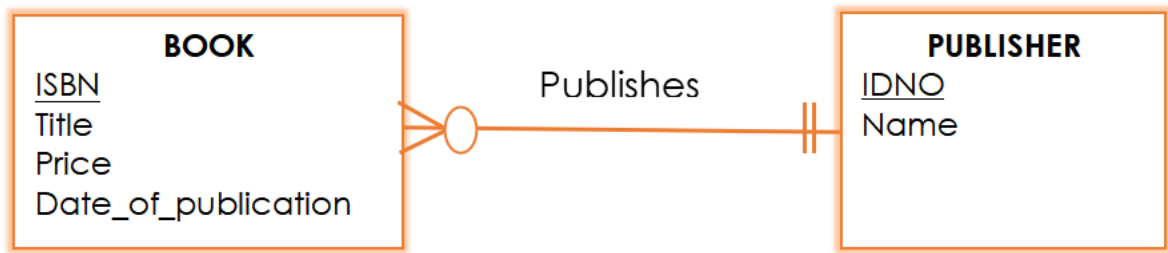
Figure 2.19: Example of a Business Rule for the ERD

2.1.7 Class Activities

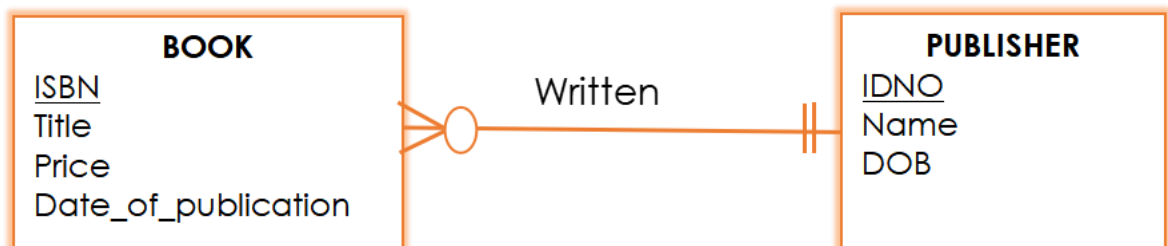
For each of the descriptions below, perform the following tasks:

- Identify the degree and cardinalities of the relationship.
- Express the relationships in each description graphically with an E-R diagram.

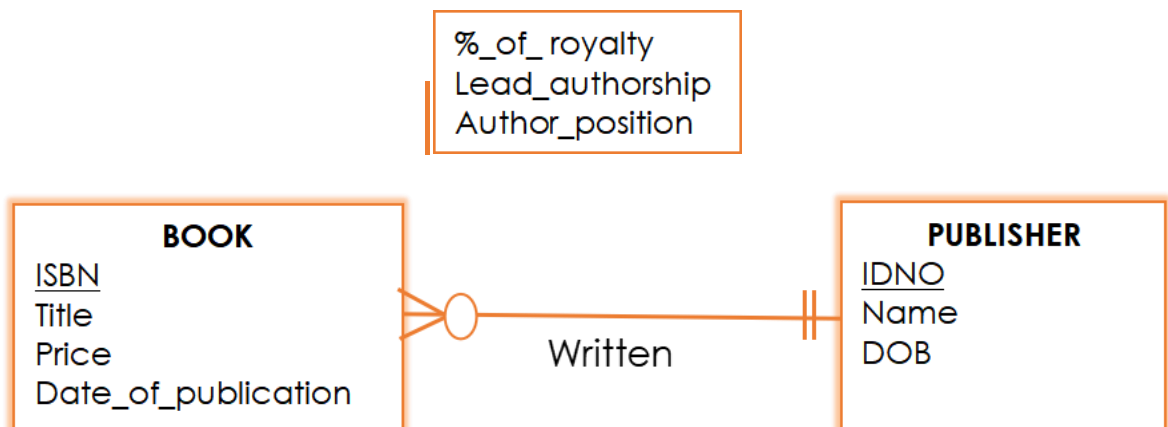
1a. A book is identified by its ISBN, and it has a title, a price, and a date of publication. It is published by a publisher that has its own ID number and name. Each book has exactly one publisher, but one publisher typically publishes multiple books over time.



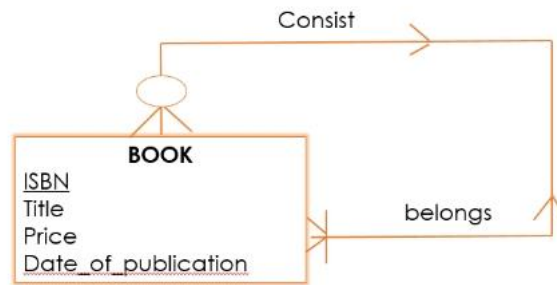
1b. A book (see 1a) is written by one or multiple authors. Each author is identified by an author number and has a name and date of birth. Each author has either one or multiple books; in addition, data are occasionally needed on prospective authors who have not yet published any books.



In the context specified in 1a and 1b, better information is needed regarding the relationship between a book and its authors. Specifically, it is essential to record the percentage of the royalties that belongs to a specific author, whether or not a specific author is a lead author of the book, and each author's position in the sequence of the book's authors.



A book (see 1a) can be part of a series, which is also identified as a book and has its own ISBN. One book can belong to several sets, and a set consists of at least one but potentially many books.



2. Look at this E-R Diagram:

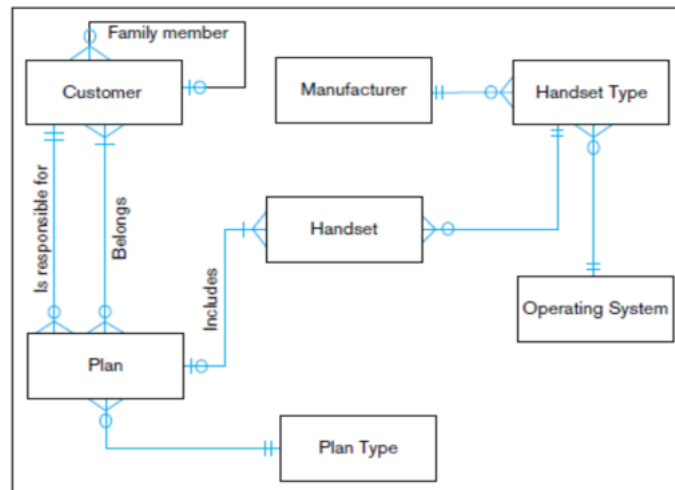


Figure 2.20: Class Activity E-R Diagram

A cellular operator needs a database to track its customers, their subscription plans, and the handsets (mobile phones) they use. The E-R diagram in Figure 2.20 illustrates the key entities of interest to the operator and their relationships. Based on the figure, answer the following questions and explain the rationale for your response. For each question, identify the element(s) in the E-R diagram that you used to determine your answer.

- Can a customer have an unlimited number of plans?
- Can a customer exist without a plan?
- Is it possible to create a plan without knowing who the customer is?
- Does the operator want to limit the types of handsets that can be linked to a specific plan type?
- Is it possible to maintain data regarding a handset without connecting it to a plan?
- Can a handset be associated with multiple plans?
- Assume a handset type exists that can utilise multiple operating systems. Could this situation be accommodated within the model included in Figure 2-1?

- h. Is the company able to track a manufacturer without maintaining information about its handsets?
- i. Can the same operating system be used on multiple handset types?

2.1.8 Conclusion and Summary

In this unit, you have learnt about data modelling, the use of a conceptual model (Entity-Relationship Model) and how business rules shape data structures. You have explored how to model entities and attributes, define relationships among entities, and understand the role of cardinality constraints in capturing real-world associations. All of these concepts are essential and useful for designing a well-structured, efficient database system.

Self-Assessment Questions

1. Is there a difference between data modelling and conceptual modelling? Justify your answer.
2. What determines how data are handled?
3. Relationship instance and Associative entity are similar but different. Explain their similarities and differences

Unit Activities

1. Read the content of Module 2, Unit 1 in the study guide. Consult the recommended texts and attempt the class activity and self-assessment questions. (90 minutes)
2. TUTOR MARKED ASSIGNMENT: Complete the following and submit for grading. (60 minutes)

Tutor-Marked Assignment

1. Give four reasons why many system designers believe that data modelling is important and arguably the most essential part of the systems development process.
2. Discuss why the ER model is a popular modelling tool.

3. Consider the situation: Faculty at a university (FACULTY entity) can also be part of the Board of Studies (BOARD entity). Is there a weak entity here? How?
4. The entity type TEACHER has the following attributes:
 Teacher Name, Email, Phone, Age, Trainer, and Training Count.
 Trainer represents a workshop organised by the teacher, and Training Count represents the number of times the teacher has organised such workshops. This implies a teacher may organise more than one workshop. Draw an ERD for this situation.
 What attribute or attributes did you designate as the identifier for the TEACHER entity? Why?

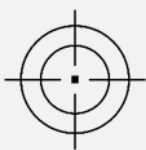
References/Further Readings

- Ramakrishnan, R., Gehrke, J.: Database Management Systems. USA: McGraw-Hill Companies (2000)
- Elmasri, R., & Navathe, S. (2000). Fundamentals of database systems (3rd ed.). Reading, Mass.: Addison-Wesley.
- Hoffer, J. A., Prescott, M. B., & McFadden, F. R. (2005). Modern database management. Upper Saddle River, N.J: Pearson/ Prentice Hall.
- Connolly, T. M., & Begg, C. E. (2002). Database systems: A practical approach to design, implementation, and management. Harlow, England: Addison-Wesley
- <http://pages.cs.wisc.edu/~dbbook/openAccess/thirdEdition/solutions/ans3ed-oddonly.pdf>
- <https://gyires.inf.unideb.hu/GyBITT/03/>

Unit 2 — Enhanced E-R Model

INTRODUCTION

In this unit, you will learn about Enhanced Entity-Relationship (E-R) modelling, an extension of the basic E-R model that provides greater expressiveness for representing real-world data. Enhanced E-R models introduce concepts such as supertypes and subtypes, which allow entities to be grouped into more general or more specific categories. You will also explore the processes of specialisation and generalisation, which help organise entities based on shared or distinct characteristics. In addition, you will learn the constraints in subtype/supertype relationships, subtype discriminators and supertype/subtype hierarchies.



LEARNING OUTCOMES

At the end of this unit, you should be able to:

- Understand the concept of Enhanced E-R modelling and its purpose in advanced database design.
- Identify and explain the use of supertypes and subtypes in representing entity hierarchies.
- Distinguish between specialisation and generalisation in modelling entity relationships.
- Apply completeness and disjointness constraints in subtype/supertype relationships.
- Understand the role of subtype discriminators in determining entity categorisation.
- Interpret and construct supertype/subtype hierarchies in an E-R diagram

2.2.1 Enhanced Entity-Relationship (EER) Model

Enhanced entity-relationship (EER) is used to identify the model that resulted from extending the original E-R model with new modelling constructs. These extensions make the EER model semantically similar to object-oriented data modelling.

- The most important modelling construct incorporated in the EER model is supertype/subtype relationships.
 - This facility enables the modelling of a general entity type (called the supertype) and then subdivides it into several specialised entity types (called subtypes)
 - Thus, for example, the entity type CAR can be modelled as a supertype, with subtypes SEDAN, SPORTS CAR, COUPE, and so on.
 - Each subtype inherits attributes from its supertype and, in addition, may have special attributes and be involved in relationships of its own.
 - Adding new notation for modelling supertype/subtype relationships has greatly improved the flexibility of the basic E-R model.

2.2.2 Supertypes and Subtypes

One of the significant challenges in data modelling is to recognise and clearly represent entities that are almost the same, that is, entity types that share common properties but also have one or more distinct properties of interest to the organisation.

A subtype

A subgrouping of the entities in an entity type that is meaningful to the organisation.

For example, STUDENT is an entity type in a university. Two subtypes of STUDENT are GRADUATE STUDENT and UNDERGRADUATE STUDENT.

In this example, we refer to STUDENT as the supertype.

A supertype

A generic entity type that has a relationship with one or more subtypes.

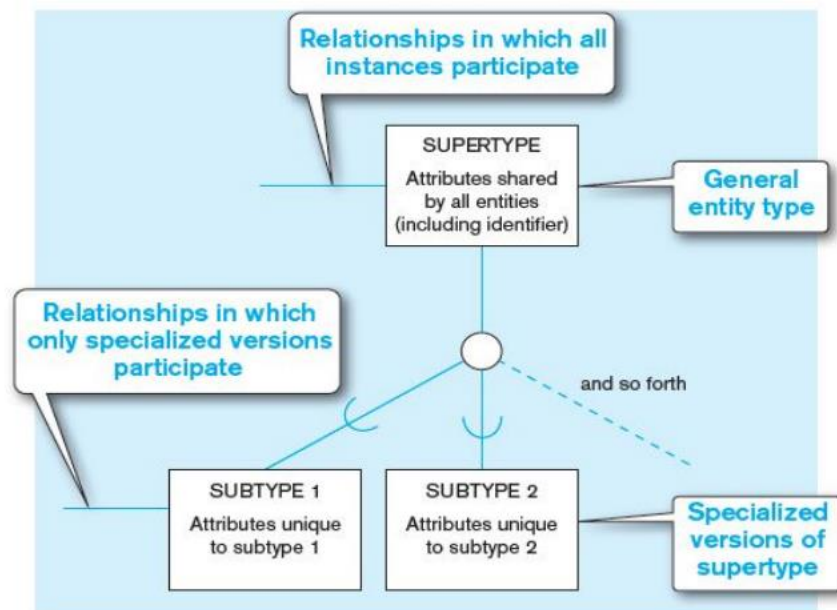


Figure 2.21 Basic Supertypes and Subtypes Notations

An Example of Supertypes and Subtypes

- Suppose that an organisation has three basic types of employees: hourly employees, salaried employees, and contract consultants.
- The following are some of the important attributes for each of these types of employees:
 - Hourly employees: Employee Number, Employee Name, Address, Date Hired, Hourly Rate.
 - Salaried employees: Employee Number, Employee Name, Address, Date Hired, Annual Salary, Stock Option.
 - Contract consultants: Employee Number, Employee Name, Address, Date Hired, Contract Number, Billing Rate.
 - Notice that all of the employee types have several attributes in common: Employee Number, Employee Name, Address, and Date Hired.

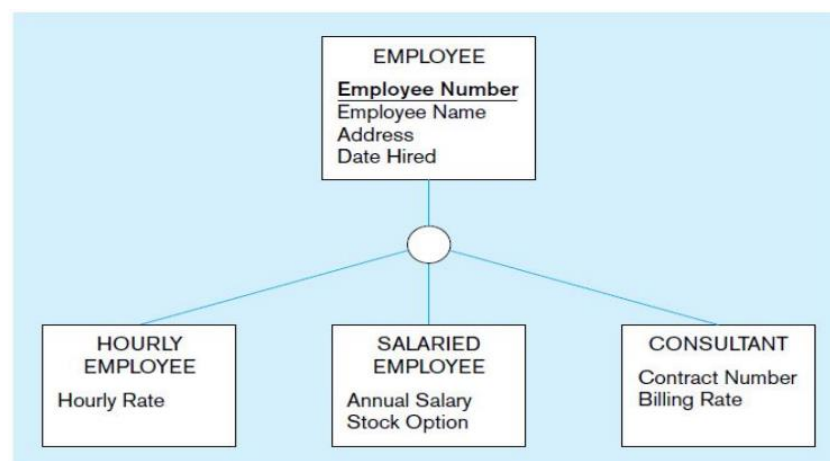


Figure 2.22: Example of Supertypes and Subtypes

Attribute inheritance

This is the property by which subtype entities inherit values of all attributes and instances of all relationships of the supertype.

- This vital property makes it unnecessary to include supertype attributes or relationships redundantly with the subtypes.

When to Use Supertype/Subtype Relationships

You should consider using subtypes when either (or both) of the following conditions are present:

- Some attributes apply to some (but not all) instances of an entity type.
- The instances of a subtype participate in a relationship unique to that subtype, as seen in Figure 2.23.

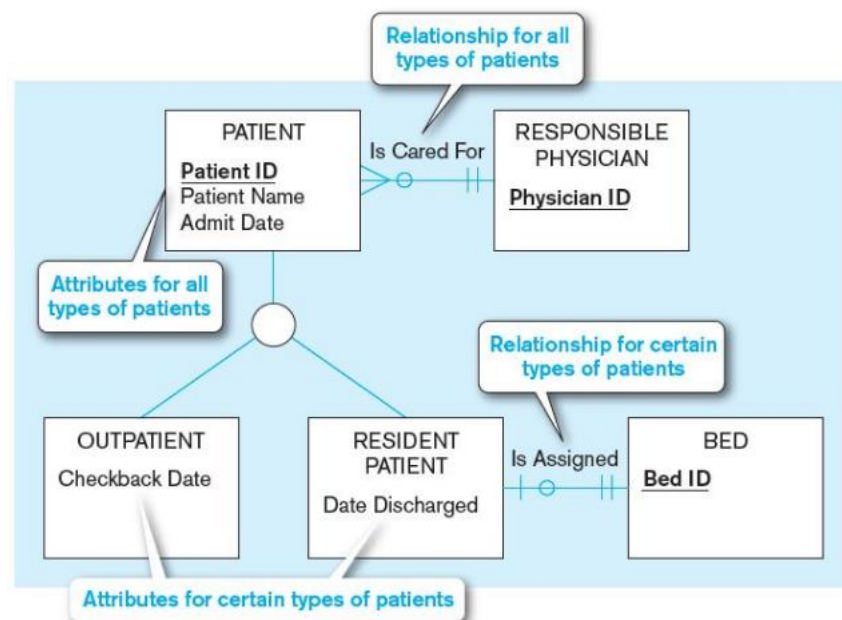


Figure 2.23: Supertype and Subtype Relationships

2.2.3 Specialisation and Generalisation

Two processes serve as mental models in developing supertype/subtype relationships.

1. Generalisation
2. Specialisation

Generalisation

- In data modelling, generalisation is the process of defining a more general entity type from a set of more specialised entity types, as seen in Figure 2.24.
- Generalisation is a bottom-up process.

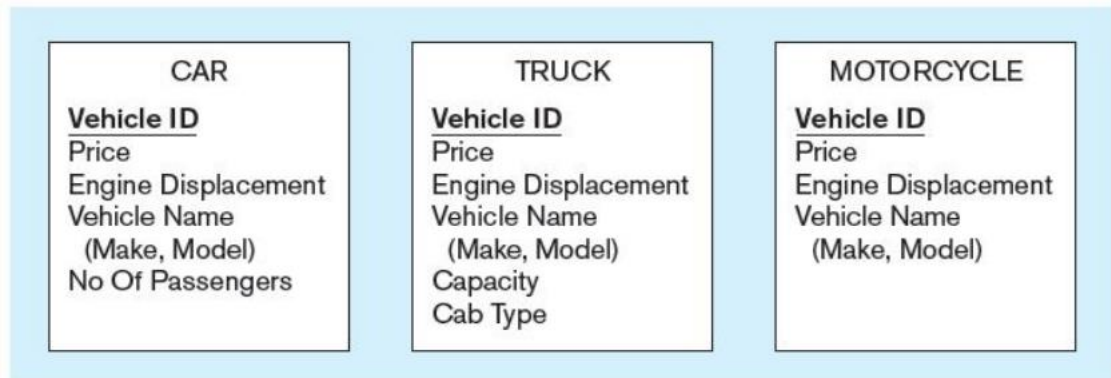


Figure 2.24: Sample of Generalisation 1

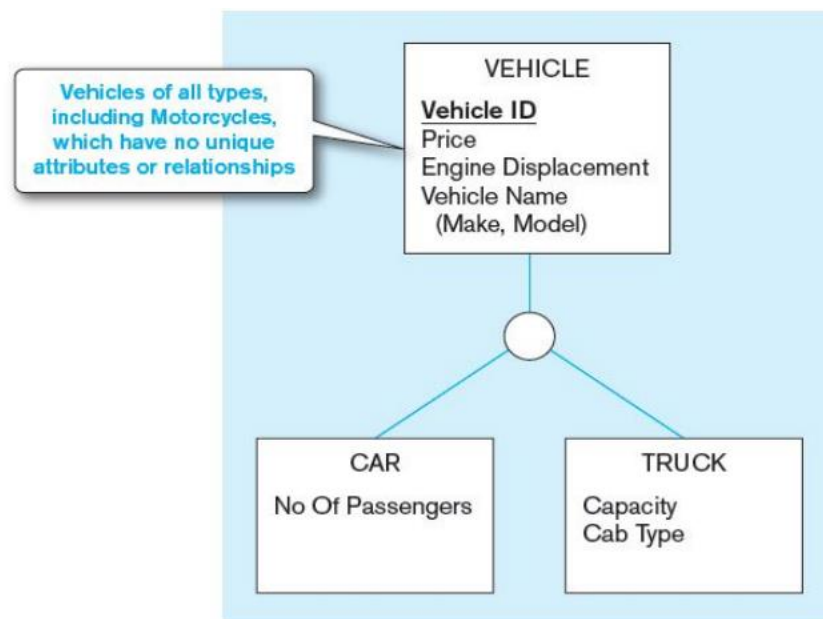


Figure 2.24: Sample of Generalisation 2

From the Figures above:

Notice that the entity type MOTORCYCLE is not included in the relationship.

Is this simply an omission? **No.**

- It is deliberately not included because it does not satisfy the conditions for a subtype as discussed earlier.

- Comparing Figure 3-4 parts a and b, you will notice that the only attributes of MOTORCYCLE are those that are common to all vehicles; there are no attributes specific to motorcycles.
- Furthermore, MOTORCYCLE does not have a relationship with another entity type. Thus, there is no need to create a MOTORCYCLE subtype.

Specialisation

Specialisation is a top-down process, the direct reverse of generalisation.

- Suppose that we have defined an entity type with its attributes.
- Specialisation is the process of defining one or more subtypes of the super-type and forming supertype/subtype relationships.

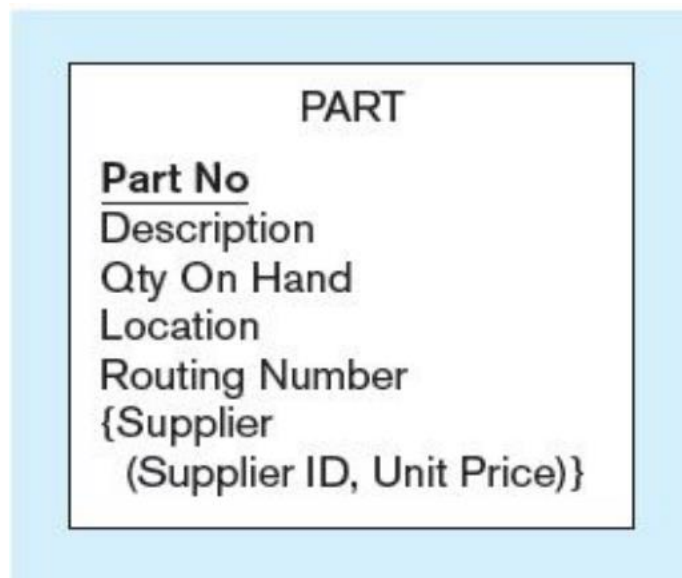


Figure 2.25: Sample of Specialisation 1

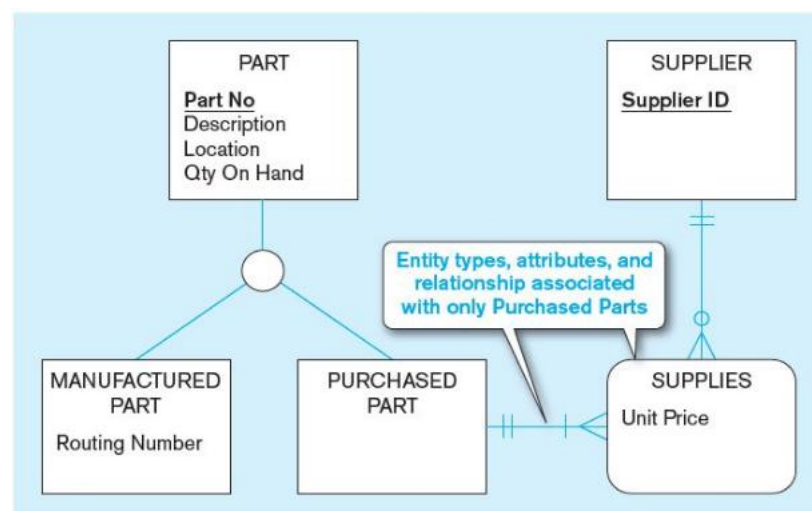


Figure 2.26: Sample of Specialisation 2

2.2.4 Constraints in Supertype/Subtype Relationships

There are additional notations to represent constraints on supertype/subtype relationships.

These constraints make it possible to capture some of the important business rules that apply to these relationships.

- The two most important types of constraints that are described in this section are:
 - Completeness constraints
 - Disjointness constraints

Completeness Constraints

- A completeness constraint addresses the question of whether an instance of a supertype must also be a member of at least one subtype.
- The completeness constraint has two possible rules:
 - Total specialisation
 - Partial specialisation
- The Total specialisation rule specifies that each entity instance of the supertype must be a member of some subtype in the relationship.
- The Partial specialisation rule specifies that an entity instance of the supertype is not allowed to belong to any subtype.

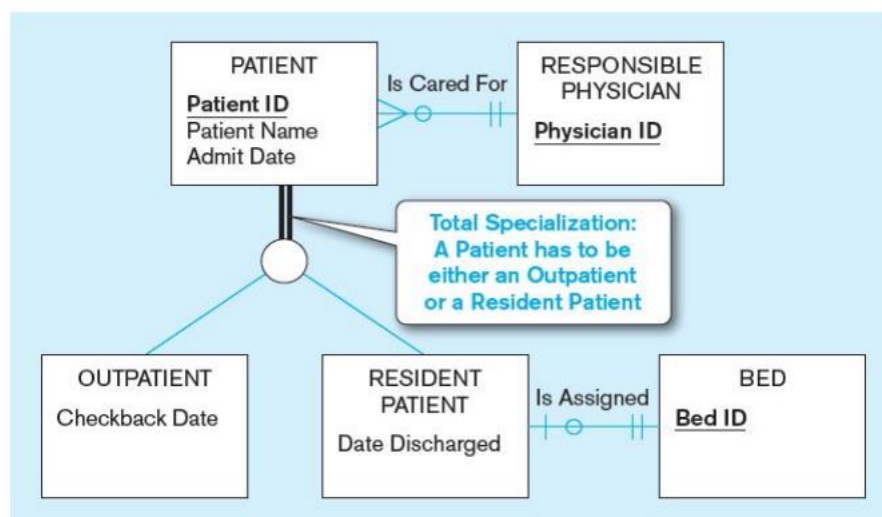


Figure 2.27: Completeness Constraints Total Specialisation Rule

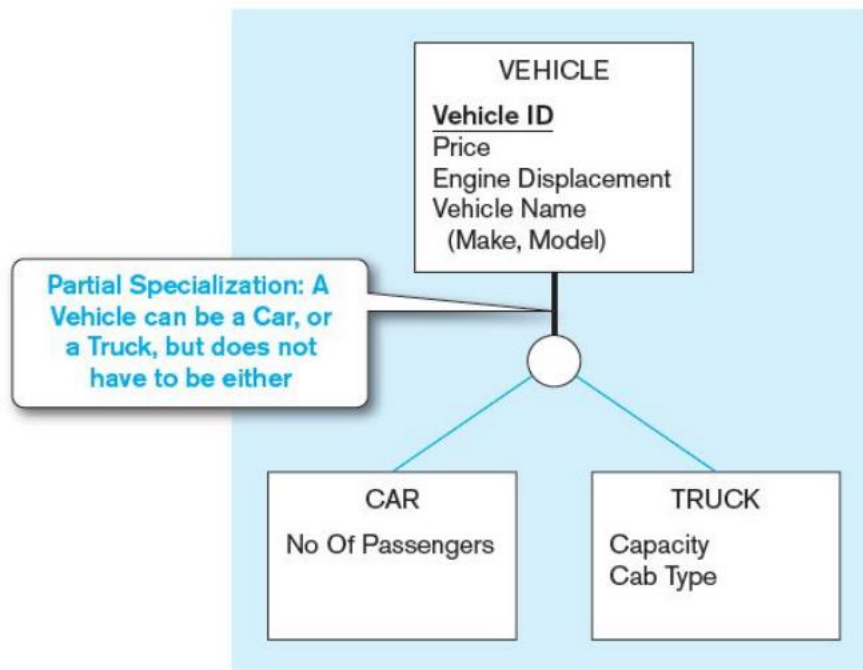


Figure 2.28: Completeness Constraints Partial Specialisation Rule

Disjointness Constraints

- A disjointness constraint addresses whether an instance of a supertype may simultaneously be a member of two (or more) subtypes.
- The disjointness constraint has two possible rules:
 - The Disjoint rule
 - The Overlap rule
- The Disjoint rule specifies that if an entity instance (of the supertype) is a member of one subtype, it cannot simultaneously be a member of any other subtype.
- The Overlap rule specifies that an entity instance can simultaneously be a member of two (or more) subtypes.

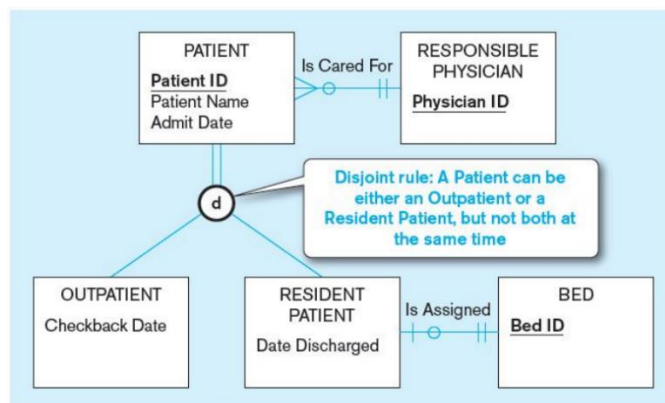


Figure 2.29: Disjointness Constraints Disjoint Rule

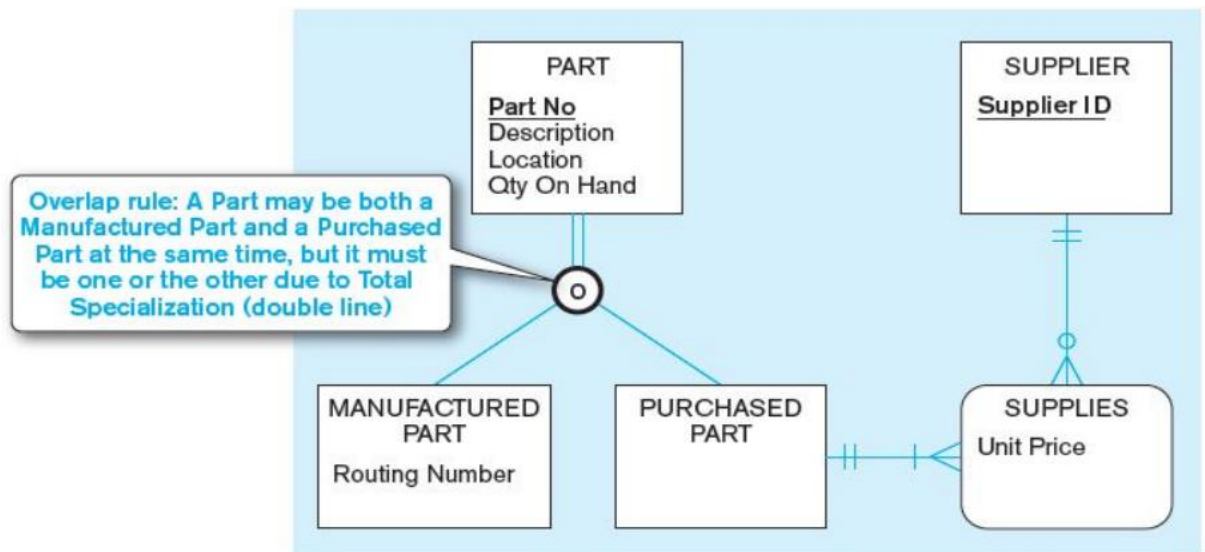


Figure 2.29: Disjointness Constraints Overlap Rule

2.2.5 Subtype Discriminators

We need a simple mechanism to implement these rules (as outlined in section 6.0 above), if one is available.

Often this can be accomplished by using a subtype discriminator.

A subtype discriminator

- This is an attribute of a supertype whose values determine the target subtype.

2.2.6 Supertype/Subtype Hierarchies

This is a hierarchical arrangement of supertypes and subtypes, where each subtype has only one supertype.

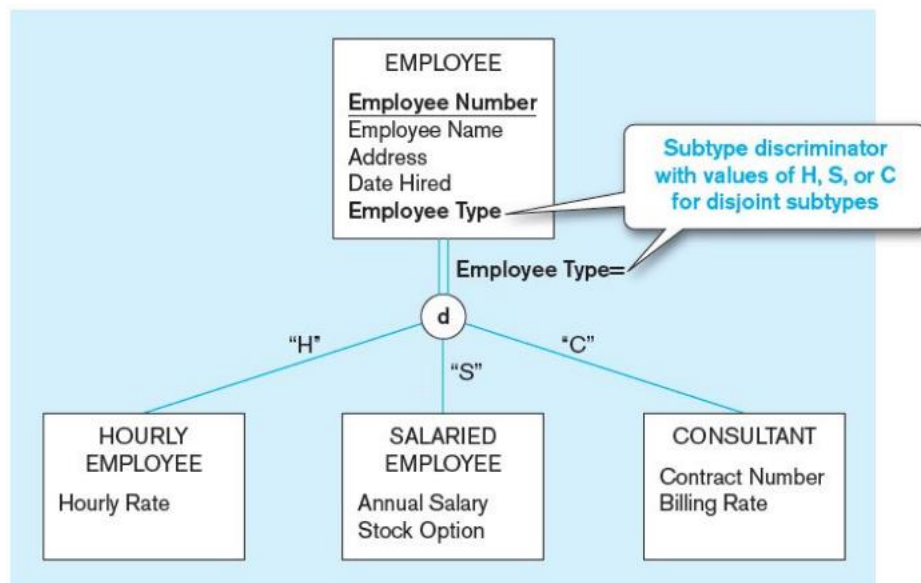


Figure 2.30: Subtype Discriminator; Disjoint

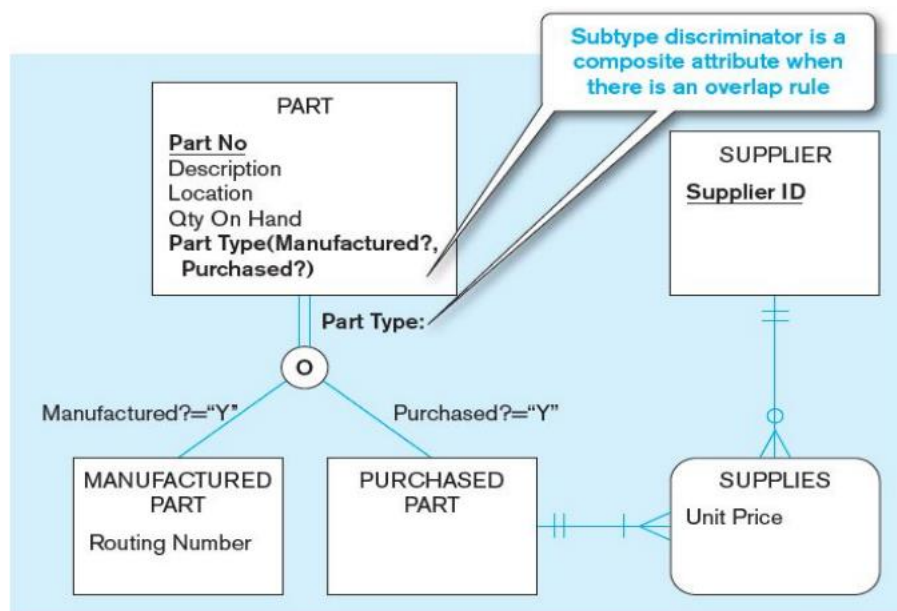


Figure 2.30: Subtype Discriminator; Overlap

Summary of Supertype/Subtype Hierarchies

2.2.7 Summary of Supertype/Subtype Hierarchies

- Attributes are assigned at the highest logical level that is possible in the hierarchy.
 - For example, because SSN (i.e., Social Security Number) applies to all persons, it is assigned to the root.
 - In contrast, Date Hired applies only to employees, so it is assigned to EMPLOYEE.

- This approach ensures that attributes can be shared by as many subtypes as possible.
- Subtypes that are lower in the hierarchy inherit attributes not only from their immediate supertype, but from all supertypes higher in the hierarchy, up to the root.
 - Thus, for example, an instance of faculty has values for all of the following attributes:
 - SSN, Name, Address, Gender, and Date of Birth (from PERSON); Date Hired and Salary (from EMPLOYEE); and Rank (from FACULTY).

2.2.8 Conclusion

In this unit, you have learned about Enhanced Entity-Relationship (E-R) modelling, supertypes and subtypes, and specialisation and generalisation, which help in organising entities based on shared or distinct characteristics. In addition, you have also learned about the constraints in subtype/supertype relationships, subtype discriminators, rules and supertype/subtype hierarchies.

Summary

In this unit, you have learned that Enhanced Entity-Relationship (E-R) modelling is an extension of the basic E-R model that provides greater expressiveness for representing real-world data. You also discovered that Enhanced E-R models introduce concepts such as supertypes and subtypes, which allow entities to be grouped into more general or more specific categories. You also explored the processes of specialisation and generalisation, constraints in subtype/supertype relationships, subtype discriminators and supertype/subtype hierarchies.

Self-Assessment Questions

1. The two most important types of constraints in Supertype/Subtype Relationships are?
2. What does the partial specialisation rule specify? How is this different from the total specialisation rule?
3. What are the processes that serve as the mental models in developing supertype/subtype relationships? Is there a difference between data modelling and conceptual modelling? Justify your answer.

Unit Activities

1. Read the content of Module 2, Unit 2 in the study guide. Consult the recommended texts and attempt the In-text and self-assessment questions. (90 minutes).
2. TUTOR MARKED ASSIGNMENT: Complete the following and submit for grading. (60 minutes)

Tutor-Marked Assignment

1. Differentiate between supertype and subtype. Using a real-life example, explain in detail the operations of each in an organisation's database system.
2. When can each of these supertype and subtype Relationships be used? What is their attribute inheritance?

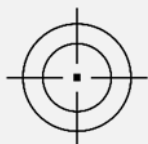
References/Further Readings

- Ramakrishnan, R., Gehrke, J.: **Database Management Systems**. USA: McGraw Hill Companies (2000)
- Elmasri, R., & Navathe, S. (2000). **Fundamentals of database systems (3rd ed.)**. Reading, Mass.: Addison-Wesley.
- Hoffer, J. A., Prescott, M. B., & McFadden, F. R. (2005). **Modern database management**. Upper Saddle River, N.J: Pearson/ Prentice Hall.
- Connolly, T. M., & Begg, C. E. (2002). **Database systems: A practical approach to design, implementation, and management**. Harlow, England: Addison-Wesley
- <http://pages.cs.wisc.edu/~dbbook/openAccess/thirdEdition/solutions/ans3ed-oddonly.pdf>
- <https://gyires.inf.unideb.hu/GyBITT/03/>

Unit 3 — Introduction to Semi-Structured Models (XML Model)

INTRODUCTION

In this unit, you will learn about semi-structured data models, which are used to represent data that does not conform to the traditional tabular structure of relational databases. Unlike structured data models, semi-structured data offers greater flexibility in data organisation and labelling, making it ideal for scenarios where data is irregular or evolves over time. This unit will also focus on popular semi-structured formats such as JSON (JavaScript Object Notation) and XML (eXtensible Markup Language). These formats use tags or key-value pairs to represent complex data structures and are widely used in modern web and data exchange applications. You will explore how JSON and XML store data, how to read and write data using these formats, and why they are essential in today's data-driven world.



LEARNING OUTCOMES

At the end of this unit, you should be able to:

- Understand the concept of semi-structured data models and their purpose.
- Identify and differentiate between JSON and XML formats.
- Read, write, and interpret data using JSON and XML.

2.3.1 Semi-structured

Semi-structured Data Models:

- Provides a flexible way to represent data that does not conform to the rigid schema of traditional relational databases but still maintains some organisational structure.
- Offer a middle ground between structured and unstructured data, providing a balance of flexibility and organisation that is particularly useful in scenarios where data formats and structures are subject to change.
- Semi-structured data are characterised by complex data whose schema changes often

Flexible Schema

Some database systems allow each tuple to have a different set of attributes; such a representation is referred to as a wide-column data representation, i.e., the set of attributes is not fixed; each tuple may have a different set of attributes, and new attributes may be added as needed.

- A more restricted form of this representation is to have a fixed but huge number of attributes, with each tuple using only those attributes that it needs, leaving the rest with null values; such a representation is called a sparse column representation.

Multivalued Data Types

Many data representations allow attributes to contain non-atomic values. Many databases support storing sets, multisets, or arrays as attribute values. For example, an application that stores a user's topics of interest and uses them to target articles or advertisements to the user may store them as a set. An example of such a set may be:

- {basketball, La Liga, cooking, anime, Jazz}

Nested Data types

Many data representations allow attributes to be structured, directly modelling composite attributes in the E-R model. For example, an attribute name may have component attributes firstname and lastname.

- There are two widely used data models that allow values to have complex internal structures and that are flexible, i.e values are not forced to adhere

to a fixed schema. These are the JavaScript Object Notation (JSON) and the Extensible Markup Language (XML).

2.3.2 JSON

- JSON: The JavaScript Object Notation (JSON), is a textual representation of complex data types that is widely used to transmit data between applications and to store complex data.
- JSON is now the primary data representation used for communication between applications and web services.

```
{
  "ID": "22222",
  "name": {
    "firstname": "Albert",
    "lastname": "Einstein"
  },
  "deptname": "Physics",
  "children": [
    { "firstname": "Hans", "lastname": "Einstein" },
    { "firstname": "Eduard", "lastname": "Einstein" }
  ]
}
```

2.3.3 eXtensible Markup Language (XML)

Many data representations allow attributes to be structured, directly modelling composite attributes in the E-R model. For example, an attribute name may have component attributes firstname and lastname

- XML is a widely used semi-structured data format that can be both schema-less (flexible) or schema-validated (strict). The way schemas interact with XML affects how data is stored, validated, and processed.
- XML is a text-based scripting language used to describe data structures hierarchically, using HTML-like tags
- XML adds tags enclosed in angle brackets, < >, to markup information in a textual representation. Tags are used in pairs, with <tag> and </tag> delimiting the beginning and the end of the portion of the text to which the tag refers

- For example, the title of a document can be marked up as:
 - <title>Semi-Structured Data Models</title>
- Such tags can be used to represent relational data specifying relation names and attribute names as tags, as shown in Figure 2.31 below:

```
<course>
  <course_id> CS-101 </course_id>
  <title> Intro. to Computer Science </title>
  <dept_name> Comp. Sci. </dept_name>
  <credits> 4 </credits>
</course>
```

Figure 2.31: XML Tags

- Furthermore, tags can be used to create hierarchical structures, which is not possible with the relational model.
- Hierarchical structures are particularly important for representing business objects that must be exchanged between organisations; examples include bills, purchase orders and so forth.

```
<purchase_order>
  <identifier> P-101 </identifier>
  <purchaser>
    <name> Cray Z. Coyote </name>
    <address> Route 66, Mesa Flats, Arizona 86047, USA </address>
  </purchaser>
  <supplier>
    <name> Acme Supplies </name>
    <address> 1 Broadway, New York, NY, USA </address>
  </supplier>
  <itemlist>
    <item>
      <identifier> RS1 </identifier>
      <description> Atom powered rocket sled </description>
      <quantity> 2 </quantity>
      <price> 199.95 </price>
    </item>
    <item>
      <identifier> SG2 </identifier>
      <description> Superb glue </description>
      <quantity> 1 </quantity>
      <unit-of-measure> liter </unit-of-measure>
      <price> 29.95 </price>
    </item>
  </itemlist>
  <total_cost> 429.85 </total_cost>
  <payment_terms> Cash-on-delivery </payment_terms>
  <shipping_mode> 1-second-delivery </shipping_mode>
</purchase_order>
```

Figure 2.32: Realistic Use of XML

The figure 2.32 above:

Shows how information about a purchase order can be represented in XML. It illustrates a more realistic use of XML. Purchase orders are typically generated by one organisation and sent to another. A purchase order contains a variety of information; the nested representation allows all of that data to be naturally represented in a single document.

to be naturally represented in a single document.

XML is not to, however, be confused with HTML. Whereas HTML tags are used to describe the appearance of content, XML tags are used to describe the content, or data, itself. Consider the following XML document stored in a file called PVFC.xml that is intended to provide the description of a product in PVFC:

```
<?xml version = "1.0"/>
<furniturecompany>
  <product ID="1">
    <description>End Table</description>
    <finish>Cherry</finish>
    <standard price>175.00</standard price>
    <line>1</line>
  </product>
</furniturecompany>
```

2.3.4 Storing XML Documents

One of the biggest questions that needs to be answered as XML data becomes more prevalent is "Where do we store these data?"

1. Store XML data in a relational database by shredding the XML document: Shredding an XML document essentially means that we store each element of an XML schema independently in a relational table and use other tables to store the relationships among the components. Modern databases such as Microsoft SQL Server and Oracle provide capabilities beyond standard SQL to help store and retrieve XML data.
2. Store the XML document using special XML columns that are made available as part of the database: These columns can be associated with an XSD, for example, to ensure that the XML document that is being inserted is a valid document.
3. Store the XML document using a native XML database: These are non-relational databases designed specifically to store XML documents.

Note: the first two options are primarily used when XML is used as a data exchange format between a browser and an application server.

The first option is used when the majority of the information being processed is originally in XML format.

Retrieving XML Documents

Modern databases provide extensive support for retrieving information from databases in XML format. The key technologies behind XML data retrieval are XPath and

XQuery. Each of the options provides specific mechanisms by which you can retrieve data in XML format:

- XPath: One of a set of XML technologies that supports XQuery development. XPath expressions are used to locate data in XML documents.
- XQuery: An XML transformation language that allows applications to query both relational databases and XML data. Built on XPath expressions, XQuery is now supported by the major relational database engines, including those from IBM, Oracle, and Microsoft

Take a look at the XML document in Figure 2.33. Now, consider the following XQuery expression that returns all product elements that have a standard price > 300.00:

```
for $p in doc("PVFC.xml")/furniture company/product
where $p/standardprice>300.00
order by $p/description
return $p/description
```

You can see the similarities between XQuery and SQL in this example. It is often said that XQuery is to XML as SQL is to relational databases.

```
<?xml version="1.0"?>
<furniture company>
  <product ID="1">
    <description>End Table</description>
    <finish>Cherry</finish>
    <standard price>175.00</standard price>
    <line>1</line>
  </product>
  <product ID="2">
    <description>Coffee Table</description>
    <finish>Natural Ash</finish>
    <standard price>200.00</standard price>
    <line>2</line>
  </product>
  <product ID="3">
    <description>Computer Desk</description>
    <finish>Natural Ash</finish>
    <standard price>375.00</standard price>
    <line>2</line>
  </product>
  <product ID="4">
    <description>Entertainment Center</description>
    <finish>Natural Maple</finish>
    <standard price>650.00</standard price>
    <line>3</line>
  </product>
  <product ID="5">
    <description>Writers Desk</description>
    <finish>Cherry</finish>
    <standard price>325.00</standard price>
    <line>1</line>
  </product>
  <product ID="6">
    <description>8-Drawer Desk</description>
    <finish>White Ash</finish>
    <standard price>750.00</standard price>
    <line>2</line>
  </product>
  <product ID="7">
    <description>Dining Table</description>
    <finish>Natural Ash</finish>
    <standard price>800.00</standard price>
    <line>2</line>
  </product>
  <product ID="8">
    <description>Computer Desk</description>
    <finish>Walnut</finish>
    <standard price>250.00</standard price>
    <line>3</line>
  </product>
</furniture company>
```

Figure 2.33: Retrieval of XML Documents

Displaying XML Documents

The display of XML data on a Web browser is controlled by a stylesheet specified using the Extensible Stylesheet Language (XSL).

Most modern browsers and programming languages provide support for XSLT. Thus, the transformation of the XML can happen either at the Web server layer or the application server layer.

One of the advantages of XSLT is that it can be used to handle the myriad devices that now use the Internet. For example, smartphones and tablets have built-in browsers that allow a user to access the Internet.

Figure 2.34 shows a sample XSLT specification for displaying Salesperson data as an HTML table.

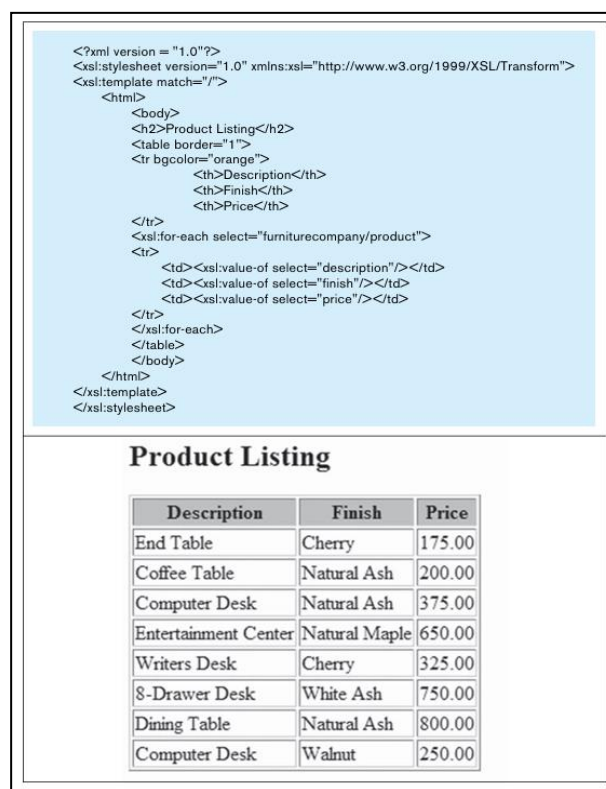


Figure 2.34: Sample XSLT specification

2.3.5 Summary of Semi-Structured Data Models

- Unlike with a relational schema, new tags can be introduced easily. With suitable names, the data are "self-documenting" in that a human can understand or guess what a particular piece of data means based on the name.
- The XQuery language was developed to support querying of XML data, even though adoption of XQuery has been relatively limited.
- The SQL language itself has been extended to support XML in several ways:
- XML data can be stored as an XML data type.

- SQL queries can generate XML data from relational data. Such extensions are very useful for packaging related data into a single XML document, which can then be sent to another application. The extensions allow the construction of XML representations from individual rows, as well as the creation of an XML document from a collection of rows by using an XMLAGG aggregate function.
- SQL queries can extract data from an XML data type value. For example, the XPath language supports “path expressions” that allow the extraction of desired parts of data from an XML document.

2.3.6 Key Features of Semi-Structured Data Models

1. Schema Flexibility:
 - No fixed schema is required upfront
 - Fields can vary per record
2. Hierarchical Structure
 - Data is often nested (e.g., trees, graphs) rather than flat (like tables).
3. Self-Describing
 - Metadata (e.g., tags, keys, or attributes) describes the data.
4. Partial Organisation: While it doesn't fit neatly into tables, semi-structured data includes organisational elements like tags or metadata, making it more manageable than unstructured data.

2.3.7 Conclusion

In this unit, you have learned about semi-structured data model, JSON (JavaScript Object Notation) and XML (eXtensible Markup Language). You have also explored how JSON and XML store data, how to read and write data using these formats, and why they are important in today's data-driven world.

Summary

In this unit, you have learned about semi-structured data models, which are used to represent data that does not conform to the traditional tabular structure of relational databases. You also learned popular semi-structured formats such as JSON (JavaScript Object Notation) and XML (eXtensible Markup Language).

Self-Assessment Questions

1. Is XML the same as HTML? Justify your answer.
2. Explain the Semi-Structured Data types.

Unit Activities

1. Read the content of Unit 1 in the study guide. Consult the recommended texts and research more information on the internet. (90 minutes)
2. Complete the self-introductory forum. Original post by Thursday. (10 minutes). Complete replies to 2 of your classmates by Sunday at 11:59 p.m. (10 minutes)
3. TUTOR MARKED ASSIGNMENT: Complete the following and submit for grading. (60 minutes))

Tutor-Marked Assignment

1. Identify and write out four (4) inconveniences of using semi-structured data models
2. What advantage do semi-structured data models have over models with rigid organisation?

References/Further Readings

Ramakrishnan, R., Gehrke, J.: Database Management Systems. USA: McGraw-Hill Companies (2000)

Elmasri, R., & Navathe, S. (2000). Fundamentals of database systems (3rd ed.). Reading, Mass.: Addison-Wesley.

Hoffer, J. A., Prescott, M. B., & McFadden, F. R. (2005). Modern database management. Upper Saddle River, N.J: Pearson/ Prentice Hall.

Connolly, T. M., & Begg, C. E. (2002). Database systems: A practical approach to design, implementation, and management. Harlow, England: Addison-Wesley

<http://pages.cs.wisc.edu/~dbbook/openAccess/thirdEdition/solutions/ans3ed-oddonly.pdf>

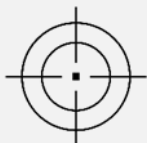
<https://gyires.inf.unideb.hu/GyBITT/03/>

Rosen, K. H. (2019). Discrete Mathematics

Unit 4 — Examples of Conceptual Models

INTRODUCTION

In this unit, you will explore real-life examples of conceptual data models. You'll see how we use the Entity-Relationship (E-R) model to represent key elements such as entities (things), attributes (details about those things), and relationships (how the things are connected) in simple and practical ways. This unit brings theory to life through relatable examples from everyday systems such as schools, hospitals, and online stores.



LEARNING OUTCOMES

At the end of this unit, you should be able to:

- Read and create basic E-R diagrams based on everyday scenarios.
- Apply conceptual modelling techniques to real-world case studies.

2.4.1 Examples of Conceptual Models

University Database

Imagine a system that tracks a university's students, courses, lecturers, and departments. Each student enrolls in courses. Lecturers teach the courses. Courses belong to departments. Each of these (Student, Course, Lecturer, Department) would be an entity, and we'd use an E-R diagram to show how they're connected.

Hospital Management System

In a hospital, we have patients, doctors, appointments, and treatments. A patient consults a doctor, receives treatment, and schedules appointments. These elements become our entities, and we draw relationships like "consults," "prescribes," and "undergoes" to connect them.

Online Retail Store

Think of a shopping app. There are customers, products, orders, and payments. Customers place orders. Orders contain products. Customers also make payments. These real-world relationships are captured visually in the conceptual model.

Library System

A library keeps records of books, members, librarians, and loans. Members borrow books. Librarians manage book loans. This typical example shows how data flows in a library and helps us practice modelling relationships like "borrows" or "returns."

Hotel Reservation System

In a hotel, guests book rooms, staff manage reservations, and each room includes details such as type and rate. This system helps us show bookings, check-ins, and staff roles in a way that captures all essential data elements and their relationships.

2.4.2 Conclusion

In this unit, you've seen how conceptual models apply to everyday systems like schools, hospitals, and stores. This step is essential for building a functional and efficient database.

Summary

In this unit, you have learned about semi-structured data models, which are used to represent data that does not conform to the traditional tabular structure of relational databases. You also learnt popular semi-structured formats such as JSON (JavaScript Object Notation) and XML (eXtensible Markup Language).

Self-Assessment Questions

1. Draw a simple E-R diagram for a hospital with patients, doctors, and treatments.

Unit Activities

1. Read the content of Unit 1 in the study guide. Consult the recommended texts and research more information on the internet. (90 minutes)
2. Complete the self-introductory forum. Original post by Thursday. (10 minutes). Complete replies to 2 of your classmates by Sunday at 11:59 p.m. (10 minutes)
3. TUTOR MARKED ASSIGNMENT: Complete the following and submit for grading. (60 minutes))

Tutor-Marked Assignment

1. Create a conceptual model for a hotel reservation system. Include guests, rooms, reservations, and staff.
2. List four attributes for each entity in a library management system.
3. Explain why creating a conceptual model is essential before building a database.

References/Further Readings

Ramakrishnan, R., Gehrke, J.: **Database Management Systems**. USA: McGraw-Hill Companies (2000)

Elmasri, R., & Navathe, S. (2000). **Fundamentals of database systems (3rd ed.)**. Reading, Mass.: Addison-Wesley.

Hoffer, J. A., Prescott, M. B., & McFadden, F. R. (2005). **Modern database management**. Upper Saddle River, N.J: Pearson/ Prentice Hall.

Connolly, T. M., & Begg, C. E. (2002). **Database systems: A practical approach to design, implementation, and management**. Harlow, England: Addison-Wesley

<http://pages.cs.wisc.edu/~dbbook/openAccess/thirdEdition/solutions/ans3ed-oddonly.pdf>

<https://gyires.inf.unideb.hu/GyBITT/03/>

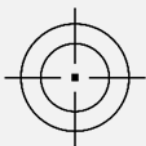
Module Three

Logical Design

Unit 1 — Introduction to Relational Model

INTRODUCTION

In this unit, you will learn about the foundational principles of database design, including the rules and specifications that guide how a database should be appropriately structured and implemented. You will be introduced to the relational data model, which organises data into tables (relations) and ensures logical consistency through clearly defined relationships. This unit will also cover the concept of relational keys, such as primary and foreign keys, which are used to uniquely identify records and maintain connections between tables. You'll learn how to remove multivalued attributes from relations to ensure each table holds atomic data values, in line with relational model rules.



LEARNING OUTCOMES

At the end of this unit, you should be able to:

- Define and identify various types of sets, including empty, singleton, finite, infinite, equal, and equivalent sets.
- Apply set-theoretic notation and understand set membership using symbols like $\in$, $\notin$, $\subseteq$, $\subset$, and $P(A)$.
- Perform basic and advanced set operations such as union, intersection, difference, symmetric difference, and complement, with appropriate use of Venn diagrams.
- Describe relationships between sets and determine subset, proper subset, and disjoint set conditions with examples

3.1.1 Database Design (Logical Database Design)

Database design involves transforming the requirements developed during data-base analysis into specifications to guide database implementation.

Specifications that Guide Database Implementation

1. Logical specifications

It maps the conceptual requirements into the data model associated with a specific database management system.

2. Physical specifications

It specifies all the parameters for data storage, which are then used as input for database implementation.

During this phase, a database is defined using a data definition language (DDL).

Logical Database Design

Logical database design is the process of transforming a conceptual data model into a logical data model that is consistent with and compatible with a specific type of database technology.

3.1.2 Relational Data Model

- The Relational Data model was first introduced by E. F. Codd around the 1970s.
- Two early research projects were launched to prove the feasibility of the relational model and to develop prototype systems.
- The first of these, at IBM's San Jose Research Laboratory, led to the development of System R (a prototype relational DBMS [RDBMS]) during the late 1970s.
- The second, at the University of California at Berkeley, led to the development of Ingres, an academically oriented RDBMS.
- Commercial RDBMS products from numerous vendors started to appear about 1980.
- The relational data model represents data in the form of tables.
- The relational model is based on mathematical theory
- The relational data model consists of three (3) components:
 - a. **Data structure:** Data are organised in the form of tables, with rows and columns.

- b. **Data manipulation:** Powerful operations (typically implemented using the SQL language) are used to manipulate data stored in the relations.
- c. **Data integrity:** The model includes mechanisms to specify business rules that maintain the integrity of data when they are manipulated.

EMPLOYEE1			
EmpID	Name	DeptName	Salary
100	Margaret Simpson	Marketing	48,000
140	Allen Beeton	Accounting	52,000
110	Chris Lucero	Info Systems	43,000
190	Lorenzo Davis	Finance	55,000
150	Susan Martin	Marketing	42,000

Figure 3.1: Relation EMPLOYEE1 consisting of four attributes (columns) and tuples (rows)

Relational Data Structure

- A relation is a named, two-dimensional table of data.
- Each relation (or table) consists of a set of named columns and an arbitrary number of unnamed rows.
- An attribute is a named column of a relation.
- Each row of a relation corresponds to a record that contains data (attribute) values for a single entity.
- In Figure 3.1, EMPLOYEE1 contains the following attributes describing employees:
 - EmpID, Name, DeptName, and Salary.
- The five rows of the table correspond to five employees.

Short Hand Notation Denoting a Relation

- We can express the structure of a relation by using a shorthand notation in which the name of the relation is followed (in parentheses) by the names of the attributes in that relation.
- For EMPLOYEE1, we would have:
 - EMPLOYEE1(EmpID, Name, DeptName, Salary)

Relational Keys

- We must be able to store and retrieve a row of data in a relation, based on the data values stored in that row.

- To achieve this goal, every relation must have a primary key.
- A primary key is an attribute or a combination of attributes that uniquely identifies each row in a relation.
- We designate a primary key by underlining the attribute name(s).
- For example, the primary key for the relation EMPLOYEE1 is EmpID.
- Notice that this attribute is underlined in Figure 3.1.
- In shorthand notation, we express this relation as follows:
 - EMPLOYEE1(EmpID, Name, DeptName, Salary).
- A **composite key** is a primary key that consists of more than one attribute.
- For example, the primary key for a relation DEPENDENT would likely consist of the combination EmpID and DependentName.
- Often, we must represent the relationship between two tables or relations.
- This is accomplished through the use of foreign keys.
- **A foreign key is an attribute in a relation that serves as the primary key of another relation in the same database.**
- For example, consider the relations EMPLOYEE1 and DEPARTMENT:

EMPLOYEE1(EmpID, Name, DeptName, Salary)

DEPARTMENT(DeptName, Location, Fax)

- The attribute DeptName is a foreign key in EMPLOYEE1.
- It allows a user to associate any employee with the department to which they are assigned.

Properties of a Relation

1. Each relation (or table) in a database has a unique name.
2. An entry at the intersection of each row and column is atomic (or single valued).
3. Each row is unique; no two rows in a relation can be identical.
4. Each attribute (or column) within a table has a unique name.
5. The sequence of columns (left to right) is insignificant.
 - The order of the columns in a relation can be changed without changing the meaning of the relation.
6. The sequence of rows (top to bottom) is insignificant.
 - As with columns, the order of the rows of a relation may be changed or stored in any sequence.

Removing Multivalued Attributes from a Relation

- Figure 3.2 below shows the employee data from the EMPLOYEE1 relation extended to include courses that those employees have taken.
- A given employee may have taken more than one course; CourseTitle and DateCompleted are multivalued attributes.
- For example, the employee with EmpID100 has taken two courses; therefore, there are two values of both CourseTitle (SPSS and Surveys) and DateCompleted (6/9/2015 and 10/7/2015) associated with one value of EmpID (100).
- However, as you will see, this new relation in Figure 3.2 has some undesirable properties.
 - The primary key column EmpID no longer uniquely identifies each of the rows.
- If an employee has not taken any courses, the CourseTitle and DateCompleted attribute values are null.
- Check the employee with EmpID 190 for an example

(a) Table with repeating groups					
EmpID	Name	DeptName	Salary	CourseTitle	DateCompleted
100	Margaret Simpson	Marketing	48,000	SPSS	6/19/2015
				Surveys	10/7/2015
140	Alan Beeton	Accounting	52,000	Tax Acc	12/8/2015
110	Chris Lucero	Info Systems	43,000	Visual Basic	1/12/2015
				C++	4/22/2015
190	Lorenzo Davis	Finance	55,000		
150	Susan Martin	Marketing	42,000	SPSS	6/16/2015
				Java	8/12/2015

(b) EMPLOYEE2 relation					
EMPLOYEE2					
EmpID	Name	DeptName	Salary	CourseTitle	DateCompleted
100	Margaret Simpson	Marketing	48,000	SPSS	6/19/2015
100	Margaret Simpson	Marketing	48,000	Surveys	10/7/2015
140	Alan Beeton	Accounting	52,000	Tax Acc	12/8/2015
110	Chris Lucero	Info Systems	43,000	Visual Basic	1/12/2015
110	Chris Lucero	Info Systems	43,000	C++	4/22/2015
190	Lorenzo Davis	Finance	55,000		
150	Susan Martin	Marketing	42,000	SPSS	6/19/2015
150	Susan Martin	Marketing	42,000	Java	8/12/2015

Figure 3.2: Removing Multivalued Attributes from a Relation

Sample Database

- A relational database may consist of any number of relations.
- The structure of the database is described through the use of a schema, which is a description of the overall logical structure of the database.
- Common Methods for Expressing a Schema
- Short text statements: each relation is named, and the names of its attributes follow in parentheses.

- A graphical representation: each relation is represented by a rectangle containing the attributes for the relation.
- Text statements have the advantage of simplicity. However, a graphical representation provides a better means of expressing referential integrity constraints.
- A schema for four relations at Pine Valley Furniture Company is shown in Fig. 3.3.
- The four relations shown in this figure are CUSTOMER, ORDER, ORDERLINE, and PRODUCT.
- The key attributes for these relations are underlined, and other important attributes are included in each relation

CUSTOMER(CustomerID, CustomerName, CustomerAddress,
 CustomerCity, CustomerState, CustomerPostalCode)
 ORDER(OrderID, OrderDate, CustomerID)
 ORDER LINE(OrderID, ProductID, OrderedQuantity)
 PRODUCT(ProductID, ProductDescription, ProductFinish,
 ProductStandardPrice, ProductLineID)

Figure 3.3: Text Description of Relation/ Schema for four relations

Text Description of Relations

- Notice that the primary key for ORDER LINE in Fig 2.1 is a composite key consisting of the attributes OrderID and ProductID.
- Also, CustomerID is a foreign key in the ORDER relation; this allows the user to associate an order with the customer who submitted the order.
- ORDER LINE has two foreign keys: OrderID and ProductID.

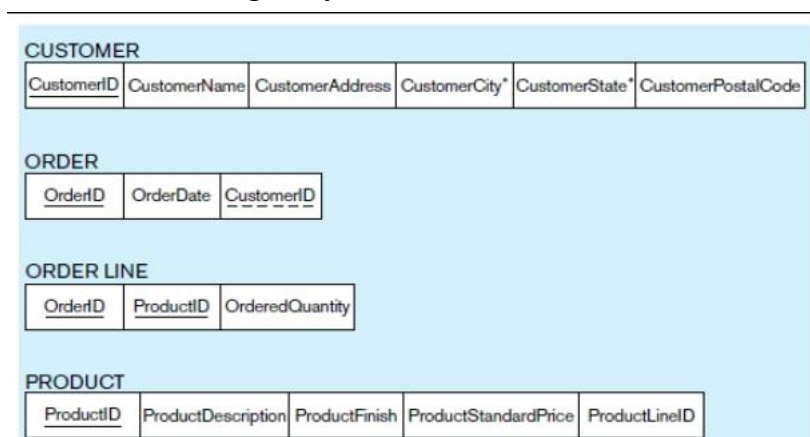


Figure 3.4: Graphical Representation of Relation

3.1.3 Integrity Constraint

- The relational data model includes several types of constraints, or rules limiting acceptable values and actions, whose purpose is to facilitate maintaining the accuracy and integrity of data in the database.
- The major types of integrity constraints are:
 - domain constraints
 - entity integrity, and
 - referential integrity

Domain Constraint

- A domain is the set of values that may be assigned to an attribute.
- A domain definition usually consists of the following components: domain name, meaning, data type, size (or length), and allowable values or allowable range (if applicable).
- All of the values that appear in a column of a relation must be from the same domain.

Attribute	Domain Name	Description	Domain
CustomerID	Customer IDs	Set of all possible customer IDs	character: size 5
CustomerName	Customer Names	Set of all possible customer names	character: size 25
CustomerAddress	Customer Addresses	Set of all possible customer addresses	character: size 30
CustomerCity	Cities	Set of all possible cities	character: size 20
CustomerState	States	Set of all possible states	character: size 2
CustomerPostalCode	Postal Codes	Set of all possible postal zip codes	character: size 10
OrderID	Order IDs	Set of all possible order IDs	character: size 5
OrderDate	Order Dates	Set of all possible order dates	date: format mm/dd/yy
ProductID	Product IDs	Set of all possible product IDs	character: size 5
ProductDescription	Product Descriptions	Set of all possible product descriptions	character: size 25
ProductFinish	Product Finishes	Set of all possible product finishes	character: size 15
ProductStandardPrice	Unit Prices	Set of all possible unit prices	monetary: 6 digits
ProductLineID	Product Line IDs	Set of all possible product line IDs	integer: 3 digits
OrderedQuantity	Quantities	Set of all possible ordered quantities	integer: 3 digits

Figure 3.5: Domain Definition

Entity Integrity

- The entity integrity rule is designed to ensure that every relation has a primary key and that the data values for that primary key are all valid.
- In particular, it guarantees that every primary key attribute is non-null.
- In some cases, a particular attribute cannot be assigned a data value.
- Either there is no applicable data value or the applicable data value is not known when values are assigned.

Referential Integrity Constraint

- A referential integrity constraint is a rule that maintains consistency among the rows of two relations.
- The rule states that if there is a foreign key in one relation, either each foreign key value must match a primary key value in another relation or the foreign key value must be null in the same database.

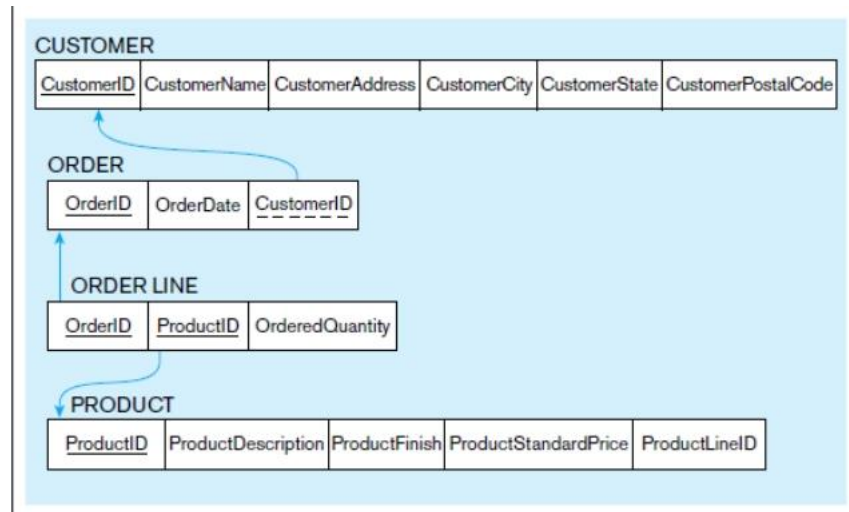


Figure 3.6: Referential Integrity Constraint

Null Constraints

- A null is a value that may be assigned to an attribute when no other value applies or when the applicable value is unknown.
- In reality, a null is not a value, but rather it indicates the absence of a value.
- For example, it is not the same as a numeric zero or a string of blanks.
- The inclusion of nulls in the relational model is somewhat controversial, because it sometimes leads to anomalous results.

3.1.4 A Well-Structured Relation

- A well-structured relation contains minimal redundancy and allows users to insert, modify, and delete the rows in a table without errors or inconsistencies.
- EMPLOYEE1 (Figure 3.7) is such a relation.
- Each row of the table contains data describing one employee, and any modification to an employee's data (such as a change in salary) is confined to one row of the table.

EMPLOYEE1			
EmpID	Name	DeptName	Salary
100	Margaret Simpson	Marketing	48,000
140	Allen Beeton	Accounting	52,000
110	Chris Lucero	Info Systems	43,000
190	Lorenzo Davis	Finance	55,000
150	Susan Martin	Marketing	42,000

Figure 3.7: Well-Structured Relation

3.1.5 NOT- Well-Structured Relation

- In contrast, EMPLOYEE2 (Figure 3.8) is not a well-structured relation. If you examine the sample data in the table, you will notice considerable redundancy.
- For example, values for EmpID, Name, DeptName, and Salary appear in two separate rows for employees 100, 110, and 150.
- Consequently, if the salary for employee 100 changes, we must record this fact in two rows.
- Redundancies in a table may result in errors or inconsistencies (called anomalies) when a user attempts to update the data in the table.

(b) EMPLOYEE2 relation

EMPLOYEE2					
EmpID	Name	DeptName	Salary	CourseTitle	DateCompleted
100	Margaret Simpson	Marketing	48,000	SPSS	6/19/2015
100	Margaret Simpson	Marketing	48,000	Surveys	10/7/2015
140	Alan Beeton	Accounting	52,000	Tax Acc	12/8/2015
110	Chris Lucero	Info Systems	43,000	Visual Basic	1/12/2015
110	Chris Lucero	Info Systems	43,000	C++	4/22/2015
190	Lorenzo Davis	Finance	55,000		
150	Susan Martin	Marketing	42,000	SPSS	6/19/2015
150	Susan Martin	Marketing	42,000	Java	8/12/2015

Figure 3.8: NOT-Well-Structured Relation

3.1.6 Anomaly

- An anomaly is an error or inconsistency that may occur when a user attempts to update a table containing redundant data.
- The three types of anomalies are:
 - insertion
 - deletion
 - modification anomalies.

Insertion Anomaly

- Suppose that we need to add a new employee to EMPLOYEE2.
- The primary key for this relation is the combination of EmpID and CourseTitle (See Figure 3.8).
- Therefore, to insert a new row, the user must supply values for both EmpID and CourseTitle (because primary key values cannot be null or nonexistent).
- This is an anomaly because the user should be able to enter employee data without supplying course data.

<u>EmpID</u>	<u>CourseTitle</u>	<u>DateCompleted</u>
100	SPSS	6/19/2015
100	Surveys	10/7/2015
140	Tax Acc	12/8/2015
110	Visual Basic	1/12/2015
110	C++	4/22/2015
150	SPSS	6/19/2015
150	Java	8/12/2015

Figure 3.9: Emp-Course Relation

Deletion Anomaly

- Suppose that the data for employee number 140 are deleted from the table in Figure 3.9.
- This will result in losing the information that this employee completed a course (Tax Acc) on 12/8/2015.
- In fact, it results in losing the information that this course had an offering that completed on that date.

Modification Anomaly

- Suppose that employee number 100 gets a salary increase.
- We must record the increase in each of the rows for that employee (two occurrences in Figure 3.8); otherwise, the data will be inconsistent.

3.1.7 Conclusion

In this unit, you have learned about the essential components of database design, including the specifications that guide database implementation and the structure of the relational data model. You also explored key concepts such as relational keys, integrity constraints, and handling multivalued attributes. Additionally, you learned how to identify well-structured and poorly structured relations, as well as how to

avoid common data anomalies. All of these elements contribute to building efficient, reliable, and logically sound database systems.

Summary

In this unit, you have learnt that database design involves organising data in a way that ensures accuracy, consistency, and efficiency. You explored how the relational data model structures data into tables using relational keys to maintain relationships. You also learnt how to remove multivalued attributes, apply integrity constraints, and recognise the difference between well-structured and not-well-structured relations. Finally, you examined how poor design can lead to data anomalies and how to prevent them through careful planning and normalisation

Self-Assessment Questions

1. What is the purpose of the relational keys?
2. What are integrity constraints? Explain them

Tutor-Marked Assignment

4. Describe the primary differences between the conceptual and logical data models.
5. List the three components of the relational data model.
6. What is a schema? Discuss two common methods of expressing a schema

References/Further Readings

- Ramakrishnan, R., Gehrke, J.: Database Management Systems. USA: McGraw-Hill Companies (2000)
- Elmasri, R., & Navathe, S. (2000). Fundamentals of database systems (3rd ed.). Reading, Mass.: Addison-Wesley.
- Hoffer, J. A., Prescott, M. B., & McFadden, F. R. (2005). Modern database management. Upper Saddle River, N.J: Pearson/ Prentice Hall.
- Connolly, T. M., & Begg, C. E. (2002). Database systems: A practical approach to design, implementation, and management. Harlow, England: Addison-Wesley

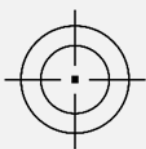
<http://pages.cs.wisc.edu/~dbbook/openAccess/thirdEdition/solutions/ans3ed-oddonly.pdf>

<https://gyires.inf.unideb.hu/GyBITT/03/>

Unit 2 — Transforming Conceptual to Logical Design

INTRODUCTION

In this unit, you will learn how to transform a conceptual data model into a logical design, which is a more detailed and structured representation of data suitable for implementation in a relational database system. This process involves converting high-level entities, attributes, and relationships (usually represented in an Entity-Relationship (ER) diagram) into relational tables with defined keys and constraints. You will also explore how to transform regular entities from the conceptual model into relations, assigning appropriate primary keys and ensuring data is represented in a normalised and consistent manner. This step is crucial for preparing a database design that is ready for implementation while preserving the semantics of the original conceptual model.



LEARNING OUTCOMES

At the end of this unit, you should be able to:

- Understand the purpose of transforming a conceptual model into a logical design.
- Convert regular entities from an ER diagram into relational tables.
- Assign appropriate primary keys and attributes during the transformation process.
- Ensure that the logical design maintains the integrity and structure of the original conceptual model.

3.2.1 Introduction to Transformation

- During logical design, E-R (and EER) diagrams are transformed into relational database schemas.
- The inputs to this process are the entity-relationship (and enhanced E-R) diagrams, while the outputs are the relational schemas.
- Transforming (or mapping) EER diagrams into relations is a relatively straightforward process with a well-defined set of rules.
- Many CASE tools can automatically perform many of the conversion steps.
- However, it is vital that you understand the steps in this process.

3.2.2 Transforming Regular Entities

- Regular entities are entities that have an independent existence and generally represent real-world objects, such as persons and products.
- Regular entity types are represented by rectangles with a single line.

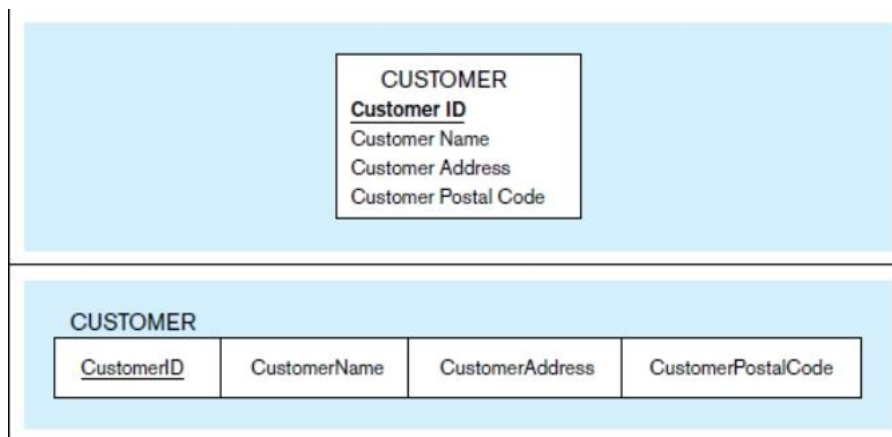
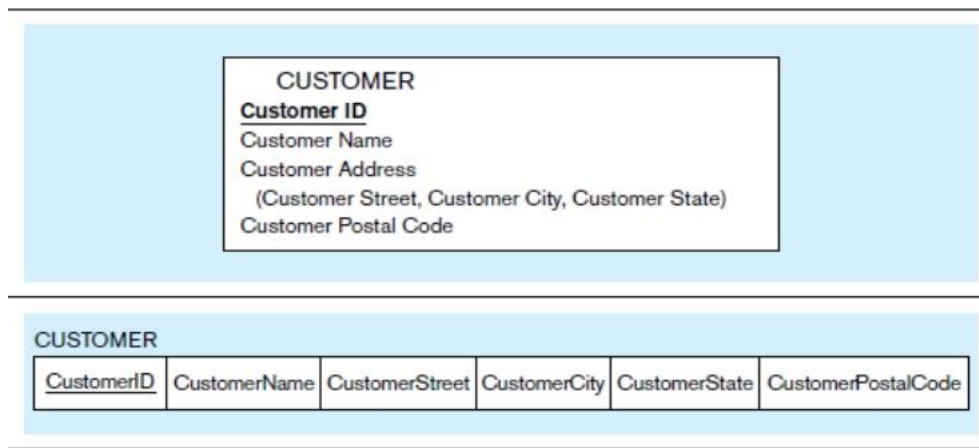


Figure 3.10: Transforming Regular Entities (with simple attributes)

Transforming Regular Entities (with composite attributes)

- When a regular entity type has a composite attribute, only the simple components of the composite attribute are included in the new relation as its attributes
- Customer Address is represented as a composite attribute with components Street, City, and State in Figure 3.11
- This entity is mapped to the CUSTOMER relation, which contains the simple address attributes.



Transforming Regular Entities (with multivalued attributes)

- When the regular entity type contains a multivalued attribute, two new relations are created.
- The **first relation** contains all of the attributes of the entity type except the multivalued attribute.
- The **second relation** contains two attributes that form the primary key of the second relation.
- The first of these attributes is the primary key from the first relation, which becomes a foreign key in the second relation, while the second is the multivalued attribute.
- **The name of the second relation should capture the meaning of the multivalued attribute** (Check Figure 3.12 for example).

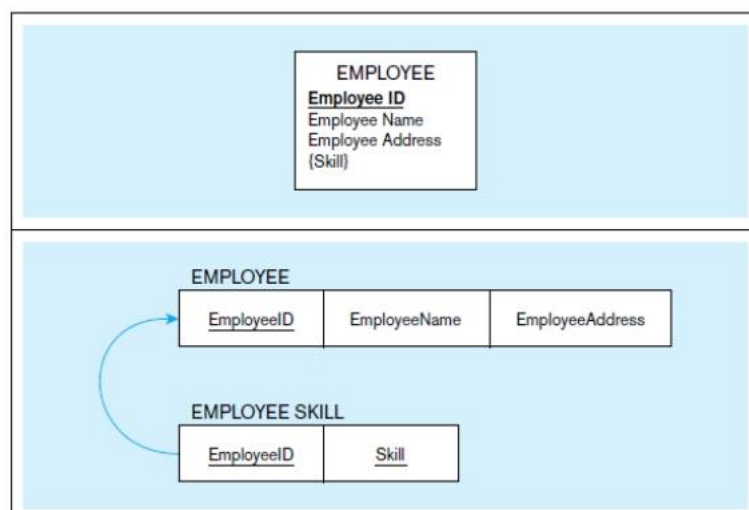


Figure 3.12: Transforming entity with multivalued attributes

Transforming EER-D (with weak entity)

- For each weak entity type, create a new relation and include all of the simple attributes (or simple components of composite attributes) as attributes of this relation.
- Then include the primary key of the identifying relation as a foreign key attribute in this new relation.
- The primary key of the new relation is the combination of this primary key of the identifying relation and the partial identifier of the weak entity type.

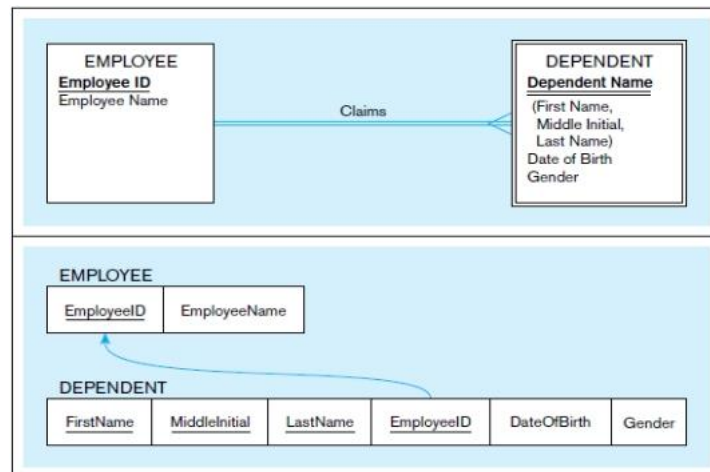


Figure 3.13: Transforming EER-D with a weak entity

- In practice, an alternative approach is often used to simplify the primary key of the DEPENDENT relation: Create a new attribute (called DependentID), which will be used as a surrogate primary key in Figure 3.13.
- With this approach, the relation DEPENDENT has the following attributes:

DEPENDENT(DependentID, EmployeeID, FirstName, MiddleInitial, LastName, DateOfBirth, Gender)

Transforming Binary One-to-many (1:M) Relationships

- First, create a relation for each of the two entity types participating in the relationship.
- Include the primary key attribute (or attributes) of the entity on the one side of the relationship as a foreign key in the relation that is on the many side of the relationship.
- Hint: The primary key migrates to the many side.

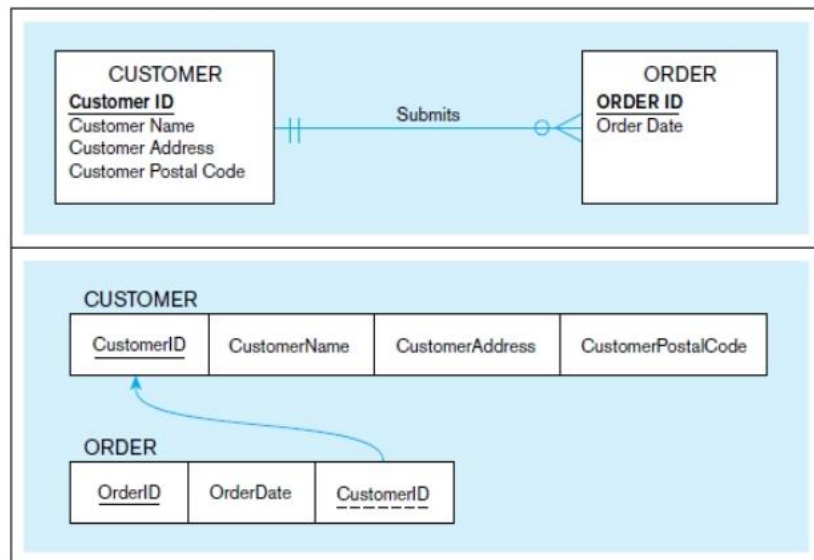


Figure 3.14: Transforming (1:M) binary Relationship

Transforming Binary Many-to-Many (M:N) Relationships

- Suppose that there is a binary many-to-many (M:N) relationship between two entity types, A and B.
- For such a relationship, create a new relation, C.
- Include as foreign key attributes in C the primary key for each of the two participating entity types.
- These attributes together become the primary key (Composite key) of C.
- Any non-key attributes that are associated with the M:N relationship are included with the relation C.

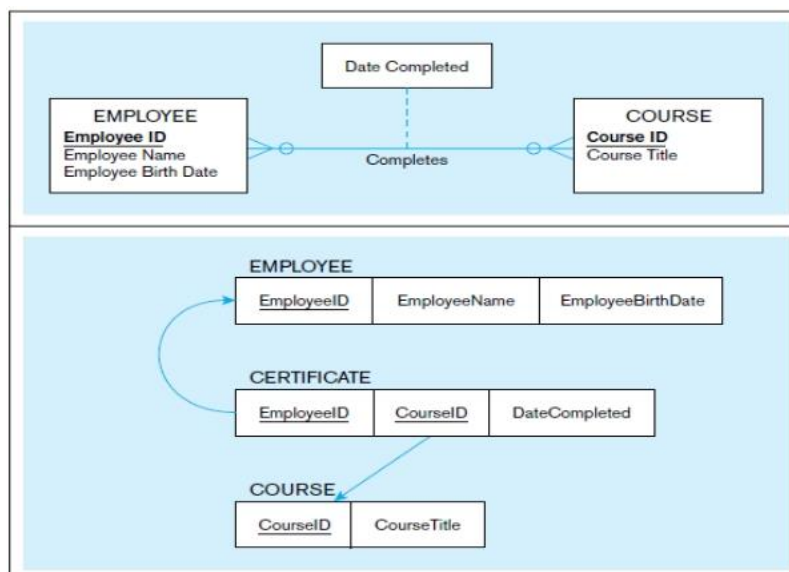


Figure 3.14: Transforming Binary Many-to-Many (M:N) Relationships

3.2.3 Transforming Binary one-to-one (1:1) Relationships

- Binary one-to-one relationships can be viewed as a special case of one-to-many relationships.
- The process of mapping such a relationship to relations requires two steps.
- First, two relations are created, one for each of the participating entity types.
- Second, the primary key of one of the relations is included as a foreign key in the other relation.
- **Note:** include in the relation on the optional side of the relationship a foreign key referencing the primary key of the entity type that has the mandatory participation in the 1:1 relationship.

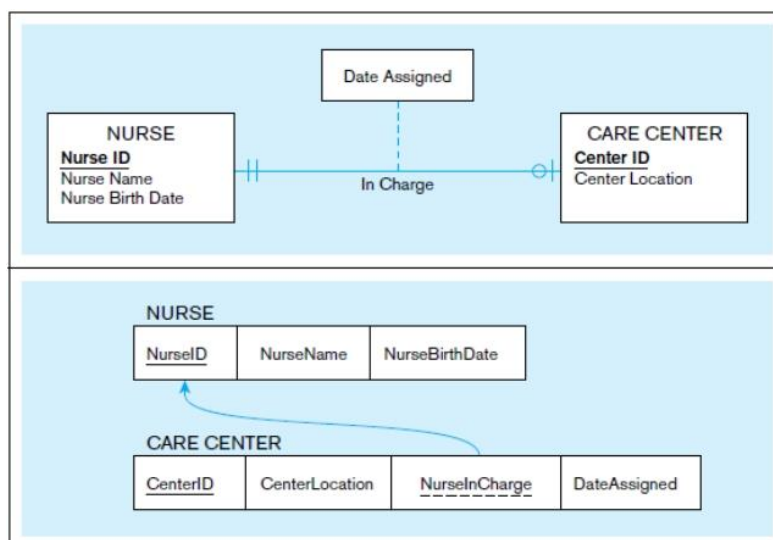


Figure 3.15: Transforming a Binary one-to-one (1:1) Relationship

3.2.4 Transforming Binary one-to-one (1:1) Relationships

- The first step is to create three relations: one for each of the two participating entity types and a third for the associative entity.
- The relation formed from the associative entity is called **the associative relation**.
- The second step then depends on whether an identifier was assigned to the associative entity in the E-R diagram.
- **Note:** include in the relation on the optional side of the relationship a foreign key referencing the primary key of the entity type that has the mandatory participation in the 1:1 relationship.

3.2.5 Mapping Associative Entity Type (with no identifier)

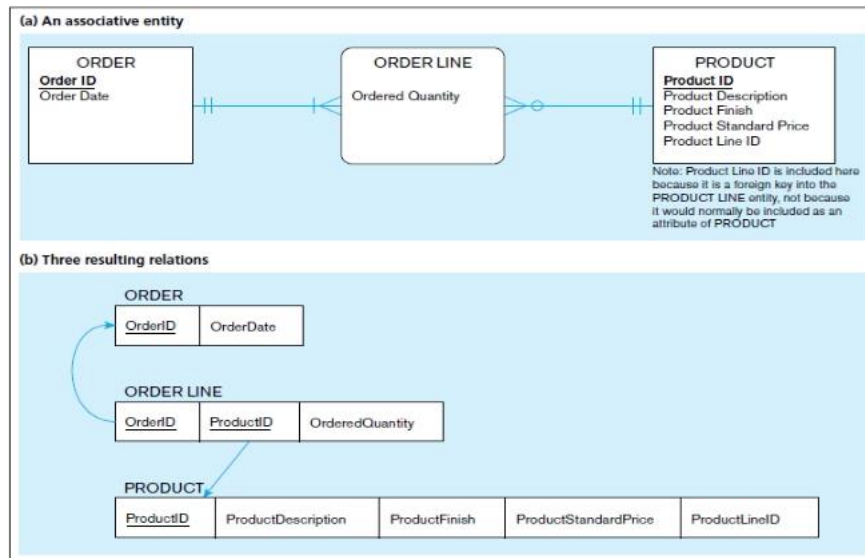


Figure 3.16: Mapping Associative Entity Type with NO identifier

Mapping Associative Entity Type (with identifier)

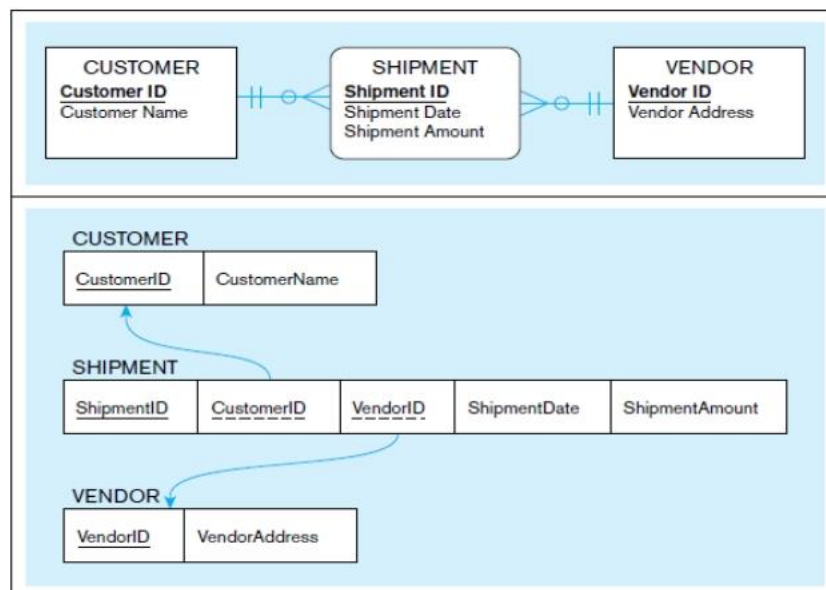


Figure 3.17: Mapping Associative Entity Type WITH identifier

3.2.6 Mapping Unary Relationships

- A unary relationship is a relationship between the instances of a single entity type.
- Unary relationships are also called **recursive relationships**.
- The two most important cases of unary relationships are one-to-many and many-to-many relationships.

Unary one-to-many Relationship

- The entity type in the unary relationship is mapped to a relation.
- Next, a foreign key attribute is added to the same relation; this attribute references the primary key values in the same relation. (This foreign key must have the same domain as the primary key.)
- This type of foreign key is called **a recursive foreign key**.
- A recursive foreign key is a foreign key in a relation that references the primary key values of the same relation.

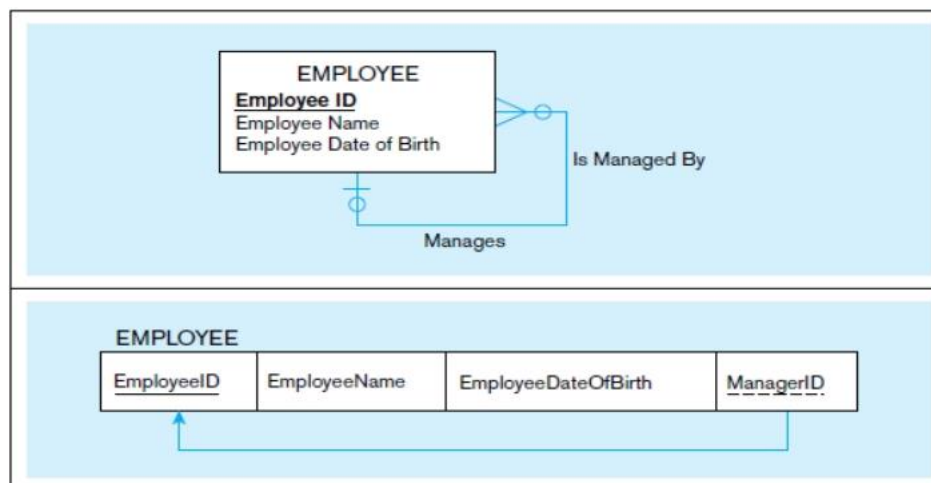


Figure 3.18: Unary one-to-many Relationship

Unary many-to-many Relationship

- With this type of relationship, two relations are created: one to represent the entity type in the relationship and an associative relation to represent the M:N relationship itself.
- The primary key of the associative relation consists of two attributes.
- These attributes (which need not have the same name) both take their values from the primary key of the other relation.
- Any non-key attribute of the relationship is included in the associative relation.

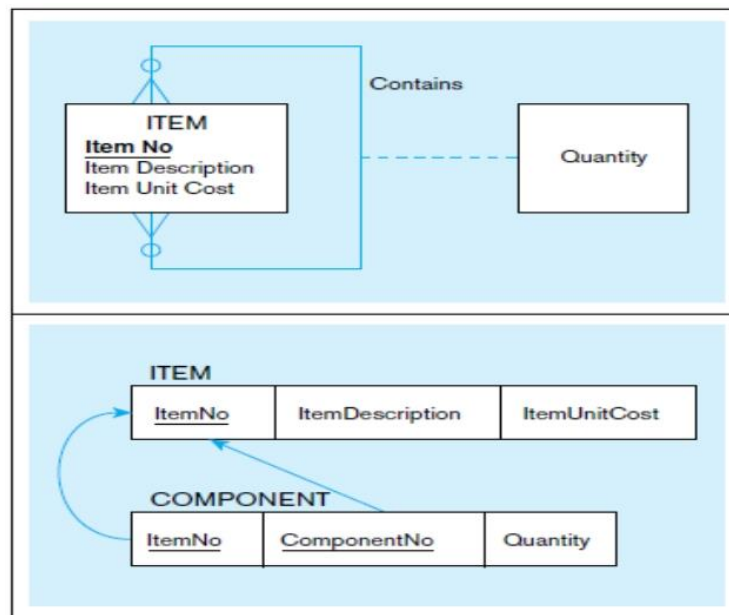


Figure 3.19: Unary many-to-many Relationship

- In Figure 3.19, the ITEM relation is mapped directly from the same entity type.
- COMPONENT is an associative relation whose primary key consists of two attributes that are arbitrarily named ItemNo and ComponentNo.
- The attribute Quantity is a non-key attribute of this relation that, for a given item, records the quantity of a particular component item used in that item.
- Notice that both ItemNo and ComponentNo reference the primary key (ItemNo) of the ITEM relation.
- It is not unusual to give this relation a surrogate key to avoid any practical complexities related to the composite key.
- These attributes then act in the role of foreign keys that reference the individual primary keys of the participating entity types.
- Any attributes of the associative entity type become attributes of the new relation.

3.2.7 Map Ternary (and n-ary) Relationships

- It is recommended that you convert a ternary relationship to an associative entity to represent participation constraints more accurately.
- To map an associative entity, type that links three regular entity types, create a new associative relation.
- The default primary key of this relation consists of the three primary key attributes for the participating entity types. (In some cases, additional attributes are required to form a unique primary key.)

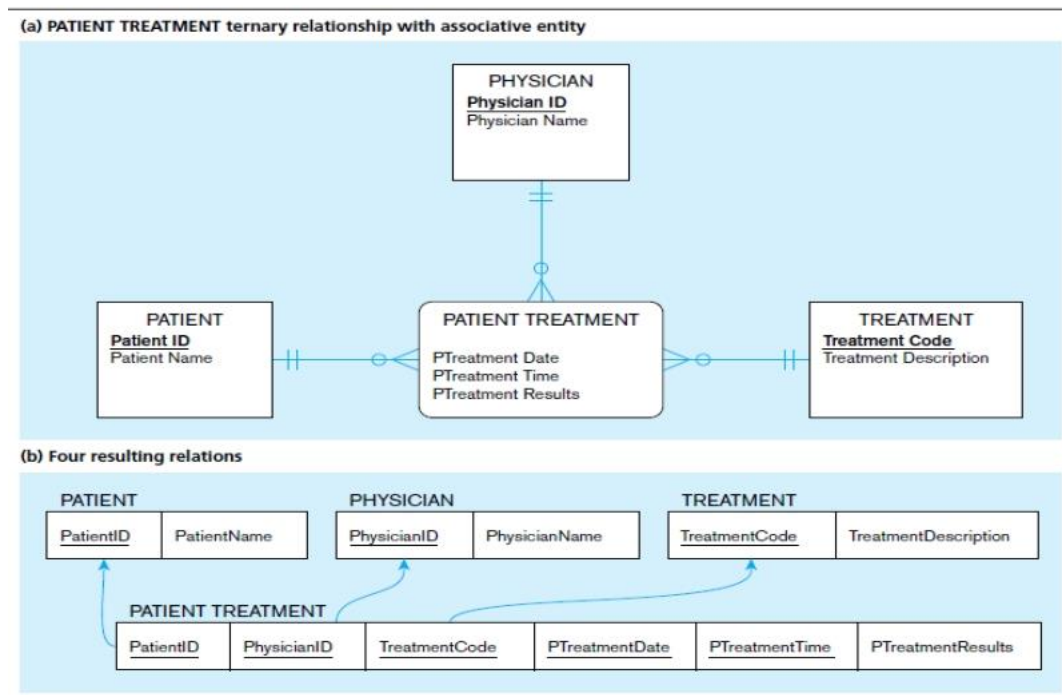


Figure 3.20: Mapping Ternary Relationships

- Although this primary key is technically correct, it is complex, difficult to manage, and prone to errors.
- A better approach is to introduce a surrogate key, such as PTreatmentID, that is, a serial number that uniquely identifies each treatment.

Map Supertype/Subtype Relationships

A commonly employed strategy to support the supertype/subtype relationship is:

1. Create a separate relation for the supertype and for each of its subtypes.
2. Assign to the relation created for the supertype the attributes that are common to all members of the supertype, including the primary key.
3. Assign to the relation for each subtype the primary key of the supertype and only those attributes that are unique to that subtype.
4. Assign one (or more) attributes of the supertype to function as the subtype discriminator.

An example of applying this strategy is shown in Figure 3:21

- Supertype:
 - EMPLOYEE
- Subtypes:
 - HOURLY EMPLOYEE,
 - SALARIED EMPLOYEE,

- CONSULTANT.
- The primary key of EMPLOYEE is Employee Number, and the attribute Employee Type is the subtype discriminator.

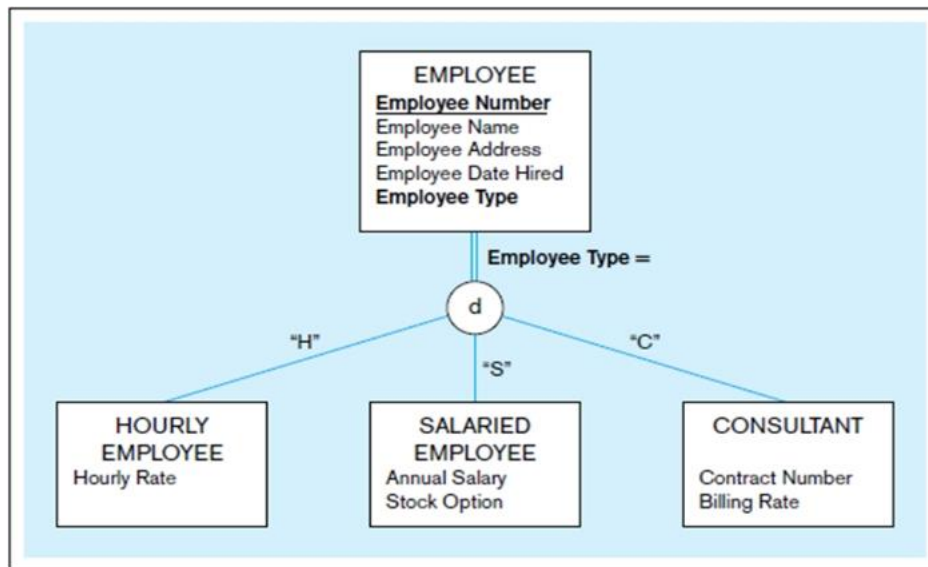


Figure 3.21: Subtype/Supertype Relationship

- The result of mapping the diagram in Figure 3.21 to relations using these rules is shown in Figure 3.21.
- There is one relation for the supertype (EMPLOYEE) and one for each of the three subtypes.
- The primary key for each of the four relations is Employee Number.
- A prefix is used to distinguish the name of the primary key for each subtype.

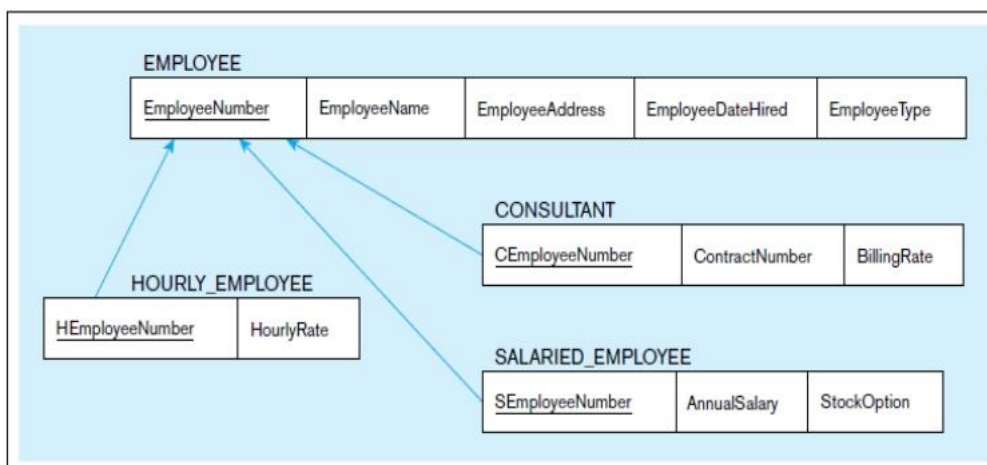


Figure 3.22: Mapping supertype/subtype Relationship

- For example, SemployeeNumber is the name for the primary key of the relation SALARIED EMPLOYEE.

- Each of these attributes is a foreign key that references the supertype primary key, as indicated by the arrows in the diagram.
- Each subtype relation contains only those attributes unique to the subtype.

3.2.8 Class Activities

1. **Figure 3.22** shows an EER diagram for a simplified credit card environment. There are two types of card accounts: debit cards and credit cards. Credit card accounts accumulate charges with merchants. Each charge is identified by the date and time of the charge, as well as the merchant and credit card primary keys.
 - Develop a relational schema.

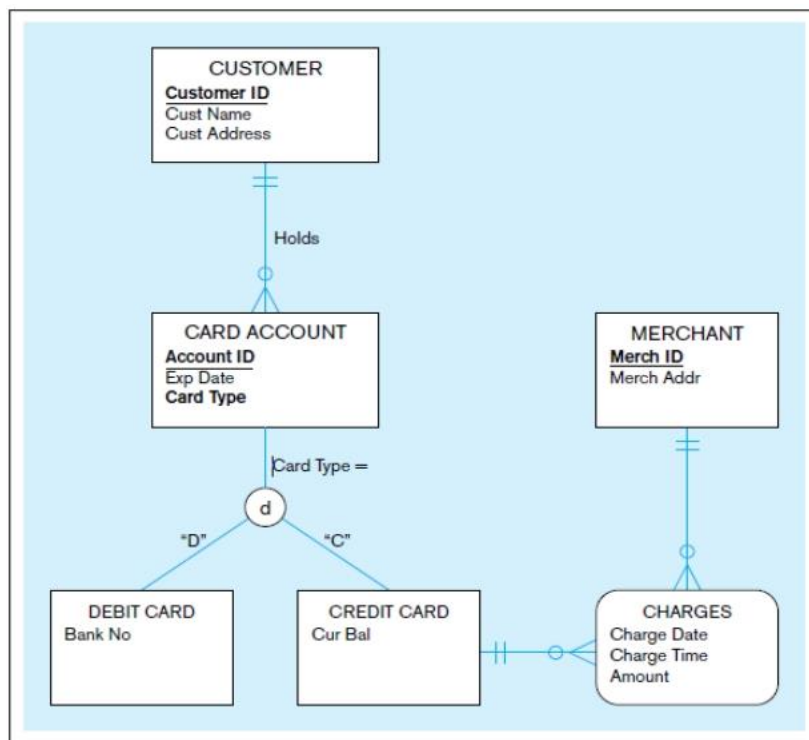


Figure 3.22: EER diagram for bank cards

2. **Figure 3.23** shows an EER diagram for Vacation Property Rentals. This organisation rents preferred properties in several states. As shown in the figure, there are two basic types of properties: beach and mountain.
 - Transform the EER diagram to a set of relations and develop a relational schema.

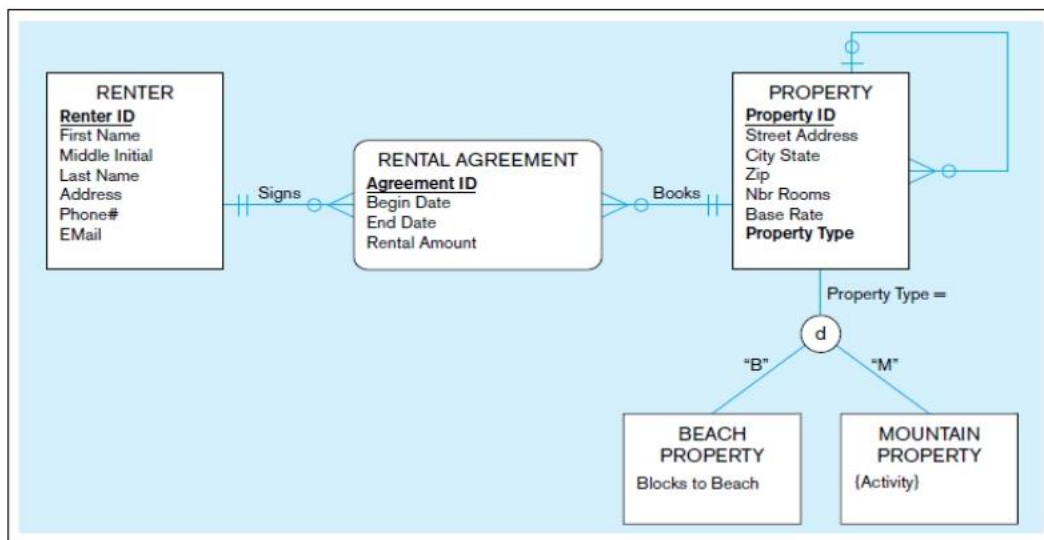


Figure 3.23: EER diagram for Vacation Property Rental

3.2.9 Conclusion

In this unit, you have learned how to transform a conceptual data model into a logical design suitable for implementation in a relational database. You explored how to convert regular entities into relational tables, assign keys, and maintain data structure and relationships. This transformation process ensures that the database design is both practical and faithful to the original business requirements.

Summary

In this unit, you have learnt that transforming a conceptual model into a logical design is a key step in database development. It involves converting entities, attributes, and relationships from an ER diagram into relational tables with well-defined structures. You also learnt how regular entities are transformed by identifying appropriate primary keys and organising data into a logical format that preserves the meaning and integrity of the original model.

Self-Assessment Questions

1. What is the difference between a conceptual model and a logical model in database design?
2. Explain how a regular entity with a composite attribute is transformed into a relational table.

3. How are multivalued attributes handled during the transformation process?
4. What is the purpose of using a surrogate key in complex relationships?
5. Describe the steps involved in transforming a binary many-to-many (M:N) relationship.
6. How is a unary one-to-many relationship represented in a relational schema?
7. Explain how to transform a supertype/subtype relationship into relational tables

Tutor-Marked Assignment

1. What is the difference between a conceptual model and a logical model in database design?
2. Explain how a regular entity with a composite attribute is transformed into a relational table.
3. How are multivalued attributes handled during the transformation process?
4. What is the purpose of using a surrogate key in complex relationships?
5. Describe the steps involved in transforming a binary many-to-many (M:N) relationship.
6. How is a unary one-to-many relationship represented in a relational schema?
7. Explain how to transform a supertype/subtype relationship into relational tables.

References/Further Readings

- Ramakrishnan, R., Gehrke, J.: **Database Management Systems**. USA: McGraw-Hill Companies (2000)
- Elmasri, R., & Navathe, S. (2000). **Fundamentals of database systems (3rd ed.)**. Reading, Mass.: Addison-Wesley.
- Hoffer, J. A., Prescott, M. B., & McFadden, F. R. (2005). **Modern database management**. Upper Saddle River, N.J: Pearson/ Prentice Hall.
- Connolly, T. M., & Begg, C. E. (2002). **Database systems: A practical approach to design, implementation, and management**. Harlow, England: Addison-Wesley
- <http://pages.cs.wisc.edu/~dbbook/openAccess/thirdEdition/solutions/ans3ed-oddonly.pdf>
- <https://gyires.inf.unideb.hu/GyBITT/03/>

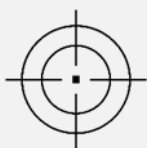
Module Four

Functional Dependencies

Unit 1 — Normalisation

INTRODUCTION

In this unit, you will learn about the principles and process of database normalisation. You will be introduced to standard forms, starting from the First Normal Form (1NF) and progressing to higher-order forms such as 2NF, 3NF, and Boyce-Codd Normal Form (BCNF). This unit also covers the concepts of functional dependency and determinants, which are essential for identifying relationships between attributes and for correctly structuring tables. Additionally, you will explore the role of relational keys in enforcing data integrity. The unit will also explain how data models are used during normalisation and outline the step-by-step process for transforming unnormalised relations into normalised structures.



LEARNING OUTCOMES

At the end of this unit, you should be able to:

- Understand the purpose and importance of normalisation in database design.
- Identify and explain different normal forms (1NF, 2NF, 3NF, BCNF).
- Understand the concepts of functional dependency and determinants.
- Recognise and apply different types of relational keys in normalisation.
- Use data models to guide the normalisation process.

4.1.1 Introduction to Normalisation

- Normalisation is the process of decomposing relations with anomalies to produce smaller, well-structured relations.
- It is a formal process for deciding which attributes should be grouped together in a relation so that all anomalies are removed.
- Normalisation is based on normal forms and functional dependencies, which define business rules, not data usage.
- Normalisation is a logical data-modelling technique used to ensure that data are well-structured from an organisation-wide perspective.
- There are two major occasions during the overall database development process when you can usually benefit from using normalisation:
 - During logical database design, you should use normalisation concepts to verify the quality of the relations that are obtained from mapping E-R diagrams.
 - When reverse-engineering older systems, Many of the tables and user views for older systems are redundant and subject to anomalies.

Goals of Normalisation

1. Minimise data redundancy to avoid anomalies and conserve storage space.
2. Simplify the enforcement of referential integrity constraints.
3. Make it easier to maintain data (insert, update, and delete).
4. Provide a better design that is an improved representation of the real world and a stronger basis for future growth.

Steps in Normalisation

- Normalisation can be accomplished and understood in stages, each of which corresponds to a normal form.
- A normal form is a state of a relation that requires that certain rules regarding relationships between attributes (or functional dependencies) are satisfied.

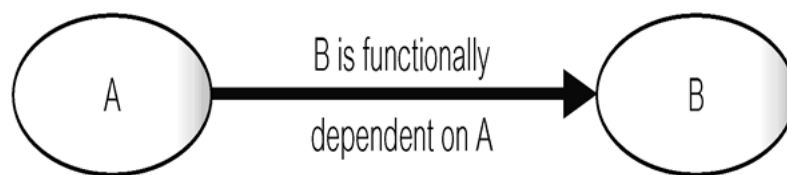
Normal Forms

1. **First normal form:** any multivalued attributes (also called repeating groups) have been removed, so there is a single value (possibly null) at the intersection of each row and column of the table
2. **Second normal form:** any partial functional dependencies have been removed (i.e., nonkey attributes are identified by the whole primary key).

3. **Third normal form:** any transitive dependencies have been removed (i.e., non-key attributes are identified by only the primary key).
4. **Boyce-Codd normal form:** Any remaining anomalies that result from functional dependencies have been removed (because there was more than one possible primary key for the same non-keys).

Functional Dependencies

- Normalisation is based on the analysis of functional dependencies.
- A functional dependency is a constraint between two attributes in which the value of one attribute is determined by the value of another attribute.
- Functional dependency could be full, partial or transitive.
- For any relation R, attribute B is functionally dependent on attribute A if, for every valid instance of A, that value of A uniquely determines the value of B.
- The functional dependency of B on A is represented by an arrow, as follows: $A \rightarrow B$.



Characteristics of functional dependencies used in normalisation

- Have a 1:1 relationship between attribute(s) on left and right-hand side of a dependency;
- Hold for all time;
- Are nontrivial.

Types of functional dependencies

Partial Functional Dependency

A Partial functional dependency is a functional dependency in which one or more non-key attributes are functionally dependent on part (but not all) of the primary key.

Full Functional Dependency

A full functional dependency exist when all non-key attributes are functionally dependent on the primary key(s).

Transitive Functional Dependency

A condition where A, B, C are attributes of a relation such that if $A \rightarrow B$ and $B \rightarrow C$, then C is transitively dependent on A through B. (Provided that A is not functionally dependent on B or C).

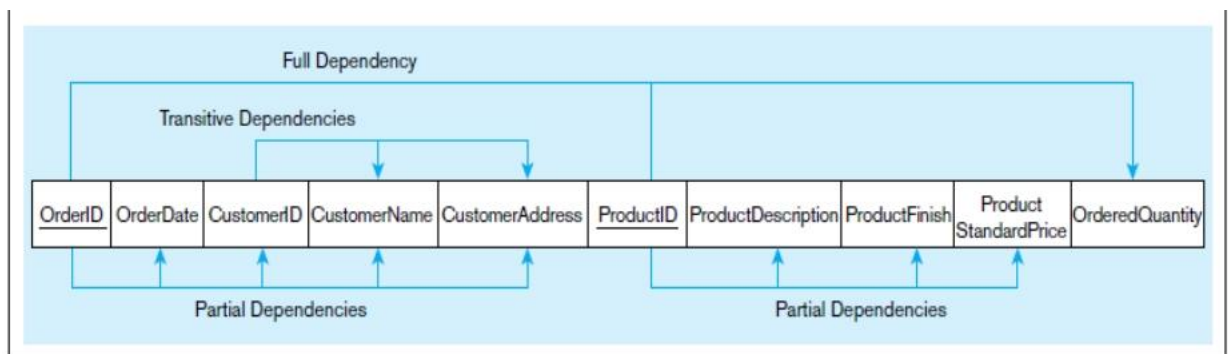


Figure 4.1: Functional dependency diagram for INVOICE

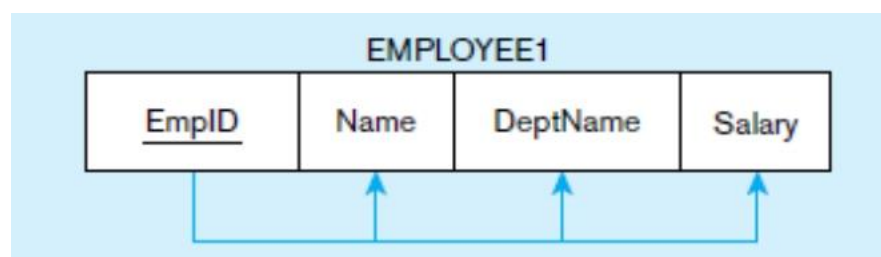
For the relation EMPLOYEE2, EmpID does not uniquely identify a row in the relation. For example, there are two rows in Table 4.1 for EmpID 100. There are two types of functional dependencies in this relation:

$\text{EmpID} \rightarrow \text{Name, DeptName, Salary}$

$\text{EmpID, CourseTitle} \rightarrow \text{DateCompleted}$

EMPLOYEE2					
EmpID	Name	DeptName	Salary	CourseTitle	DateCompleted
100	Margaret Simpson	Marketing	48,000	SPSS	6/19/2015
100	Margaret Simpson	Marketing	48,000	Surveys	10/7/2015
140	Alan Beeton	Accounting	52,000	Tax Acc	12/8/2015
110	Chris Lucero	Info Systems	43,000	Visual Basic	1/12/2015
110	Chris Lucero	Info Systems	43,000	C++	4/22/2015
190	Lorenzo Davis	Finance	55,000		
150	Susan Martin	Marketing	42,000	SPSS	6/19/2015
150	Susan Martin	Marketing	42,000	Java	8/12/2015

Figure 4.1a: Functional dependencies sample1



$\text{EmpID} \rightarrow \text{Name}, \text{DeptName}, \text{Salary}$

EMPLOYEE1			
EmpID	Name	DeptName	Salary
100	Margaret Simpson	Marketing	48,000
140	Allen Beeton	Accounting	52,000
110	Chris Lucero	Info Systems	43,000
190	Lorenzo Davis	Finance	55,000
150	Susan Martin	Marketing	42,000

Figure 4.1b: Functional dependencies sample1

An attribute may be functionally dependent on a combination of two (or more) attributes rather than on a single attribute.

For example, consider this relation:

EMPCOURSE (EmpID, CourseTitle, DateCompleted)

<u>EmpID</u>	<u>CourseTitle</u>	DateCompleted
100	SPSS	6/19/2015
100	Surveys	10/7/2015
140	Tax Acc	12/8/2015
110	Visual Basic	1/12/2015
110	C++	4/22/2015
150	SPSS	6/19/2015
150	Java	8/12/2015

Figure 4.2: EMP-COURSE Relation

- We represent the functional dependency in this relation (in Figure 4.2) as follows:

$\text{EmpID, CourseTitle} \rightarrow \text{DateCompleted}$

- The comma between EmpID and CourseTitle stands for the logical AND operator, because DateCompleted is functionally dependent on EmpID and CourseTitle in combination.
- The functional dependencies (in Figure 4.3) indicate that the combination of EmpID and CourseTitle is the only candidate key (and therefore the primary key) for EMPLOYEE2.
- In other words, the primary key of EMPLOYEE2 is a composite key.
- Neither EmpID nor CourseTitle uniquely identifies a row in this relation and therefore (according to property 1) cannot by itself be a candidate key.

(b) Functional dependencies in EMPLOYEE2

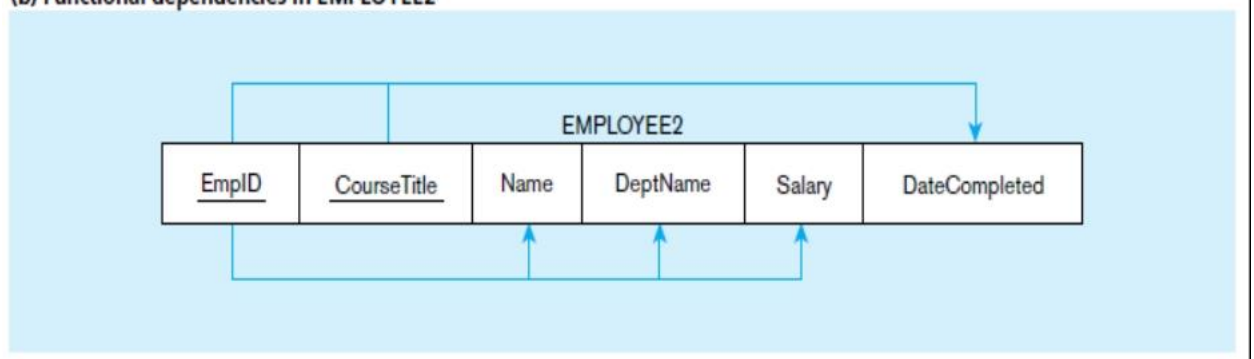


Figure 4.3: Functional Dependencies in EMPLOYEE2

Examples of functional dependencies

1. SSN $\rightarrow$ Name, Address, Birthdate

A person's name, address, and birth date are functionally dependent on that person's Social Security number (in other words, there can be only one Name, one Address, and one Birthdate for each SSN).

2. VIN $\rightarrow$ Make, Model, Colour

The make, model, and original colour of a vehicle are functionally dependent on the vehicle identification number (as above, there can be only one value of Make, Model, and Colour associated with each VIN).

3. ISBN $\rightarrow$ Title, FirstAuthorName, Publisher

The title of a book, the name of the first author, and the publisher are functionally dependent on the book's international standard book number (ISBN).

Determinants

The attribute on the left side of the arrow in a functional dependency is called a determinant.

SSN, VIN, and ISBN are determinants in the previous three examples.

In the EMP COURSE relation, the combination of EmpID and CourseTitle is a determinant.

<u>EmpID</u>	<u>CourseTitle</u>	DateCompleted
100	SPSS	6/19/2015
100	Surveys	10/7/2015
140	Tax Acc	12/8/2015
110	Visual Basic	1/12/2015
110	C++	4/22/2015
150	SPSS	6/19/2015
150	Java	8/12/2015

EmpID, CourseTitle $\rightarrow$ DateCompleted

4.1.2 Relational Keys

One or more attributes that determine other attributes are called the key attributes. Keys may be Single or Composite (composed of several key attributes).

1. Superkey /Primary Key
2. Candidate Key/Composite Key
 - A primary key that contains more than one attribute of which no proper subset is a super key within the relation.
 - Uniqueness: In each tuple of R, values of K uniquely identify that tuple
 - Irreducibility: no proper subset of K has the uniqueness property
3. Alternate Keys
 - Candidate keys/Composite keys that are not selected to be the primary key.
4. Foreign Key

An attribute in a relation of a database that serves as the primary key of another relation in the same database.

Candidate Keys

A candidate key is an attribute, or combination of attributes, that uniquely identifies a row in a relation. A candidate key must satisfy the following properties, which are a subset of the six properties of a relation.

- Unique identification: For every row, the value of the key must uniquely identify that row. This property implies that each non-key attribute is functionally dependent on that key.
- Non-redundancy: No attribute in the key can be deleted without destroying the property of unique identification.

Relationship between Candidate Key and Determinant

- A candidate key is always a determinant, whereas a determinant may or may not be a candidate key.
- For example, in EMPLOYEE2, EmpID is a determinant but not a candidate key.
- A candidate key is a determinant that uniquely identifies the remaining (nonkey) attributes in a relation.
- A determinant may be

- a candidate key (such as EmpID in EMPLOYEE1),
- part of a composite candidate key (such as EmpID in EMPLOYEE2), or a nonkey attribute.

4.1.3 Data Models for Normalisation

If an EER data model has been transformed into a comprehensive set of relations for the database, then each of these relations needs to be normalised. In other cases in which the logical data model is being derived from user interfaces, such as screens, forms, and reports, you will want to create relations for each user interface and normalise those relations.

Check out the Invoice (PineValley Furniture Company) below.

PVFC Customer Invoice					
Customer ID	2	Order ID	1006		
Customer Name	Value Furniture	Order Date	10/24/2015		
Address	15145 S.W. 17th St. Plano TX 75022				
Product ID	Product Description	Finish	Quantity	Unit Price	Extended Price
7	Dining Table	Natural Ash	2	\$800.00	\$1,600.00
5	Writer's Desk	Cherry	2	\$325.00	\$650.00
4	Entertainment Center	Natural Maple	1	\$650.00	\$650.00
				Total	\$2,900.00

Figure 4.4: Customer invoice from Pine Valley Furniture Company

The process of Normalisation

- Formal technique for analysing a relation based on its primary key and functional dependencies between its attributes.
- Often executed as a series of steps.
- Each step corresponds to a specific normal form, which has known properties.
- As normalisation proceeds, relations become progressively more restricted (stronger) in format and also less vulnerable to update anomalies.

Unnormalised Form (UNF)

A table that contains one or more repeating groups. To create an unnormalized table: transform data from an information source (e.g., a form) into a table format with columns and rows.

4.1.4 Steps in Normalising a data model

Step 0

- The first step (preliminary to normalisation) is to represent the user view (in this case, an invoice) as a single table, or relation, with the attributes recorded as column headings.
- Sample data should be recorded in the rows of the table, including any repeating groups that are present in the data.

<u>OrderID</u>	<u>Order Date</u>	<u>Customer ID</u>	<u>Customer Name</u>	<u>Customer Address</u>	<u>ProductID</u>	<u>Product Description</u>	<u>Product Finish</u>	<u>Product StandardPrice</u>	<u>Ordered Quantity</u>
1006	10/24/2015	2	Value Furniture	Plano, TX	7	Dining Table	Natural Ash	800.00	2
					5	Writer's Desk	Cherry	325.00	2
					4	Entertainment Center	Natural Maple	650.00	1
1007	10/25/2015	6	Furniture Gallery	Boulder, CO	11	4-Dr Dresser	Oak	500.00	4
					4	Entertainment Center	Natural Maple	650.00	3

Figure 4.5: Outcome of step 0 (using the invoice from Pine Valley Furniture Company)

Convert to First Normal Form: Step 1

- A relation is in first normal form (1NF) if the following two constraints both apply:
 - There are no repeating groups in the relation
 - There is a single fact at the intersection of each row and column of the table.
 - A primary key has been defined, which uniquely identifies each row in the relation.

OrderID	Order Date	Customer ID	Customer Name	Customer Address	ProductID	Product Description	Product Finish	Product StandardPrice	Ordered Quantity
1006	10/24/2015	2	Value Furniture	Plano, TX	7	Dining Table	Natural Ash	800.00	2
1006	10/24/2015	2	Value Furniture	Plano, TX	5	Writer's Desk	Cherry	325.00	2
1006	10/24/2015	2	Value Furniture	Plano, TX	4	Entertainment Center	Natural Maple	650.00	1
1007	10/25/2015	6	Furniture Gallery	Boulder, CO	11	4-Dr Dresser	Oak	500.00	4
1007	10/25/2015	6	Furniture Gallery	Boulder, CO	4	Entertainment Center	Natural Maple	650.00	3

Figure 4.6: Outcome of step 1 (using the invoice relation (1NF) from Pine Valley Furniture Company)

First Normal Form

Remove Repeating Groups

- As you can see, the invoice data contains a repeating group for each product that appears on a particular order.
- Thus, OrderID 1006 contains three repeating groups, corresponding to the three products on that order.

Select the Primary Key

- There are four determinants in INVOICE, and their functional dependencies are the following:
 - OrderID → OrderDate, CustomerID, CustomerName, CustomerAddress
 - CustomerID → CustomerName, CustomerAddress
 - ProductID → ProductDescription, ProductFinish, ProductStandardPrice
 - OrderID, ProductID → OrderedQuantity

Anomalies in 1NF

- Although repeating groups have been removed, the data still contain considerable redundancy.
- For example, CustomerID, CustomerName, and CustomerAddress for Value Furniture are recorded in three rows (at least) in the table.

- As a result of these redundancies, manipulating the data in the table can lead to insertion, deletion and update anomalies.
1. Insertion anomaly
 - Example: The Pine Valley Furniture Company cannot introduce a new product (say, Breakfast Table with ProductID 8) and add it to the database before it is ordered for the first time: No entries can be added to the table without both ProductID and OrderID.
 - Another example is, if a customer calls and requests that another product be added to his OrderID 1007, a new row must be inserted in which the order date and all of the customer information must be repeated.
 - This leads to data replication and potential data entry errors (e.g., the customer name may be entered as "Valley Furniture").
 2. Deletion Anomaly
 - If a customer calls and requests that the Dining Table be deleted from her OrderID 1006, this row must be deleted from the relation, and we lose the information concerning this item's finish (Natural Ash) and price (\$800.00).
 - Update Anomaly
 - If Pine Valley Furniture (as part of a price adjustment) increases the price of the Entertainment Center (ProductID 4) to \$750.00, this change must be recorded in all rows containing that item.

Second Normal Form; Step 2

- Step 2: Convert to Second Normal Form
- A relation is in second normal form (2NF) if it is in first normal form and contains no partial functional dependencies.
- A partial functional dependency exists when a non-key attribute is functionally dependent on part (but not all) of the primary key.
- The following partial dependencies exist in the INVOICE data for (Pine Valley Furniture Company):
 - OrderID → OrderDate, CustomerID, CustomerName, CustomerAddress
 - ProductID → ProductDescription, ProductFinish, ProductStandard-Price
- The first of these partial dependencies (for example) states that the date on an order is uniquely determined by the order number and has nothing to do with the ProductID.

- To convert a relation with partial dependencies to second normal form, the following steps are required:
 1. Create a new relation for each primary key attribute (or combination of attributes) that is a determinant in a partial dependency.
 2. That attribute is the primary key in the new relation.
 3. Move the non-key attributes that are only dependent on this primary key attribute (or attributes) from the old relation to the new relation.
 4. Removal of the partial dependencies results in the formation of two new relations: PRODUCT and CUSTOMER ORDER.
 5. The INVOICE relation is now left with just the primary key attributes (OrderID and ProductID) and OrderedQuantity, which is functionally dependent on the whole key.
 6. We rename this relation ORDER LINE, because each row in this table represents one line item on an order.

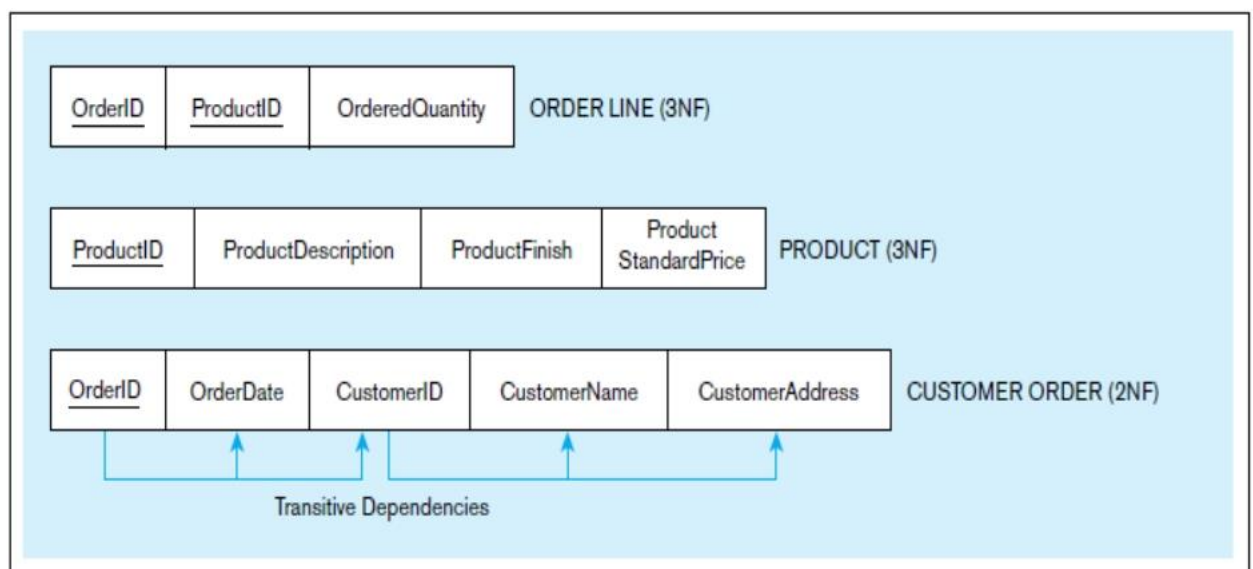


Figure 4.7: Outcome of step 2 (using the invoice relation (2NF, 3NF) from Pine Valley Furniture Company)

As indicated in Figure 4.7, the relations ORDER LINE and PRODUCT are in third normal form.

However, CUSTOMER ORDER contains transitive dependencies and therefore (although in second normal form) is not yet in third normal form.

- A relation that is in first normal form will be in second normal form if any one of the following conditions applies:
- The primary key consists of only one attribute (e.g., the attribute ProductID in the PRODUCT relation in Figure k).

- By definition, there cannot be a partial dependency in such a relation.
- No nonkey attributes exist in the relation (thus all of the attributes in the relation are components of the primary key).
- There are no functional dependencies in such a relation.
- Every non-key attribute is functionally dependent on the full set of primary key attributes (e.g., the attribute OrderedQuantity in the ORDER LINE relation in Figure k).

Towards Third Normal Form; Step 3

- Step 3: Convert to Third Normal Form
- A relation is in third normal form (3NF) if it is in second normal form and no transitive dependencies exist.
- A transitive dependency in a relation is a functional dependency between the primary key and one or more non-key attributes that are dependent on the primary key via another non-key attribute.
- For example, there are two transitive dependencies in the CUSTOMER ORDER relation shown in Figure 4.7:
 - $\text{OrderID} \rightarrow \text{CustomerID} \rightarrow \text{CustomerName}$
 - $\text{OrderID} \rightarrow \text{CustomerID} \rightarrow \text{CustomerAddress}$
- In other words, both customer name and address are uniquely identified by CustomerID, but CustomerID is not part of the primary key (as we noted earlier).
- Transitive dependencies create unnecessary redundancy that may lead to the type of anomalies discussed earlier.

Removing Transitive Dependencies

You can easily remove transitive dependencies from a relation by means of a three-step procedure:

- For each nonkey attribute (or set of attributes) that is a determinant in a relation, create a new relation.
 - That attribute (or set of attributes) becomes the primary key of the new relation.
 - Move all of the attributes that are functionally dependent only on the primary key of the new relation from the old to the new relation.
 - Leave the attribute that serves as a primary key in the new relation in the old relation to serve as a foreign key that allows you to associate the two relations.

- The results of applying these steps to the relation CUSTOMER ORDER are shown in Figure 4.9.
- A new relation named CUSTOMER has been created to receive the components of the transitive dependency.
- The determinant CustomerID becomes the primary key of this relation, and the attributes CustomerName and CustomerAddress are moved to the relation.

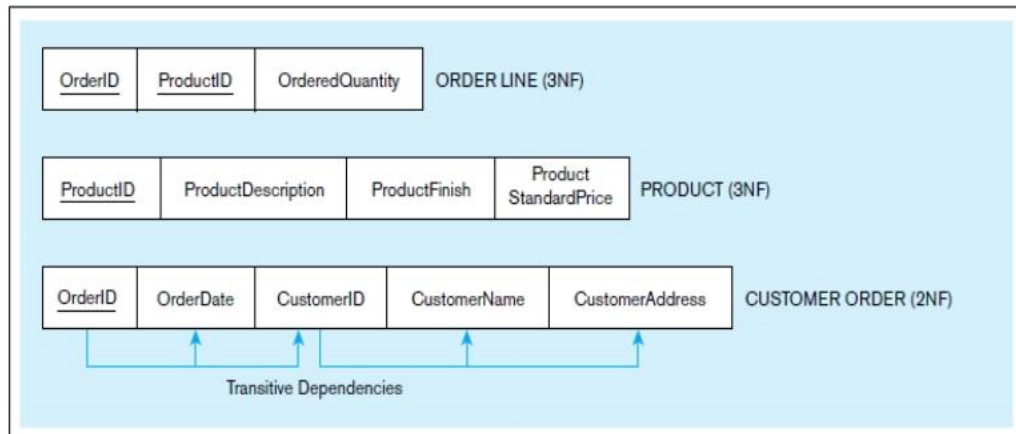


Figure 4.8: Customer Order Relation in 2NF

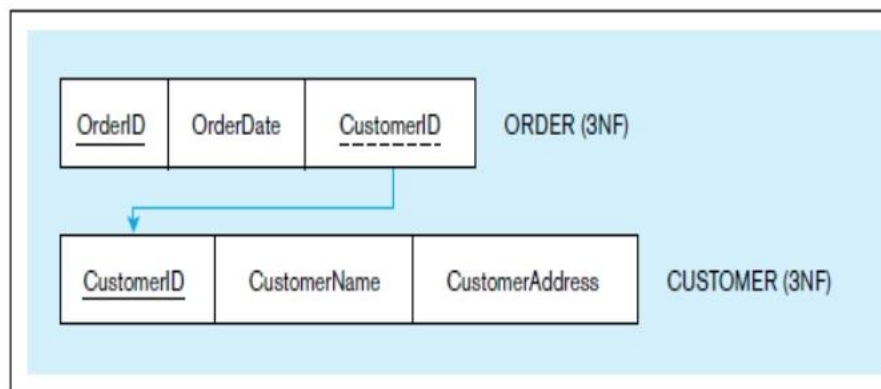


Figure 4.9: Customer Order Relation in 2NF

CUSTOMER ORDER is renamed ORDER, and the attribute CustomerID remains as a foreign key in that relation.

- This allows us to associate an order with the customer who submitted the order.
- As indicated in Figure 4.9, these relations are now in third normal form.

Relational schema for INVOICE data

Normalising the data in the INVOICE view has resulted in the creation of four relations in third normal form: CUSTOMER, PRODUCT, ORDER, and ORDER LINE.

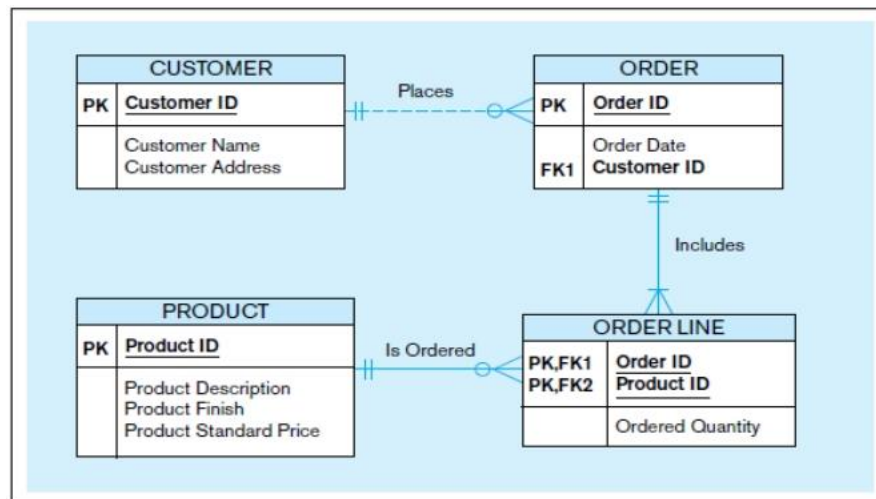


Figure 4.10: Relational schema for INVOICE data

Boyce–Codd Normal Form (BCNF)

Based on functional dependencies that take into account all candidate keys in a relation, BCNF also imposes additional constraints beyond the general definition of 3NF.

Normalising 3NF to BCNF

- A relation is in BCNF if and only if every determinant is a candidate key.
- Note: The attribute on the left side of the arrow in a functional dependence is called the determinant.
- The difference between 3NF and BCNF is that for a functional dependency $A \twoheadrightarrow B$,
- 3NF allows this dependency in a relation if B is a primary-key attribute and A is not a candidate key.
- Whereas BCNF insists that for this dependency to remain in a relation, A must be a candidate key.
- To test whether a relation is in BCNF, we identify all the determinants and make sure that they are candidate keys. Recall that a determinant is an attribute, or a group of attributes, on which some other attribute is fully functionally dependent.

Higher Order Normal forms

- Fourth normal form: Any multivalued dependencies have been removed.
- Fifth normal form: Any remaining anomalies have been removed.

Page 1

Dreamed House
Customer Rental Details

Date 7th April 2009

Customer Name John Kay
Customer Number CR76

Property Number	Property Address	Rent Start	Rent Finish	Rent	Owners Number	Owners Name
PG4	6, Lawrence St. Ikoyi	1-Jul-04	31-Aug-06	350	CO40	Tina Murphy
PG16	5, Novar St, Oshodi	1-Sep-06	1-Sep-08	450	CO93	Tony Shaw

Figure 4.11: Dream Home Customer Detail form

Customer_No	CName	Property_No	Paddress	RentStart	RentFinish	Rent	Owner_No	OName
CR76	John Kay	PG4	6, Lawrence St. Ikoyi	1-Jul-04	31-Aug-06	350	CO40	Tina Murphy
		PG16	5, Novar St, Oshodi	1-Sep-06	1-Sep-08	450	CO93	Tony Shaw
CR56	Fred Wey	PG4	6, Lawrence St. Ikoyi	1-Sep-02	10-Jun-04	350	CO40	Tina Murphy
		PG36	2, Fred St. Apapa	10-Oct-04	1-Dec-06	375	CO93	Tony Shaw
		PG16	5, Novar St, Oshodi	1-Jan-06	10-Aug-06	450	CO93	Tony Shaw

Table 3.1: Unnormalised Customer Rental Relation

Note:

- There are multiple values at intersection of certain rows and columns.
- To transform an unnormalised table to 1NF, we must ensure that there is a single value at the intersection of each row and column.
- So, 1NF is achieved as shown in the next table 2.

Table 3.2: 1st Normal Form(1NF) Customer Rental Relation

Customer_No	Property_No	CName	Paddress	RentStart	RentFinish	Rent	Owner_No	OName
CR76	PG4	John Kay	6, Lawrence St. Ikoyi	1-Jul-04	31-Aug-06	350	CO40	Tina Murphy
CR76	PG16	John Kay	5, Novar St, Oshodi	1-Sep-06	1-Sep-08	450	CO93	Tony Shaw
CR56	PG4	Fred Wey	6, Lawrence St. Ikoyi	1-Sep-02	10-Jun-04	350	CO40	Tina Murphy
CR56	PG36	Fred Wey	2, Fred St. Apapa	10-Oct-04	1-Dec-06	375	CO93	Tony Shaw
CR56	PG16	Fred Wey	5, Novar St, Oshodi	1-Jan-06	10-Aug-06	450	CO93	Tony Shaw

- We identify the three candidate keys for the Customer Rental Relation as being composite keys comprising:
 - (Customer_No, Property_No),
 - (Customer_No, RentStart),
 - (Property_No, RentStart)
- Among these, we select (Customer_No, Property_No) as the Primary Key for the relation.
- To normalise this relation to the 2nd Normal form, we need to identify the functional dependencies.

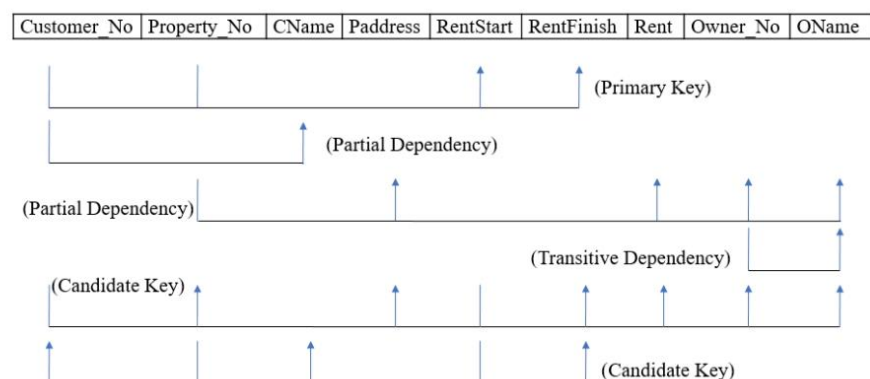


Figure 4.11: Dependency diagram for Customer_Rental relation

Functional dependencies of the Customer_Rental relation with Customer_No, Property_No as the primary key

- fd1: Customer_No, Property_No ® RentStart, RentFinish (Pk)
- fd2: Customer_No ® CName (Pd)
- fd3: Property_No ® Paddress, Rent, Owner_No, OName (Pd)
- fd4: Owner_No ® OName (Td)

- fd5: Customer_No , RentStart ® Property_No, Paddress, Rent , Owner_No, OName (Ck)
- fd6: Property_No, RentStart ® Customer_No, CName, RentFinish (Ck)

So, to decompose the customer_Rental relation to 2nd Normal form all the partial dependencies have to be removed on separate tables.

So as to have all non-key attribute to be fully dependent on their determinant key.

Table 4.3: 2nd Normal Form(2NF) Customer Rental Relation

Customer	(Customer_No, CName)
Rental	(Customer_No , Property_No , RentStart, RentFinish)
Property_Owner	(Property_No , Paddress, Rent, Owner_No, OName)

Customer Relation		Rental Relation			
Customer_No	CName	Customer_No	Property_No	RentStart	RentFinish
CR76	John Kay	CR76	PG4	1-Jul-04	31-Aug-06
CR56	Fred Wey	CR76	PG16	1-Sep-06	1-Sep-08
		CR56	PG4	1-Sep 02	10-Jun-04
		CR56	PG36	10-Oct-04	1-Dec-06
		CR56	PG16	1-Jan-06	10-Aug-06

Property_Owner Relation				
Property_No	Paddress	Rent	Owner_No	OName
PG4	6, Lawrence St. Ikoyi	350	CO40	Tina Murphy
PG16	5, Novar St, Oshodi	450	CO93	Tony Shaw
PG36	2, Fred St. Apapa	375	CO93	Tony Shaw

- To transform the decomposed relation to 3rd Normal form, we have to identify transitive dependency(ies), if there exist in any of the three decomposed relations.
- All the relations are in 3rd Normal form except property_Owner relation which consist of transitive dependency by considering fd4,
 - fd4: Owner_No® OName (Td)
- So, to decompose property_Owner to 3rd Normal form the transitive dependency attribute has to be removed as a separate relation with a copy of Owners_No as the Primary Key, to have:
 - Customer (Customer_No, CName)
 - Rental (Customer_No , Property_No , RentStart, RentFinish)
 - Property_for _Rent (Property_No , Paddress, Rent, Owner_No)
 - Owner (Owner_No, OName)

Table 4.4: 3rd Normal form (3NF) decomposed relations

Customer Relation		Rental Relation			
<i>Customer_No</i>	<i>CName</i>	<i>Customer_No</i>	<i>Property_No</i>	<i>RentStart</i>	<i>RentFinish</i>
CR76	John Kay	CR76	PG4	1-Jul-04	31-Aug-06
CR56	Fred Wey	CR76	PG16	1-Sep-06	1-Sep-08
		CR56	PG4	1-Sep-02	10-Jun-04
		CR56	PG36	10-Oct-04	1-Dec-06
		CR56	PG16	1-Jan-06	10-Aug-06

Property Relation				Owner Relation	
<i>Property_No</i>	<i>Paddress</i>	<i>Rent</i>	<i>Owner_No</i>	<i>Owner_No</i>	<i>OName</i>
PG4	6, Lawrence St. Ikoyi	350	CO40	CO40	Tina Murphy
PG16	5, Novar St, Oshodi	450	CO93	CO93	Tony Shaw
PG36	2, Fred St. Apapa	375	CO93		

- To transform the decomposed relation to Boyce Codd Normal form, we have to identify if there is any case where a determinant is not a candidate key Table 3.4.
- Table 4.4 reveals that all decomposed relation are in 3rd Normal form and non-violate Boyce Codd condition:
- fd1: Customer_No, Property_No ® RentStart, RentFinish (*Pk*)
- fd5: Customer_No , RentStart ® Property_No, Paddress, Rent , Owner_No, OName (*Ck*)
- fd6: Property_No, RentStart ® Customer_No, CName, RentFinish (*Ck*)
- As the three determinants of the Rental relation are also candidate keys, the Rental relation is also already in BCNF. Violation of BCNF is quite rare.

4.1.5 Class Activities

Using the property inspection report below, generate the UNF, INF, 2NF and BCNF requirements.

DreamHome Property Inspection Report					
DreamHome Property Inspection Report					
Property Number <u>PG4</u>					
Property Address <u>6 Lawrence St, Glasgow</u>					
Inspection Date	Inspection Time	Comments	Staff no	Staff Name	Car Registration
18-Oct-00	10.00	Need to replace crockery	SG37	Ann Beech	M231 JGR
22-Apr-01	09.00	In good order	SG14	David Ford	M533 HDR
1-Oct-01	12.00	Damp rot in bathroom	SG14	David Ford	N721 HFR

Page 1

4.1.6 Conclusion

In this unit, you have learned how to transform a conceptual data model into a logical design. In this unit, you have learned how normalisation helps improve the structure and quality of a database by reducing redundancy and organising data into well-defined tables. You explored various normal forms, understood the role of functional dependencies and relational keys, and followed practical steps to normalise data.

Summary

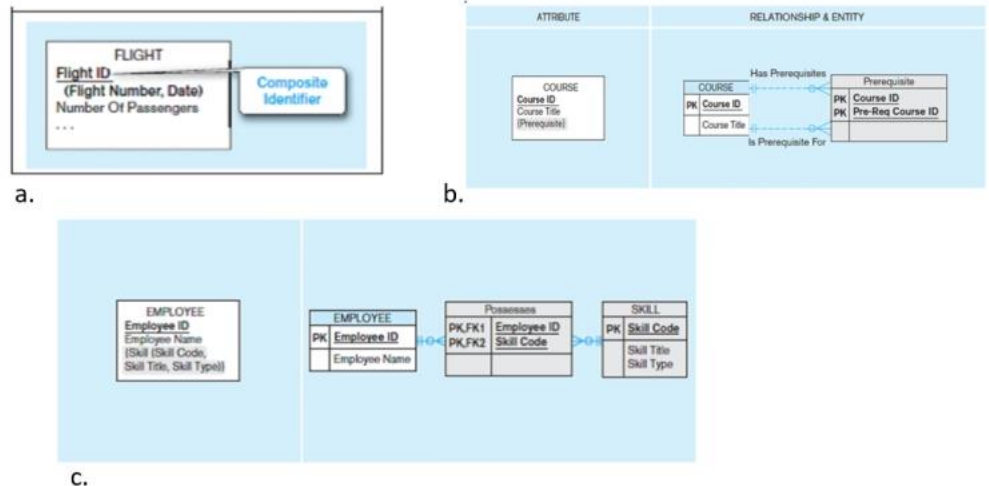
In this unit, you have learned that normalisation is a key technique in database design used to organise data efficiently and reduce redundancy. You explored different normal forms, from 1NF to higher-order forms, and learned how they help create clean, consistent tables. You also learnt about functional dependencies, determinants, and the role of relational keys in ensuring data integrity. By following the steps of normalisation, you can transform poorly structured data into well-organised relations that support reliable and efficient database systems.

Self-Assessment Questions

1. What is normalisation, and why is it important in relational database design?
2. Differentiate between First Normal Form (1NF) and Third Normal Form (3NF).
3. Explain the concept of functional dependency and how it is used in the normalisation process

Tutor-Marked Assignment

- For each of the following ER diagrams;
 - Transform the diagram to a relational schema that shows referential integrity constraints.
 - For each relation, diagram the functional dependencies.
 - If any of the relations are not in 3NF, transform them to 3NF. What is the difference between a conceptual model and a logical model in database design?



73

- For each of the following relations, indicate the normal form for that relation. If the relation is not in third normal form, decompose it into 3NF relations. Functional dependencies (other than those implied by the primary key) are shown where appropriate.
 - EMPLOYEE(EmployeeNo, ProjectNo)
 - EMPLOYEE(EmployeeNo, ProjectNo, Location)
 - EMPLOYEE(EmployeeNo, ProjectNo, Location, Allowance)
 - [FD: Location $\rightarrow$ Allowance]
 - EMPLOYEE(EmployeeNo, ProjectNo, Duration, Location, Allowance)
 - [FD: Location $\rightarrow$ Allowance; FD: ProjectNo $\rightarrow$ Duration]

References/Further Readings

- Ramakrishnan, R., Gehrke, J.: **Database Management Systems**. USA: McGraw Hill Companies (2000)
- Elmasri, R., & Navathe, S. (2000). **Fundamentals of database systems (3rd ed.)**. Reading, Mass.: Addison-Wesley.
- Hoffer, J. A., Prescott, M. B., & McFadden, F. R. (2005). **Modern database management**. Upper Saddle River, N.J: Pearson/ Prentice Hall.

Connolly, T. M., & Begg, C. E. (2002). **Database systems: A practical approach to design, implementation, and management**. Harlow, England: Addison-Wesley

<http://pages.cs.wisc.edu/~dbbook/openAccess/thirdEdition/solutions/ans3ed-oddonly.pdf>

<https://gyires.inf.unideb.hu/GyBITT/03/>

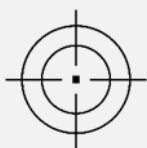
Module Five

Relational Theory and Query Languages

Unit 1 — Relational Algebra

INTRODUCTION

In this unit, you will learn about the relational model, which is the foundation of modern relational databases. You will be introduced to the structure of the relational model, where data is organised into tables (relations) with clearly defined rows and columns. This unit will also cover relational algebra and the key operations of relational algebra.



LEARNING OUTCOMES

At the end of this unit, you should be able to:

- Understand the purpose and structure of the relational model in database systems.
- Identify the components of a relation, including attributes, tuples, and keys.
- Explain the basic principles of relational algebra as a query language.
- Apply key relational algebra operations such as selection, projection, union, and join.
- Use relational algebra to express and solve database queries.

5.1.1 Relational Model

- The Relational Model (RM) remains the primary data model for commercial data-processing applications.
- The RM consists of a collection of tables, each of which is assigned a unique name.
- In a Relational Model, the term relation is used to refer to a table, while the term tuple is used to refer to a row. Similarly, the term attribute refers to a column of a table.

For example, consider this course table:

Table 5.1: Course Table

BIO-101	Intro. to Biology	Biology	4
BIO-301	Genetics	Biology	4
BIO-399	Computational Biology	Biology	3
CS-101	Intro. to Computer Science	Comp. Sci.	4
CS-190	Game Design	Comp. Sci.	4
CS-315	Robotics	Comp. Sci.	3
CS-319	Image Processing	Comp. Sci.	3
CS-347	Database System Concepts	Comp. Sci.	3
EE-181	Intro. to Digital Systems	Elec. Eng.	3
FIN-201	Investment Banking	Finance	3
HIS-351	World History	History	3
MU-199	Music Video Production	Music	3
PHY-101	Physical Principles	Physics	4

- This table has four attributes that store information about courses, consisting of a course ID, title, department name and credits, for each course.
- The relationship between 'BIO-101' and '4' is that BIO-101 is the course code and that 4 is the total credits obtainable in the course.
- Each tuple (row) contains and explains the relationship between the fields in the table.
- BIO-101 and 4 are 'relations' in the course table.

5.1.2 Relational Algebra

- The relational algebra consists of a set of operations that take one or two relations as input and produce a new relation as their result.
- Some of the operations could be unary: select, project, rename

- Or they could be binary: union, Cartesian product, set difference
- The Relational Algebra is a functional query language that forms the theoretical basis of the SQL query language.

What are these Relational Algebra Operations, and how do they work?

Recall that relational algebra takes one or two relations, acts on them and produces a new relation.

- The first operation is “Select”: The select operation selects tuples (rows) that satisfy a given predicate (a particular condition).

For instance, in a CU course registration relation, the HoD might select all male students, which can be output into another relation (table) entirely.

▪ *How does ~~select~~ work?*

We use the lowercase Greek letter sigma (σ) to denote **selection**. The **predicate** (condition) appears as a **subscript** to σ . The **argument relation** is in parentheses after the σ .

$\sigma_{\text{predicate}}$ (**argument relation**)

Using the CU course registration relation to get all male students, we write:

$\sigma_{\text{sex} = \text{'male'}}$ (**course registration**)

The variables must be attributes (columns) in the relation (table)

Select Operation Example

Consider this instructor relation in Table 3.2, which consists of four columns (attributes) - lecturerID, lecturerName, lecturerDept and lecturerSalary;

Table 5.2: Instructor relation

lecturerID	lecturerName	lecturerDept	lecturerSalary
10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	60000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
58583	Califieri	History	62000
76543	Singh	Finance	80000
76766	Crick	Biology	72000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000

Using the select operation, find all lecturers with a salary below 50,000:

$\sigma_{\text{salary} < 50000}(\text{instructor})$

15151	Mozart	Music	40000
-------	--------	-------	-------

- In general, comparisons are made using $=$, $\neq$, $<$, $\leq$, $>$, $\geq$ in the selection predicate. Furthermore, several predicates can be combined into a larger predicate by using the **connectives** - **and** ($\wedge$), **or** ($\vee$), and **not** ($\neg$).
- The selection predicate can also include comparisons between two attributes.

Quick Classwork

1. Write a select statement to find all courses taken by the Computer Science department

Course ID	Course Name	Department	Credits
BIO-101	Intro. to Biology	Biology	4
BIO-301	Genetics	Biology	4
BIO-399	Computational Biology	Biology	3
CS-101	Intro. to Computer Science	Comp. Sci.	4
CS-190	Game Design	Comp. Sci.	4
CS-315	Robotics	Comp. Sci.	3
CS-319	Image Processing	Comp. Sci.	3
CS-347	Database System Concepts	Comp. Sci.	3
EE-181	Intro. to Digital Systems	Elec. Eng.	3
FIN-201	Investment Banking	Finance	3
HIS-351	World History	History	3
MU-199	Music Video Production	Music	3
PHY-101	Physical Principles	Physics	4

2. Write a select statement to find all instructors with a salary greater than 50,000 but less than 70,000.

Instructor ID	Instructor Name	Department	Salary
10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	60000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
58583	Califieri	History	62000
76543	Singh	Finance	80000
76766	Crick	Biology	72000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000

- The second operation is “Project”: The Project operation is a unary operation (because it acts on only one relation) that returns its argument relation (table), with some attributes left out.
 - Still using the example of the course registration relation, attributes we can find are: (...). Assume we wish to get only the names, matric and reg. no. of all students, we use the Project operation to produce the output.
 - Project operation works on attributes.

▪ *How does **Project** work?*

Projection is denoted by the uppercase Greek letter pi (Π). We list those attributes that we wish to appear in the result as a subscript to Π . The argument relation follows in parentheses.

$\Pi_{\text{predicate}}(\text{relation})$

If we want to list out only the names, matric. no and reg. no of all students in CIS department, we write it as:

$\Pi_{\text{name, matric_no, reg_no}}(\text{course registration})$

The new output relation contains only these three attributes and eliminates the rest.

Quick Classwork

1. Write a projection statement to find the course codes and their corresponding units

Course Code	Course Name	Department	Units
BIO-101	Intro. to Biology	Biology	4
BIO-301	Genetics	Biology	4
BIO-399	Computational Biology	Biology	3
CS-101	Intro. to Computer Science	Comp. Sci.	4
CS-190	Game Design	Comp. Sci.	4
CS-315	Robotics	Comp. Sci.	3
CS-319	Image Processing	Comp. Sci.	3
CS-347	Database System Concepts	Comp. Sci.	3
EE-181	Intro. to Digital Systems	Elec. Eng.	3
FIN-201	Investment Banking	Finance	3
HIS-351	World History	History	3
MU-199	Music Video Production	Music	3
PHY-101	Physical Principles	Physics	4

- The third operation is “Cartesian-Product”: The Cartesian-Product operation, denoted by a cross ($\times$), allows us to combine information from any two relations. We write the Cartesian product of relations as where refers to the original relations. The steps are:

- Combine all attributes in both relations
- Multiply the tuples in both relations, forming all possible outcomes
- Represent the result of each multiplication as a single tuple.

Table 5.3: Teaches relation

10101	CS-101	1	Fall	2017
10101	CS-315	1	Spring	2018
10101	CS-347	1	Fall	2017
12121	FIN-201	1	Spring	2018
15151	MU-199	1	Spring	2018
22222	PHY-101	1	Fall	2017
32343	HIS-351	1	Spring	2018
45565	CS-101	1	Spring	2018
45565	CS-319	1	Spring	2018
76766	BIO-101	1	Summer	2017
76766	BIO-301	1	Summer	2018
83821	CS-190	1	Spring	2017
83821	CS-190	2	Spring	2017
83821	CS-319	2	Spring	2018
98345	EE-181	1	Spring	2017

Since the same attribute name may appear in the schemas of both, we need to devise a naming schema to distinguish between these attributes. We do so here by attaching to an attribute the name of the relation from which the attribute originally came. For example, the relation schema for is:

(instructor.ID, instructor.name, instructor.deptname, instructor. salary, teaches.ID, teaches.courseid, teaches.secid, teaches.semester, teaches.year)

Table 5.4: Instructor × teaches relation schema

10101	Srinivasan	Comp. Sci.	65000	10101	CS-101	1	Fall	2017
10101	Srinivasan	Comp. Sci.	65000	10101	CS-315	1	Spring	2018
10101	Srinivasan	Comp. Sci.	65000	10101	CS-347	1	Fall	2017
10101	Srinivasan	Comp. Sci.	65000	12121	FIN-201	1	Spring	2018
10101	Srinivasan	Comp. Sci.	65000	15151	MU-199	1	Spring	2018
10101	Srinivasan	Comp. Sci.	65000	22222	PHY-101	1	Fall	2017
...	...	...	...	...	...	...	...	...
...	...	...	...	...	...	...	...	...
12121	Wu	Finance	90000	10101	CS-101	1	Fall	2017
12121	Wu	Finance	90000	10101	CS-315	1	Spring	2018
12121	Wu	Finance	90000	10101	CS-347	1	Fall	2017
12121	Wu	Finance	90000	12121	FIN-201	1	Spring	2018
12121	Wu	Finance	90000	15151	MU-199	1	Spring	2018
12121	Wu	Finance	90000	22222	PHY-101	1	Fall	2017
...	...	...	...	...	...	...	...	...
...	...	...	...	...	...	...	...	...
15151	Mozart	Music	40000	10101	CS-101	1	Fall	2017
15151	Mozart	Music	40000	10101	CS-315	1	Spring	2018
15151	Mozart	Music	40000	10101	CS-347	1	Fall	2017
15151	Mozart	Music	40000	12121	FIN-201	1	Spring	2018
15151	Mozart	Music	40000	15151	MU-199	1	Spring	2018
15151	Mozart	Music	40000	22222	PHY-101	1	Fall	2017
...	...	...	...	...	...	...	...	...
...	...	...	...	...	...	...	...	...
22222	Einstein	Physics	95000	10101	CS-101	1	Fall	2017
22222	Einstein	Physics	95000	10101	CS-315	1	Spring	2018
22222	Einstein	Physics	95000	10101	CS-347	1	Fall	2017
22222	Einstein	Physics	95000	12121	FIN-201	1	Spring	2018
22222	Einstein	Physics	95000	15151	MU-199	1	Spring	2018
22222	Einstein	Physics	95000	22222	PHY-101	1	Fall	2017
...	...	...	...	...	...	...	...	...
...	...	...	...	...	...	...	...	...

With this schema, we can distinguish instructor.ID from teaches.ID. For those attributes that appear in only one of the two schemas, we shall usually drop the relation-name prefix. This simplification does not lead to any ambiguity. We can then write the relation schema for r as:

- (**instructor.ID**, name, dept name, salary, **teaches.ID**, course id, sec id, semester, year)

- The fourth operation is “Assignment”: The assignment operation, denoted by $\leftarrow$, works like an assignment operator in a programming language. It is a convenient way to express complex queries as it assigns parts of the query to temporary relation variables that can later be reused.

Note that the evaluation of an assignment does not result in any relation (table) being displayed to the user.

$$\begin{aligned}
 \text{courses_fall_2017} &\leftarrow \Pi_{\text{course_id}}(\sigma_{\text{semester} = \text{"Fall"} \wedge \text{year} = 2017}(\text{section})) \\
 \text{courses_spring_2018} &\leftarrow \Pi_{\text{course_id}}(\sigma_{\text{semester} = \text{"Spring"} \wedge \text{year} = 2018}(\text{section})) \\
 \text{courses_fall_2017} &\cap \text{courses_spring_2018}
 \end{aligned}$$

5. The fifth operation is “Join”: The join operation allows us to combine a selection and a Cartesian product into a single operation.

- The most common type is the Natural Join, which joins tuples based on equality of attributes with the same name in both relations. The duplicate attribute is usually eliminated in the result.
- The notation for a Natural Join of R and S is $R \bowtie S$.

5.1.3 Conclusion

In this unit, you have learned about the relational model, which provides the foundation for how data is structured and managed in modern relational databases. You explored the structure of relations. Additionally, you were introduced to relational algebra, the formal language for querying and manipulating relational data. You also examined the primary operations of relational algebra, such as selection, projection, and joins, and saw how they are applied to retrieve and combine data from multiple relations.

Summary

In this unit, you have learnt that the relational model organises data into tables (relations), each made up of rows and columns, to ensure structured and efficient data storage. You also explored the structure of a relation, including its key components such as attributes, tuples, and keys. In addition, you were introduced to relational algebra, a formal query language used to manipulate relational data. You learned about core relational algebra operations and how these operations are used to extract meaningful information from one or more tables in a database system.

Self-Assessment Questions

1. Explain how the select and project operations in relational algebra work, and provide an example of each.
2. What is the purpose of the Cartesian product and join operations, and how do they differ in terms of output and use cases?

Tutor-Marked Assignment

Create your own relation and perform the following RA operations on it:

- Select Operation
- Projection Operation
- Cartesian-Product Operation

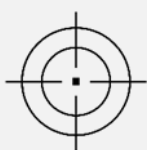
References/Further Readings

- Ramakrishnan, R., Gehrke, J.: Database Management Systems. USA: McGraw-Hill Companies (2000)
- Elmasri, R., & Navathe, S. (2000). Fundamentals of database systems (3rd ed.). Reading, Mass.: Addison-Wesley.
- Hoffer, J. A., Prescott, M. B., & McFadden, F. R. (2005). Modern database management. Upper Saddle River, N.J: Pearson/ Prentice Hall.
- Connolly, T. M., & Begg, C. E. (2002). **Database systems: A practical approach to design, implementation, and management**. Harlow, England: Addison-Wesley
- <http://pages.cs.wisc.edu/~dbbook/openAccess/thirdEdition/solutions/ans3ed-oddonly.pdf>
- <https://gyires.inf.unideb.hu/GyBITT/03/>

Unit 2 — Introduction to Structured Query Language and Introduction to Query Processing and Optimisation

INTRODUCTION

In this unit, you will learn about two key categories of SQL commands: Data Definition Language (DDL) and Data Manipulation Language (DML). You will explore the CREATE command, which is used to build new database structures. You will also focus on the core DML operations: INSERT for adding data, SELECT for retrieving data, and UPDATE for modifying existing data. These commands are essential for interacting with and managing data within a database system. Practical examples will guide you in understanding how to use these SQL statements effectively.



LEARNING OUTCOMES

At the end of this unit, you should be able to:

- Understand DDL commands and use the CREATE, ALTER and DROP commands efficiently
- Manipulate a given database practically using DML commands such as SELECT.

5.2.1 DDL, DML, DCL, and the Database Development Process

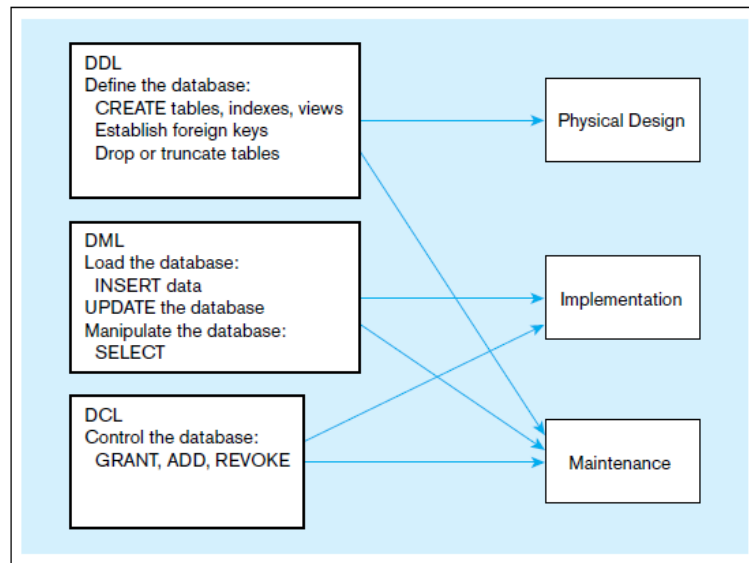


Figure 5.1: DDL, DML, DCL, and the database development process

5.2.2 Database Creation in MySQL

- A database can be created with the SQL statement:
CREATE DATABASE databasename;
- However, always check if a database already exists in the MySQLserver with the query:
SHOW DATABASES;
- You must select a database to work with from the server with the query:
USE nameofdatabase;

5.2.3 Datatypes in MySQL Environment

Data type	Declarations			
boolean	BOOLEAN			
character	CHAR	VARCHAR		
bit	BIT	BIT VARYING		
exact numeric	NUMERIC	DECIMAL	INTEGER	SMALLINT
approximate numeric	FLOAT	REAL	DOUBLE PRECISION	
datetime	DATE	TIME	TIMESTAMP	
interval	INTERVAL			
large objects	CHARACTER LARGE OBJECT		BINARY LARGE OBJECT	

Figure 5.2: ISO SQL Data Types

SQL (Relational Model Terms)

- Table as **Relation**
- Row as **Tuple**
- Column as **Attribute**

5.2.4 Data Definition Language (DDL)

The main SQL command for data definition:

- **CREATE** statement
 - create schemas, tables (relations), and domains (as well as other constructs such as views, indexes, assertions, and triggers).

Main SQL DDL

- CREATE SCHEMA DROP SCHEMA
- CREATE/ALTER DOMAIN DROP DOMAIN
- CREATE/ALTER TABLE DROP TABLE
- CREATE VIEW DROP VIEW
- Many DBMSs also provide: CREATE INDEX DROP INDEX

Environment

DBMS

Catalog

- a named collection of schemas in an SQL environment.
- Schema
- Domain

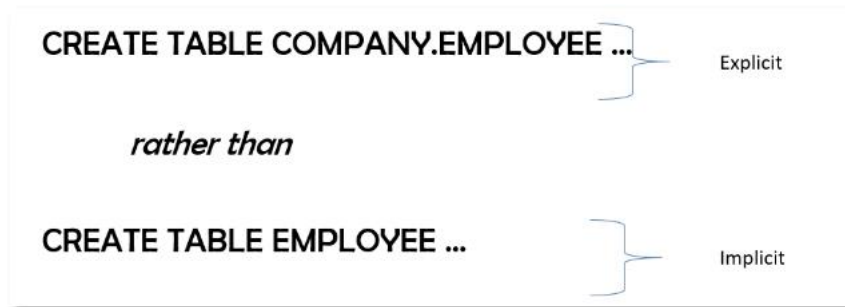
Objects in a schema

- Tables, Views, Domains, assertions, etc
- All have same owner

CREATE Schema

- **CREATE SCHEMA** Name_of_Schema;
- **DROP SCHEMA** Name_of_schema [RESTRICT | CASCADE];
- Example 1
- **CREATE SCHEMA** COMPANY;

CREATE Table



- Creates a table with one or more columns of the specified data type.
- With NOT NULL, the system rejects any attempt to insert a null in the column.
- Can specify a DEFAULT value for the column.
- Primary keys should always be specified as NOT NULL.
- FOREIGN KEY clause specifies FK along with the referential action.
- *Example:*

```

CREATE TABLE      EMPLOYEE
(Fname VARCHAR(15) NOT NULL,
 Minit  CHAR,
 Lname  VARCHAR(15) NOT NULL,
 Ssn    CHAR(9)     NOT NULL,
 Bdate  DATE,
 Address VARCHAR(30),
 Sex    CHAR,
 Salary  DECIMAL(10,2),
 Super_ssn CHAR(9),
 Dno    INT         NOT NULL,
PRIMARY KEY (Ssn));
  
```

To Rename a Table

- **RENAME TABLE** Old_Table_Name to New_Table_Name;
- Example: To rename the table EMPLOYEE in the previous slide to WORKER, use:
- **RENAME TABLE** EMPLOYEE to WORKER;

Solution

Use the query.

ALTER TABLE employee

MODIFY COLUMN ssn char(9) **FIRST**;

The query results are shown in Table 5.6.

Table 5.6: Modified Table, repositioning SSN as the first attribute

Result Grid											Filter Rows:			Edit:			Export/Import:			Wrap Cell Content:		
	ssn	Fname	Minit	Lname	Bdate	Address		Sex	Salary	Super_ssn	Dno											
▶	101	Henry	N	Alaska	1846-04-25	Pool office D, CIS dept, CU		M	10000.00	101	100											
	102	Oliva	E	Bolaji	2000-10-02	Pool office C, CIS dept, CU		F	110000.00	103	306											
	103	Julia	O	Abba	1990-04-02	Pool office A, CIS dept, CU		F	250000.00	102	209											
	104	Opeyemi	E	Salami	2009-02-15	Pool office E, CIS dept, CU		M	90000.00	104	119											
	105	Ufuoma	N	Cooker	1820-01-23	Pool office F, CIS dept, CU		F	120000.00	151	6											
*	NULL	NULL	NULL	NULL	NULL	NULL		NULL	NULL	NULL	NULL											

Dropping a column

- To drop a column use:
 - ALTER TABLE** table_name
 - DROP** column name_of_column_tobedropped;
- Example
- Drop the column *Dno* in the employee table.

Table 5.7: Dropping column

Result Grid													Filter Rows:		Edit:				Export/Import:			Wrap Cell Content:	
	ssn	Fname	Minit	Lname	Bdate	Address	Sex	Salary	Super_ssn	Dno													
	101	Henry	N	Alaska	1846-04-25	Pool office D, CIS dept, CU	M	10000.00	101	100													
	102	Oliva	E	Bolaji	2000-10-02	Pool office C, CIS dept, CU	F	110000.00	103	306													
	103	Julia	O	Abba	1990-04-02	Pool office A, CIS dept, CU	F	250000.00	102	209													
	104	Opeyemi	E	Salami	2009-02-15	Pool office E, CIS dept, CU	M	90000.00	104	119													
	105	Ufuoma	N	Cooker	1820-01-23	Pool office F, CIS dept, CU	F	120000.00	151	6													
	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL													

Solution

Use the query.

ALTER TABLE employee

DROP COLUMN Dno;

Result: The column Dno has been dropped from the table employee

Table 5.8: Dropped column Dno

ssn	Fname	Minit	Lname	Bdate	Address	Sex	Salary	Super_ssn
101	Henry	N	Alaska	1846-04-25	Pool office D, CIS dept, CU	M	10000.00	101
102	Oliva	E	Bolaji	2000-10-02	Pool office C, CIS dept, CU	F	110000.00	103
103	Julia	O	Abba	1990-04-02	Pool office A, CIS dept, CU	F	250000.00	102
104	Opeyemi	E	Salami	2009-02-15	Pool office E, CIS dept, CU	M	90000.00	104
105	Ufuoma	N	Cooker	1820-01-23	Pool office F, CIS dept, CU	F	120000.00	151
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Unique Constraint

- Example:

```
CREATE TABLE table_name(
  product_id INT, PRIMARY KEY NOT NULL,
  Product_name VARCHAR(30) UNIQUE,
  Amount DECIMAL (4,2)
);
```

- If you forgot to include the unique constraint at the point of creating a table, you can add it by using:

```
ALTER TABLE table_name
ADD CONSTRAINT UNIQUE(attribute_to_be_unique);
```

NOT NULL Constraint

- To include the NOT NULL constraint to an attribute on a table that has already been created, use the query:

```
ALTER TABLE table_name
MODIFY column_Inview datatype (size) NOT NULL
```

Check Constraint

- A check constraint is a rule that specifies the allowable values in the column of every row of the base table.

- Example: create a check constraint on the EMPLOYEES table to ensure that the minimum salary of employees is 10000.

```
CREATE TABLE      EMPLOYEE
(Fname      VARCHAR(15) NOT NULL,
Minit  CHAR,
Lname VARCHAR(15) NOT NULL,
Ssn   CHAR(9)   NOT NULL,
Bdate  DATE,
Address  VARCHAR(30),
Sex      CHAR,
Salary  DECIMAL(10,2) CONSTRAINT chk_Salary CHECK(salary>=10000.00),
Super_ssn  CHAR(9),
Dno   INT      NOT NULL,
PRIMARY KEY (Ssn);
```

- To add a check constraint to a table that already exists, use the query:
ALTER TABLE table_name
ADD CONSTRAINT constraint_name **CHECK**(logical condition);

Default Constraint

- The DEFAULT CONSTRAINT is used to set a default value for a specified column in a table.
- Example: Create a table named SKILL with attributes Skill_Id INT, ssn char(9), Skill_Type varchar(50), Number_of_Skill INT. Set the default Skill_Type as Coding.

```
CREATE TABLE SKILL(skill_Id INT PRIMARY KEY NOT NULL,
Ssn char(9),
skill_Type varchar(20) DEFAULT "Coding",
Number_of_Skill INT);
```

Insert Values into a Table with DEFAULT Constraint

- The columns must be specified after the insert into table_name clause; otherwise, the default constraint won't take effect.
- Example:
 Insert into Skill(skill_Id, ssn, Number_of_Skill)
 Values (010, 3, 9),
 (011, 9, 2),
 (012, 6, 12);

Quick Exercise

1. Insert a row into the skill table with the value of ssn as 3:
 - a) query the table skill to display its content
 - b) Apply the unique constraint on the column ssn (state your observation)
 - c) delete the record where the skill_Id is 010 and repeat the step in (b)
 - d) Try to insert a new record with ssn value of 3 again and interpret the outcome.

Timestamp using default Date and Time

- Timestamp datatype is an 8-byte value representing a date and time value in the format: YYYY-MM-DD and HH:MM:SS.
- Example: Create a transaction table with attributes Transaction_no INT primary key Not Null, Product varchar(50), Amount decimal(9,2), DateTime Default Now().

Solution

```
CREATE TABLE transaction (Transaction_no INT PRIMARY KEY
                           NOTNULL,
                           Product varchar(50),
                           Amount decimal(9,2),
                           Transaction_Date DateTime Default Now()
                           );
```

Inserting Multi Rows to Transaction Table

```
INSERT INTO transaction (Transaction_no, Product , Amount)
values (8, "Apple", 200.20),
        (2, "sharwama", 2500),
        (3, "Diaper", 5500),
        (4, "Calculator", 7000),
        (5, "T-Shirt", 15000);
```

- Result of Query Using Timestamp

Table 5.9: Transaction Table showing the default Timestamp for each transaction

Result Grid				
		Filter Rows:	Edit:	
	Transaction_no	Product	Amount	Transaction_Date
▶	1	Apple	200.20	2024-01-16 09:43:19
	2	sharwama	2500.00	2024-01-16 09:47:22
	3	Diaper	5500.00	2024-01-16 09:47:22
	4	Calculator	7000.00	2024-01-16 09:47:22
	5	T-Shirt	15000.00	2024-01-16 09:47:22
	8	Apple	200.20	2024-01-16 09:47:22
•	NULL	NULL	NULL	NULL

Auto_Increment Constraint

- Auto_Increment constraint is used to automatically increment the value of the column it is applied to.
- It works only for KEY ATTRIBUTES
- Example: Create a customer table consisting of attributes: customer_Id of type integer, FName of type varchar(30), LName of type Varchar(30) and Address of type varchar(200).
- The customer_Id should be the primary key and set to auto_Increment.

Solution

```
CREATE TABLE customers ( customer_Id INT PRIMARY KEY
AUTO_INCREMENT,
FName varchar(30),
LName varchar(30),
Address varchar(200)
);
```

```
INSERT INTO customers(FName, LName, Address)
VALUES ("Henry", "Ogbu", " Flat I1, New Estate"),
("Sinmisola", "Abiona", " Flat Z4, Esther Hall"),
("Barnabas", "Fortunatus", " Flat A1, Daniel Hall"),
("Joseph", "Alamu", " Room D101, Joseph Hall"),
("David", "Ukachi", " KM 5, Lekki Phase 2"),
("Nwafor", "Caleb", " No.10, Ring Rd, Benin-City"),
("Okon", "Favour", " Snake Island, Apapa, Lagos");
```

- Result of query

Table 5.10: Result of Auto_Increment constraint query

	customer_Id	FName	LName	Address
▶	1	Henry	Ogbu	Flat I1, New Estate
	2	Sinmisola	Abiona	Flat Z4, Esther Hall
	3	Barnabas	Fortunatus	Flat A1, Daniel Hall
	4	Joseph	Alamu	Room D101, Joseph Hall
	5	David	Ukachi	KM 5, Lekki Phase 2
	6	Nwafor	Caleb	No. 10, Ring Rd, Benin-City
	7	Okon	Favour	Snake Island, Apapa, Lagos
•	NULL	NULL	NULL	NULL

Foreign Key Constraint

- MySQL supports foreign key constraints, which permit cross-referencing related data across tables and foreign key constraints, which help keep the related data.
- A foreign key relationship involves a base table that holds the initial column values, and a child table with column values that reference the base column values.
- Example:
- Create a transaction table with attributes: Transaction_no INT, Product varchar(50), Amount decimal(9,2), Transaction_Date DateTime Default Now(), customer_ID INT.
 - The primary key should be Transaction_no. and it should accept no null values
 - Specify the customer_ID attribute as the foreign key
 - The attribute Transaction_Date should be set to DEFAULT

Solution

```
CREATE TABLE transaction ( Transaction_no INT PRIMARY KEY
                           NOT NULL,
                           Product varchar(50),
                           Amount decimal(9,2),
                           Transaction_Date DateTime Default Now(),
                           Customer_Id INT,
                           CONSTRAINT FK_Customer_Id FOREIGN KEY(customer_Id)
                           REFERENCES customers(customer_Id)

                           );
```

Let's insert values into the transaction table

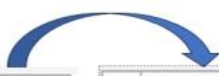
```
INSERT INTO transaction (Transaction_no, Product , Amount,
customer_Id)
values (8, "Apple", 200.20, 5),
      (2, "sharwama", 2500, 2),
      (3, "Diaper", 5500, 1),
      (4, "Calculator", 7000, 6),
      (5, "T-Shirt", 15000, 1),
      (9, "Calculator", 200.20, 2);
```

- Result of query

Table 5.11: Result of foreign key constraint query

Primary-Foreign Key Relationship

Table 5.12: Primary-foreign key relationship



Transaction_no	Product	Amount	Transaction_Date	customer_Id
2	sharwama	2500.00	2024-01-18 10:48:28	5
3	Diaper	5500.00	2024-01-18 10:48:28	2
4	Calculator	7000.00	2024-01-18 10:48:28	1
5	T-Shirt	15000.00	2024-01-18 10:48:28	6
8	Apple	200.20	2024-01-18 10:48:28	1
9	Calculator	200.20	2024-01-18 10:58:09	2
*	NULL	NULL	NULL	NULL

customer_Id	FName	LName	Address
1	Henry	Ogbu	Flat I1, New Estate
2	Sinmisola	Abiona	Flat Z4, Esther Hall
3	Barnabas	Fortunatus	Flat A1, Daniel Hall
4	Joseph	Alamu	Room D101, Joseph Hall
5	David	Ukachi	KM 5, Lekki Phase 2
6	Nwafor	Caleb	No. 10, Ring Rd, Benin-City
7	Okon	Favour	Snake Island, Apapa, Lagos
*	NULL	NULL	NULL

Applying a Foreign Key to a Table that Already existed

- To apply a FK constraint to a table that already existed, use the query:
- **ALTER TABLE** TableName
- **ADD CONSTRAINT** constraintname **FOREIGN KEY** (TargetAttribute) **REFERENCES** basetable (TargetAttribute);

Dropping a Foreign Key (FK)

- To drop a FK, use the query:
- **ALTER TABLE** TableName
- **DROP FOREIGN KEY** Name_of_FK;

5.2.5 Data Manipulation Language (DML)

Inserting rows into a table

- To insert rows into a table, use:
INSERT into table_name
VALUES (values separated by comma);

Updating data in a table

- To update data in a table, use:
UPDATE table_name
SET column_to_update = newValue
WHERE identifier=value;

Join

- A join is a clause used to combine rows from two or more tables based on a related column between them.
- **Joins can be:**
 - Inner join
 - Left join and
 - Right join

Inner join on Table customers and Transactions

SELECT * FROM

customer **INNER JOIN** transactions

ON customer.customer_Id= Transaction.customer_Id;

- Result from Inner Join

Table 5.13: Inner Join

```

16 • select * from customers inner join transaction on
17   customers.customer_Id=transaction.customer_Id;

```

	customer_Id	FName	LName	Address	Transaction_no	Product	Amount	Transaction_Date	customer_Id
▶	2	Sinmisola	Abiona	Flat Z4, Esther Hall	2	sharwama	2500.00	2024-01-18 11:45:38	2
	1	Henry	Ogbu	Flat I1, New Estate	3	Diaper	5500.00	2024-01-18 11:45:38	1
	6	Nwafor	Caleb	No.10, Ring Rd, Benin-City	4	Calculator	7000.00	2024-01-18 11:45:38	6
	1	Henry	Ogbu	Flat I1, New Estate	5	T-Shirt	15000.00	2024-01-18 11:45:38	1
	5	David	Ukachi	KM 5, Lekki Phase 2	8	Apple	200.20	2024-01-18 11:45:38	5
	2	Sinmisola	Abiona	Flat Z4, Esther Hall	9	Calculator	200.20	2024-01-18 11:45:38	2

Left join on Table customer and Transaction

SELECT * From

customer **LEFT JOIN** transaction

ON customer.customer_Id= transaction.customer_Id;

- Result from Left Join

Table 5.14: Left Join

```

16 • select * from customers left join transaction on
17   customers.customer_Id=transaction.customer_Id;

```

	customer_Id	FName	LName	Address	Transaction_no	Product	Amount	Transaction_Date	customer_Id
▶	1	Henry	Ogbu	Flat I1, New Estate	3	Diaper	5500.00	2024-01-18 11:45:38	1
	1	Henry	Ogbu	Flat I1, New Estate	5	T-Shirt	15000.00	2024-01-18 11:45:38	1
	2	Sinmisola	Abiona	Flat Z4, Esther Hall	2	sharwama	2500.00	2024-01-18 11:45:38	2
	2	Sinmisola	Abiona	Flat Z4, Esther Hall	9	Calculator	200.20	2024-01-18 11:45:38	2
	3	Barnabas	Fortunatus	Flat A1, Daniel Hall	NULL	NULL	NULL	NULL	NULL
	4	Joseph	Alamu	Room D101, Joseph Hall	NULL	NULL	NULL	NULL	NULL
	5	David	Ukachi	KM 5, Lekki Phase 2	8	Apple	200.20	2024-01-18 11:45:38	5
	6	Nwafor	Caleb	No.10, Ring Rd, Benin-City	4	Calculator	7000.00	2024-01-18 11:45:38	6
	7	Okon	Favour	Snake Island, Apapa, Lagos	NULL	NULL	NULL	NULL	NULL

Right join on Table customer and Transaction

SELECT * FROM

customer **RIGHT JOIN** transaction

ON customer.customer_Id= transaction.customer_Id;

- Result from Right Join

Table 5.15: Right Join

```

16 • select * from customers right join transaction on
17   customers.customer_Id=transaction.customer_Id;

```

customer_Id	FName	LName	Address	Transaction_no	Product	Amount	Transaction_Date	customer_Id
2	Sinmisola	Abiona	Flat Z4, Esther Hall	2	sharwama	2500.00	2024-01-18 11:45:38	2
1	Henry	Ogbu	Flat I1, New Estate	3	Diaper	5500.00	2024-01-18 11:45:38	1
6	Nwafor	Caleb	No.10, Ring Rd, Benin-City	4	Calculator	7000.00	2024-01-18 11:45:38	6
1	Henry	Ogbu	Flat I1, New Estate	5	T-Shirt	15000.00	2024-01-18 11:45:38	1
5	David	Ukachi	KM 5, Lekki Phase 2	8	Apple	200.20	2024-01-18 11:45:38	5
2	Sinmisola	Abiona	Flat Z4, Esther Hall	9	Calculator	200.20	2024-01-18 11:45:38	2

More Concepts on DML

- Support the basic data manipulation operations on the data held in the database.
- Enables users to access and manipulate data
- Goal is to provide efficient human interaction with the system.
- Query language.

5.2.6 Writing SQL Commands

- SQL statement consists of reserved words and user-defined words.
- Reserved words are a fixed part of SQL and must be spelt exactly as required and cannot be split across lines.
- User-defined words are made up by the user and represent names of various database objects, such as relations, columns, and views.
- Most components of an SQL statement are case insensitive, except for literal character data.
- More readable with indentation and lineation:
 - Each clause should begin on a new line.
 - Start of a clause should line up with the start of other clauses.
 - If a clause has several parts, each should appear on a separate line and be indented under the start of the clause.

5.2.7 Literals

- Literals are constants used in SQL statements.
- All non-numeric literals must be enclosed in single quotes (e.g. 'London').
- All numeric literals must not be enclosed in quotes (e.g. 650.00).

SELECT Statement

```
SELECT [DISTINCT | ALL] { * | [columnExpression [AS newName]] [, ...]
}
FROM      TableName [alias] [, ...]
[WHERE    condition]
[GROUP BY columnList]
[HAVING   condition]
[ORDER BY columnList]
```

- **FROM** Specifies table(s) to be used.
- **WHERE** Filters rows.
- **GROUP BY** Forms groups of rows with same column value.
- **HAVING** Filters groups subject to some condition.
- **SELECT** Specifies which columns are to appear in output.
- **ORDER BY** Specifies the order of the output.
 - Order of the clauses cannot be changed.
 - Only **SELECT** and **FROM** are mandatory.

Example 1 (All Columns, All Rows):

List full details of all customers.

SELECT customer_Id, FName, LName, Address **FROM** customers;

- Can use * as an abbreviation for 'all columns':

SELECT * FROM customers;

Table 5.16: All column and All rows from table customers

	customer_Id	FName	LName	Address
▶	1	Henry	Ogbu	Flat I1, New Estate
	2	Sinmisola	Abiona	Flat Z4, Esther Hall
	3	Barnabas	Fortunatus	Flat A1, Daniel Hall
	4	Joseph	Alamu	Room D101, Joseph Hall
	5	David	Ukachi	KM 5, Lekki Phase 2
	6	Nwafor	Caleb	No. 10, Ring Rd, Benin-City
	7	Okon	Favour	Snake Island, Apapa, Lagos
*	NULL	NULL	NULL	NULL

Example 2 (Specific Columns, All Rows):

Produce a list of addresses for all customers, showing only customer_Id, FName and address.

SELECT, customer_Id, Fname, address

FROM customers;

Table 5.17: Specific column and All rows from table customers

	customer_Id	Fname	address
▶	1	Henry	Flat I1, New Estate
	2	Sinmisola	Flat Z4, Esther Hall
	3	Barnabas	Flat A1, Daniel Hall
	4	Joseph	Room D101, Joseph Hall
	5	David	KM 5, Lekki Phase 2
	6	Nwafor	No. 10, Ring Rd, Benin-City
	7	Okon	Snake Island, Apapa, Lagos
*	NULL	NULL	NULL

Example 3: Use of DISTINCT

- List the address of all everyone that is a customer.

SELECT address

FROM customers;

Table 5.18a: Use of distinct

	address
▶	Flat I1, New Estate
	Flat Z4, Esther Hall
	Flat A1, Daniel Hall
	Room D101, Joseph Hall
	KM 5, Lekki Phase 2
	No. 10, Ring Rd, Benin-City
	Snake Island, Apapa, Lagos
	Room 35, New Estate
	Room 35, New Estate
	Room 35, New Estate

- Use DISTINCT to eliminate duplicates:

SELECT DISTINCT address

FROM customers;

Table 5.18b: Use of distinct

	address
▶	Flat I1, New Estate
	Flat Z4, Esther Hall
	Flat A1, Daniel Hall
	Room D101, Joseph Hall
	KM 5, Lekki Phase 2
	No. 10, Ring Rd, Benin-City
	Snake Island, Apapa, Lagos
	Room 35, New Estate

SELECT Statement - Aggregates

- ISO standard defines five aggregate functions:

COUNT returns the number of values in the specified column.

SUM returns the sum of values in the specified column.

AVG returns the average of values in the specified column.

MIN returns the smallest value in the specified column.

MAX returns the largest value in the specified column.

- COUNT(*)** counts all rows of a table, regardless of whether nulls or duplicate values occur.

Can use DISTINCT before column name to eliminate duplicates.

- DISTINCT has no effect with MIN/MAX, but may have with **SUM/AVG**.

Example 1: Use of COUNT(*)

How many properties cost more than \$350 per month to rent?

SELECT COUNT(*) AS count

FROM PropertyForRent

WHERE rent > 350;

count
5

Example 2: Use of COUNT(DISTINCT)

How many different properties were viewed in May '04?

```
SELECT COUNT (DISTINCT propertyNo) AS count
FROM Viewing
WHERE viewDate BETWEEN '1-May-04' AND '31-May-04';
```

count
2

Example 3: Use of COUNT and SUM

Find the number of Managers and the sum of their salaries.

```
SELECT COUNT(staffNo) AS count,
SUM(salary) AS sum
FROM Staff
WHERE position = 'Manager';
```

count	sum
2	54000.00

Example 4: Use of MIN, MAX, AVG

Find the minimum, maximum, and average staff salary.

```
SELECT MIN(salary) AS min,
MAX(salary) AS max,
AVG(salary) AS avg
FROM Staff;
```

min	max	avg
9000.00	30000.00	17000.00

SELECT Statement – Grouping

- Use the GROUP BY clause to get sub-totals.
- SELECT and GROUP BY are closely integrated: each item in the SELECT list must be single-valued per group, and the SELECT clause may only contain:
 - column names
 - aggregate functions
 - constants
 - expression involving combinations of the above.
- All column names in the SELECT list must appear in the GROUP BY clause unless the name is used only in an aggregate function.
- If WHERE is used with GROUP BY, WHERE is applied first, then groups are formed from the remaining rows satisfying the predicate.
- ISO considers two nulls to be equal for purposes of GROUP BY.

Example 1: Use of GROUP BY

Find the number of staff in each branch and their total salaries.

SELECT branchNo,

COUNT(staffNo) **AS** count,

SUM(salary) **AS** sum

FROM Staff

GROUP BY branchNo

ORDER BY branchNo;

branchNo	count	sum
B003	3	54000.00
B005	2	39000.00
B007	1	9000.00

Restricted Groupings – HAVING clause

- HAVING clause is designed for use with GROUP BY to restrict groups that appear in final result table.
- Similar to WHERE, but WHERE filters individual rows whereas HAVING filters groups.
- Column names in HAVING clause must also appear in the GROUP BY list or be contained within an aggregate function.

Example 1: Use of HAVING

For each branch with more than 1 member of staff, find number of staff in each branch and sum of their salaries.

```
SELECT branchNo,
        COUNT(staffNo) AS count,
        SUM(salary) AS sum
FROM Staff
GROUP BY branchNo
HAVING COUNT(staffNo) > 1
ORDER BY branchNo;
```

branchNo	count	sum
B003	3	54000.00
B005	2	39000.00

Subqueries

- Some SQL statements can have a SELECT embedded within them.
- A subselect can be used in WHERE and HAVING clauses of an outer SELECT, where it is called a subquery or nested query.
- Subselects may also appear in INSERT, UPDATE, and DELETE statements.

Example 1: Subquery with Equality

List staff who work in the branch at '163 Main St'.

```
SELECT staffNo, fName, lName, position
FROM Staff
WHERE branchNo =
    (SELECT branchNo
     FROM Branch
     WHERE street = '163 Main St');
```

- Inner SELECT finds branch number for branch at '163 Main St' ('B003').
- Outer SELECT then retrieves details of all staff who work at this branch.
- Outer SELECT then becomes:

```
SELECT staffNo, fName, lName, position
FROM Staff
WHERE branchNo = 'B003';
```

staffNo	fName	lName	position
SG37	Ann	Beech	Assistant
SG14	David	Ford	Supervisor
SG5	Susan	Brand	Manager

Example 2: Subquery with Aggregate

List all staff whose salary is greater than the average salary, and show by how much.

```
SELECT staffNo, fName, lName, position,
salary – (SELECT AVG(salary) FROM Staff) As SalDiff
FROM Staff
WHERE salary >
    (SELECT AVG(salary)
     FROM Staff);
```

- Cannot write 'WHERE salary > AVG(salary)'

- Instead, use a subquery to find the average salary (17000), and then use an outer SELECT to find those staff with a salary greater than this:

SELECT staffNo, fName, lName, position,

salary – 17000 **As** salDiff

FROM Staff

WHERE salary > 17000;

staffNo	fName	lName	position	salDiff
SL21	John	White	Manager	13000.00
SG14	David	Ford	Supervisor	1000.00
SG5	Susan	Brand	Manager	7000.00

Subquery Rules

- ORDER BY clause may not be used in a subquery (although it may be used in outermost SELECT).
- Subquery SELECT list must consist of a single column name or expression, except for subqueries that use EXISTS.
- When subquery is an operand in a comparison, the subquery must appear on the right-hand side.

Example 1: Nested subquery: use of IN

List properties handled by staff at '163 Main St'.

SELECT propertyNo, street, city, postcode, type, rooms, rent

FROM PropertyForRent

WHERE staffNo IN

(**SELECT** staffNo

FROM Staff

WHERE branchNo =

(**SELECT** branchNo

FROM Branch

WHERE street = '163 Main St'));

propertyNo	street	city	postcode	type	rooms	rent
PG16	5 Novar Dr	Glasgow	G12 9AX	Flat	4	450
PG36	2 Manor Rd	Glasgow	G32 4QX	Flat	3	375
PG21	18 Dale Rd	Glasgow	G12	House	5	600

Example 2: Create Horizontal View

Create a view so that the manager at branch B003 can only see details for staff who work in the manager's office.

CREATE VIEW Manager3Staff

AS SELECT *

FROM Staff

WHERE branchNo = 'B003';

staffNo	fName	lName	position	sex	DOB	salary	branchNo
SG37	Ann	Beech	Assistant	F	10-Nov-60	12000.00	B003
SG14	David	Ford	Supervisor	M	24-Mar-58	18000.00	B003
SG5	Susan	Brand	Manager	F	3-Jun-40	24000.00	B003

Example 3: Create Vertical View

Create view of staff details for branch B003, excluding salaries.

CREATE VIEW Staff3

AS SELECT staffNo, fName, lName, position, sex

FROM Staff

WHERE branchNo = 'B003';

staffNo	fName	lName	position	sex
SG37	Ann	Beech	Assistant	F
SG14	David	Ford	Supervisor	M
SG5	Susan	Brand	Manager	F

5.2.8 Writing SQL Commands More Practice Examples on SQL Queries

- Our SQL queries shall be based on the relations below.

Table 5.19: Instance of the DreamHome rental database

Branch									
branchNo	street	city	postcode						
B005	22 Deer Rd	London	SW1 4EH						
B007	16 Argyll St	Aberdeen	AB2 3SU						
B003	163 Main St	Glasgow	G11 9QX						
B004	32 Manae Rd	Bristol	BS99 1NZ						
B002	56 Clover Dr	London	NW10 6EU						

						Client			
						clientNo	fName	lName	telNo
						CR76	John	Kay	0207-774-5632
						CR56	Aline	Stewart	0141-848-1825
						CR74	Mike	Ritchie	01475-392178
						CR62	Mary	Tregear	01224-196720

								Staff	
staffNo	fName	lName	position	sex	DOB	salary	branchNo		
SL21	John	White	Manager	M	1-Oct-45	30000	B005		
SG37	Ann	Beech	Assistant	F	10-Nov-60	12000	B003		
SG14	David	Ford	Supervisor	M	24-Mar-58	18000	B003		
SA9	Mary	Howe	Assistant	F	19-Feb-70	9000	B007		
SG5	Susan	Brand	Manager	F	3-Jun-40	24000	B003		
SL41	Julie	Lee	Assistant	F	13-Jun-65	9000	B005		

						PrivateOwner			
						ownerNo	fName	lName	address
						CO46	Joe	Keogh	2 Fergus Dr, Aberdeen AB2 7SX
						CO87	Carol	Farrel	6 Achray St, Glasgow G32 9DX
						CO40	Tina	Murphy	63 Well St, Glasgow G42
						CO93	Tony	Shaw	12 Park Pl, Glasgow G4 0QR

										PropertyForRent	
propertyNo	street	city	postcode	type	rooms	rent	ownerNo	staffNo	branchNo		
PA14	16 Holthead	Aberdeen	AB7 5SU	House	6	650	CO46	SA9	B007		
PL94	6 Argyll St	London	NW2	Flat	4	400	CO87	SL41	B005		
PG4	6 Lawrence St	Glasgow	G11 9QX	Flat	3	350	CO40		B003		
PG36	2 Manor Rd	Glasgow	G32 4QX	Flat	3	375	CO93	SG37	B003		
PG21	18 Dale Rd	Glasgow	G12	House	5	600	CO87	SG37	B003		
PG16	5 Novar Dr	Glasgow	G12 9AX	Flat	4	450	CO93	SG14	B003		

				Viewing		Registration			
clientNo	propertyNo	viewDate	comment			clientNo	branchNo	staffNo	dateJoined
CR56	PA14	24-May-04	too small			CR76	B005	SL41	2-Jan-04
CR76	PG4	20-Apr-04	too remote			CR56	B003	SG37	11-Apr-03
CR56	PG4	26-May-04				CR74	B003	SG37	16-Nov-02
CR62	PA14	14-May-04	no dining room			CR62	B007	SA9	7-Mar-03
CR56	PG36	28-Apr-04							

Example 1 (Calculated Fields)

- Produce a list of monthly salaries for all staff, showing staff number, first and last names, and salary details.

SELECT staffNo, fName, lName, salary/12

FROM Staff;

staffNo	fName	lName	col4
SL21	John	White	2500.00
SG37	Ann	Beech	1000.00
SG14	David	Ford	1500.00
SA9	Mary	Howe	750.00
SG5	Susan	Brand	2000.00
SL41	Julie	Lee	750.00

- To name a column, use the AS clause:

SELECT staffNo, fName, lName, salary/12

AS monthlySalary

FROM Staff;

Example 2 (Comparison Search Condition)

- List all staff with a salary greater than 10,000.

SELECT staffNo, fName, lName, position, salary

FROM Staff **WHERE** salary > 10000;

staffNo	fName	lName	position	salary
SL21	John	White	Manager	30000.00
SG37	Ann	Beech	Assistant	12000.00
SG14	David	Ford	Supervisor	18000.00
SG5	Susan	Brand	Manager	24000.00

9.0.1 Example 3 (Compound Comparison Search Condition)

- List addresses of all branch offices in London or Glasgow.

SELECT *

FROM Branch

WHERE city = "London" OR city = "Glasgow";

branchNo	street	city	postcode
B005	22 Deer Rd	London	SW1 4EH
B003	163 Main St	Glasgow	G11 9QX
B002	56 Clover Dr	London	NW10 6EU

Example 4 (Range Search Condition)

- List all staff with a salary between 20,000 and 30,000.

SELECT staffNo, fName, lName, position, salary

FROM Staff

WHERE salary **BETWEEN** 20000 **AND** 30000;

- BETWEEN** test includes the endpoints of the range.

staffNo	fName	lName	position	salary
SL21	John	White	Manager	30000.00
SG5	Susan	Brand	Manager	24000.00

- Also a negated version **NOT BETWEEN**.
- **BETWEEN** does not add much to SQL's expressive power. Could also write:
SELECT staffNo, fName, lName, position, salary
FROM Staff
WHERE salary >= 20000 AND salary <= 30000;
- Useful, though, for a range of values.

9.0.2 Example 5 (Set Membership)

- List all managers and supervisors.
SELECT staffNo, fName, lName, position
FROM Staff
WHERE position IN ('Manager', 'Supervisor');

staffNo	fName	lName	position
SL21	John	White	Manager
SG14	David	Ford	Supervisor
SG5	Susan	Brand	Manager

- There is a negated version (**NOT IN**).
- **IN** does not add much to SQL's expressive power.
- Could have expressed this as:
- **SELECT** staffNo, fName, lName, position
FROM Staff
WHERE position='Manager' OR
position='Supervisor';

Example 6 (Pattern Matching)

- Find all owners with the string 'Glasgow' in their address.
SELECT clientNo, fName, lName, address, telNo
FROM PrivateOwner
WHERE address LIKE '%Glasgow%';

ownerNo	fName	lName	address	telNo
CO87	Carol	Farrel	6 Achray St, Glasgow G32 9DX	0141-357-7419
CO40	Tina	Murphy	63 Well St, Glasgow G42	0141-943-1728
CO93	Tony	Shaw	12 Park Pl, Glasgow G4 0QR	0141-225-7025

- SQL has two special pattern-matching symbols:
- LIKE '%Glasgow%' means a sequence of characters of any length containing 'Glasgow'.

NOTE: PRACTICE MORE EXAMPLES ON THE POWERPOINT SLIDES

Class Activity 2 (Database Tables)

5.2.9 Class Activity 2 (Database Tables)

Customer_ID	Customer_Name	Customer_Address	City	State	Postal_Code
*	Contemporary Casuals	1355 S Hines Blvd	Gainesville	FL	32601-2871
*	Value Furniture	15145 S.W. 17 th St.	Plano	T	75094-7743
*	Home Furnishings	1900 Allard Ave	Albany	NY	12209-1125
*	Eastern Furniture	1925 Beltline Road	Carteret	NJ	07008-3188
*	Impressions	5586 Westcott Ct.	Sacramento	CA	94206-4056
*	Furniture Gallery	325 Flatiron Dr.	Boulder	CO	80514-4432
*	Period Furniture	394 Rainbow Dr.	Seattle	WA	97954-5589
*	California Classics	816 Peach Rd.	Santa Clara	CA	96915-7754
*	M and N Casual Furniture	3709 First Street	Clearwater	FL	34620-2314
*	Seminole Interiors	2400 Rocky Point Dr	Seminole	FL	34646-4423
*	American Euro Lifestyles	2424 Missouri Ave. N	Prospect Park	NJ	07508-5621
*	Battle Creek Furniture	Capitol Ave SW	Battle Creek	MI	49015-3401
*	Heritage furnishings	66789 College Ave	Carlisle	PA	17013-8834
*	Kaneohe Homas	112 Kiowai St	Kaneohe	HI	96744-2537
*	Mountain Scenes	4132 Main Street	Ogden	UT	84403-4432
AutoNumber					

Customer_t

Order_ID	Product_ID	Order_Quantity
1001	1	2
1001	2	2
1001	4	1
1002	3	5
1003	3	3
1004	6	2
1004	8	2
1005	4	4
1006	4	1
1006	5	2
1006	7	2
1007	1	3
1007	2	2
1008	3	3
1008	8	3
1009	4	2
1009	7	3
1010	8	10

Orderline_t

Order_ID	Order_Date	Customer_ID
1001	10/21/2006	1
1002	10/21/2006	8
1003	10/22/2006	15
1004	10/22/2006	5
1005	10/24/2006	3
1006	10/24/2006	2
1007	10/27/2006	11
1008	10/30/2006	12
1009	11/5/2006	4
1010	11/5/2006	1

Order_t

Product_ID	Product_Description	Product_Finish	Standard_Price	Product_Line_ID
1	End Table	Cherry	\$175.00	1
2	Coffee Table	Natural Ash	\$200.00	2
3	Computer Desk	Natural Ash	\$375.00	2
4	Entertainment Center	Natural Maple	\$650.00	3
5	Winters Desk	Cherry	\$325.00	1
6	Drawer Desk	White Ash	\$750.00	2
7	Dining Table	Natural Ash	\$800.00	2
8	Computer Desk	Walnut	\$250.00	3
AutoNumber				

Product_t

- Which products have a standard price of less than 275?

Solution

```
SELECT PRODUCT_DESCRIPTION, STANDARD_PRICE
FROM PRODUCT_V
WHERE STANDARD_PRICE < 275;
```

Result:

PRODUCT_DESCRIPTION	STANDARD_PRICE
End Table	175
Computer Desk	250
Coffee Table	200

2. What is the address of the customer named Home Furnishings? Use an alias NAME for the customer.

```
SELECT CUSTOMER_NAME AS NAME, CUSTOMER_ADDRESS
FROM CUSTOMER_t
WHERE CUSTOMER_NAME = 'Home Furnishings';
```

Solution

Result:

NAME	CUSTOMER_ADDRESS
Home Furnishings	1900 Allard Ave.

3. List the unit price, product name and product ID for all products in the product table.

Solution

```
SELECT STANDARD_PRICE, PRODUCT_DESCRIPTION, PRODUCT_ID
FROM PRODUCT_T;
```

Result:

STANDARD_PRICE	PRODUCT_DESCRIPTION	PRODUCT_ID
175	End Table	1
200	Coffee Table	2
375	Computer Desk	3
650	Entertainment Center	4
325	Writer's Desk	5
750	8-Drawer Desk	6
800	Dining Table	7
250	Computer Desk	8

4. What is the average standard price for the product in inventory?

```
SELECT AVG (STANDARD_PRICE) AS AVERAGE
FROM PRODUCT_V;
```

AVERAGE

440.625

5. How many different items were ordered on order number 1004

```
SELECT COUNT (*)
FROM ORDER_LINE_V
WHERE ORDER_ID = 1004;
```

Result:

COUNT (*)
2

6. Which orders have been placed since 10/24/2006

```
SELECT ORDER_ID, ORDER_DATE
FROM ORDER_V
WHERE ORDER_DATE > '24-OCT-2006';
```

Result:

ORDER_ID	ORDER_DATE
1007	27-OCT-06
1008	30-OCT-06
1009	05-NOV-06
1010	05-NOV-06

7. What furniture does Pine Valley carry that isn't made of cherry?

```
SELECT PRODUCT_DESCRIPTION, PRODUCT_FINISH
FROM PRODUCT_V
WHERE PRODUCT_FINISH != 'Cherry';
```

Result:

PRODUCT	PRODUCT_FINISH
Coffee Table	Natural Ash
Computer Desk	Natural Ash
Entertainment Center	Natural Maple
8-Drawer Desk	White Ash
Dining Table	Natural Ash
Computer Desk	Walnut

8. List products, finish, and unit price for all desks and all tables that cost more than 300 in the PRODUCT view.

Solution

```
SELECT PRODUCT_DESCRIPTION, PRODUCT_FINISH, STANDARD_PRICE
FROM PRODUCT_V
WHERE PRODUCT_DESCRIPTION LIKE '%Desk'
OR PRODUCT_DESCRIPTION LIKE '%Table'
AND STANDARD_PRICE > 300;
```

Result:

PRODUCT_DESCRIPTION	PRODUCT_FINISH	STANDARD_PRICE
Computer Desk	Natural Ash	375
Writer's Desk	Cherry	325
8-Drawer Desk	White Ash	750
Dining Table	Natural Ash	800
Computer Desk	Walnut	250

9. Which products in the PRODUCT view have a standard price between 200 and 300?

```
SELECT PRODUCT_DESCRIPTION, STANDARD_PRICE
FROM PRODUCT_V
WHERE STANDARD_PRICE > 199 AND STANDARD_PRICE < 301;
```

Result:

PRODUCT_NAME	STANDARD_PRICE
Coffee Table	200
Computer Desk	250

10. Count the number of customers with an address in each state to which we ship

```
SELECT CUSTOMER_STATE, COUNT (CUSTOMER_STATE)
FROM CUSTOMER_V
GROUP BY CUSTOMER_STATE;
```

Result:

CUSTOMER_STATE	COUNT(CUSTOMER_STATE)
CA	2
CO	1
FL	3
HI	1
MI	1
NJ	2
NY	1
PA	1
TX	1
UT	1
WA	1

11. Find only states with more than one customer

```
SELECT State, COUNT(STATE) AS StateNumber
FROM Customer_t
GROUP BY State
Having COUNT(State)>1
```

State	StateNumber
CA	2
FL	3
NJ	2

12. List the product finish and average standard price for cherry, natural ash, natural maple and white ash, where the average standard price is less than 750.

```
SELECT PRODUCT_FINISH, AVG (STANDARD_PRICE)
FROM PRODUCT_V
WHERE PRODUCT_FINISH IN ('Cherry', 'Natural Ash', 'Natural Maple', 'White Ash')
GROUP BY PRODUCT_FINISH
HAVING AVG (STANDARD_PRICE) < 750
ORDER BY PRODUCT_FINISH;
```

Result:

PRODUCT_FINISH	AVG(UNIT_PRICE)
Cherry	250
Natural Ash	458.333333
Natural Maple	650

5.2.10 Exercises

Write a database description for each of the relations given. Assume the following attribute data types.

- StudentID(integer, primary key)
- StudentName(25 characters)
- FacultyID(integer, primary key)

- FacultyName(25 characters)
- CourseID(8 characters, primary key)
- CourseName(15 characters)
- DateQualified(date)
- SectionNo(integer, primary key)
- Semester(7 characters)

STUDENT (StudentID, StudentName)

<u>StudentID</u>	StudentName
38214	Letersky
54907	Altwater
66324	Aiken
70542	Marra
...	

QUALIFIED (FacultyID, CourseID, DateQualified)

<u>FacultyID</u>	<u>CourseID</u>	DateQualified
2143	ISM 3112	9/1988
2143	ISM 3113	9/1988
3467	ISM 4212	9/1995
3467	ISM 4930	9/1995
4756	ISM 3113	9/1991
4756	ISM 3112	9/1991
...		

FACULTY (FacultyID, FacultyName)

<u>FacultyID</u>	FacultyName
2143	Birkin
3467	Berndt
4756	Collins
...	

SECTION (SectionNo, Semester, CourseID)

<u>SectionNo</u>	<u>Semester</u>	<u>CourseID</u>
2712	I-2008	ISM 3113
2713	I-2008	ISM 3113
2714	I-2008	ISM 4212
2715	I-2008	ISM 4930
...		

COURSE (CourseID, CourseName)

<u>CourseID</u>	CourseName
ISM 3113	Syst Analysis
ISM 3112	Syst Design
ISM 4212	Database
ISM 4930	Networking
...	

REGISTRATION (StudentID, SectionNo, Semester)

<u>StudentID</u>	<u>SectionNo</u>	<u>Semester</u>
38214	2714	I-2008
54907	2714	I-2008
54907	2715	I-2008
66324	2713	I-2008
...		

5.2.11 Conclusion

In this unit, you have explored the fundamental SQL command categories: Data Definition Language (DDL) and Data Manipulation Language (DML). You learned how the CREATE command is used to define new database structures, and how essential DML commands like INSERT, SELECT, and UPDATE allow for adding, retrieving, and modifying data, respectively. Together, these SQL operations form the foundation of effective database interaction and management. Through practical examples, you also gained insight into how these commands are applied in real-world scenarios to support data-driven applications.

Summary

In this unit, you have learned that SQL commands are grouped into two main categories: Data Definition Language (DDL) and Data Manipulation Language (DML). The CREATE DDL command is used to define new database objects, such as tables. DML commands, INSERT, SELECT, and UPDATE, are essential for handling data within those objects. These operations allow you to populate, retrieve, and modify data in a structured and controlled manner. Together, DDL and DML form the basis for how databases are created and used in practical applications.

References/Further Readings

- Ramakrishnan, R., Gehrke, J.: Database Management Systems. USA: McGraw-Hill Companies (2000)
- Elmasri, R., & Navathe, S. (2000). Fundamentals of database systems (3rd ed.). Reading, Mass.: Addison-Wesley.
- Hoffer, J. A., Prescott, M. B., & McFadden, F. R. (2005). Modern database management. Upper Saddle River, N.J: Pearson/ Prentice Hall.
- Connolly, T. M., & Begg, C. E. (2002). Database systems: A practical approach to design, implementation, and management. Harlow, England: Addison-Wesley
- <http://pages.cs.wisc.edu/~dbbook/openAccess/thirdEdition/solutions/ans3ed-oddonly.pdf>
- <https://gyires.inf.unideb.hu/GyBITT/03/>

5.2.12 Extra

SQL Server Data Types

Data type	Description	Max size	Storage
char(n)	Fixed width character string	8,000 characters	Defined width
varchar(n)	Variable width character string	8,000 characters	2 bytes + number of chars
varchar(max)	Variable width character string	1,073,741,824 characters	2 bytes + number of chars
text	Variable width character string	2GB of text data	4 bytes + number of chars
nchar	Fixed width Unicode string	4,000 characters	Defined width x 2
nvarchar	Variable width Unicode string	4,000 characters	
nvarchar(max)	Variable width Unicode string	536,870,912 characters	
ntext	Variable width Unicode string	2GB of text data	
binary(n)	Fixed width binary string	8,000 bytes	
varbinary	Variable width binary string	8,000 bytes	
varbinary(max)	Variable width binary string	2GB	
image	Variable width binary string	2GB	

String Data Types

Data type	Description	Storage
bit	Integer that can be 0, 1, or NULL	
tinyint	Allows whole numbers from 0 to 255	1 byte
smallint	Allows whole numbers between -32,768 and 32,767	2 bytes
int	Allows whole numbers between -2,147,483,648 and 2,147,483,647	4 bytes
bigint	Allows whole numbers between -9,223,372,036,854,775,808 and 9,223,372,036,854,775,807	8 bytes
decimal(p,s)	Fixed precision and scale numbers. Allows numbers from $-10^{38} + 1$ to $10^{38} - 1$. The p parameter indicates the maximum total number of digits that can be stored (both to the left and to the right of the decimal point). p must be a value from 1 to 38. Default is 18. The s parameter indicates the maximum number of digits stored to the right of the decimal point. s must be a value from 0 to p. Default value is 0	5-17 bytes

numeric(p,s)	Fixed precision and scale numbers. Allows numbers from $-10^{38} + 1$ to $10^{38} - 1$. The p parameter indicates the maximum total number of digits that can be stored (both to the left and to the right of the decimal point). p must be a value from 1 to 38. Default is 18. The s parameter indicates the maximum number of digits stored to the right of the decimal point. s must be a value from 0 to p. Default value is 0	5-17 bytes
smallmoney	Monetary data from -214,748.3648 to 214,748.3647	4 bytes
money	Monetary data from -922,337,203,685,477.5808 to 922,337,203,685,477.5807	8 bytes
float(n)	Floating precision number data from $-1.79E + 308$ to $1.79E + 308$. The n parameter indicates whether the field should hold 4 or 8 bytes. float(24) holds a 4-byte field and float(53) holds an 8-byte field. Default value of n is 53.	4 or 8 bytes
real	Floating precision number data from $-3.40E + 38$ to $3.40E + 38$	4 bytes

Number Data Types

Data type	Description	Storage
datetime	From January 1, 1753 to December 31, 9999 with an accuracy of 3.33 milliseconds	8 bytes
datetime2	From January 1, 0001 to December 31, 9999 with an accuracy of 100 nanoseconds	6-8 bytes
smalldatetime	From January 1, 1900 to June 6, 2079 with an accuracy of 1 minute	4 bytes
date	Store a date only. From January 1, 0001 to December 31, 9999	3 bytes
time	Store a time only to an accuracy of 100 nanoseconds	3-5 bytes
datetimeoffset	The same as datetime2 with the addition of a time zone offset	8-10 bytes
timestamp	Stores a unique number that gets updated every time a row gets created or modified. The timestamp value is based upon an internal clock and does not correspond to real time. Each table may have only one timestamp variable	

Date Data Type

Data type	Description
sql_variant	Stores up to 8,000 bytes of data of various data types, except text, ntext, and timestamp
uniqueidentifier	Stores a globally unique identifier (GUID)
xml	Stores XML formatted data. Maximum 2GB
cursor	Stores a reference to a cursor used for database operations
table	Stores a result-set for later processing

Answers to Self-Assessment Questions

Module One

6.1.1 Unit 1: Answers to Self-Assessment Questions

Q1: What is the purpose of indexing in a database, and how does it improve performance?

Indexing creates lookup structures to improve query speed. It enhances performance by allowing the database to quickly locate relevant records without scanning the entire dataset. Techniques like B-trees and hash indexes are used to build index structures. Search mechanisms utilise these indices to accelerate retrieval, especially in large datasets.

Q2: Explain the differences between data privacy, data integrity, and data security.

1. Data Privacy ensures that sensitive data, such as medical or financial records, is not accessed by unauthorised individuals. It is enforced using access control (e.g., role-based permissions), data anonymization, and compliance with privacy laws like GDPR or HIPAA.
2. Data Integrity ensures that the data remains accurate and consistent throughout its lifecycle. This is achieved through constraints (e.g., NOT NULL, UNIQUE, CHECK), referential integrity (via foreign keys), and transaction management using the ACID properties.
3. Data Security protects the database against unauthorised access, tampering, or destruction. Measures include authentication, encryption, firewalls, database hardening, regular backups, and disaster recovery planning.

6.1.2 Unit 2: Answers to Self-Assessment Questions

Q1: Define a database and list at least three of its key components.

A database is an extensive collection of data organised especially for rapid search and retrieval. It is a shared collection of logically related data (and its description), designed to meet an organisation's information needs.

Three key components of a database include:

- Tables (relations) that store data.
- Metadata or system catalogue describing data
- Application programs that interact with the data

Q2: What is data abstraction, and what are the three levels involved in it?

Data abstraction is the process of hiding the complexity of data representation from the user while allowing interaction with only the necessary information.

The three levels of abstraction are:

- External level – the highest level; users see data in forms like tables and rows.
- Logical/Conceptual level – describes actual data stored and the relationships.
- Physical level – the lowest level that defines how data is stored on hardware devices.

Q3: Differentiate between physical data independence and logical data independence with examples.

- Physical Data Independence is the ability to modify the physical schema without affecting application programs.
- Example: Changing from a sequential file to a B+ tree file structure does not require changes to the logical schema or application programs.
- Logical Data Independence is the ability to change the logical schema without requiring changes to the external schema or applications.

Example: Adding a new column, such as 'skills', to a student table will not affect views that only display names and ages.

Q4: What are the primary functions of a Database Management System (DBMS)?

The primary functions of a DBMS include:

- Defining data structures through DDL
- Manipulating data through DML
- Managing storage and access paths
- Controlling concurrent data access
- Enforcing integrity constraints
- Ensuring data security and recovery

Q5: Describe two types of database systems and their typical use cases

- **Personal Database Systems** – Support one user and application, involve few tables, and operate on a single computer.
Use Case: A personal expense tracker or standalone desktop application.
- **Enterprise-Level Database Systems** – Support multiple users and applications across various systems with complex designs.
Use Case: Company-wide ERP systems managing finance, HR, and operations.

Q6: Explain the role of the database administrator and list three key responsibilities

The database administrator (DBA) is a person with central control over data and the programs that access it.

Three key responsibilities are:

- Schema definition and modification using DDL
- Granting authorisation for data access

- Specifying integrity constraints to maintain accuracy
- The primary functions of a DBMS include:

Q6: What is the purpose of a Data Manipulation Language (DML)? List four common operations.

A Data Manipulation Language (DML) provides a set of operations for accessing and manipulating data in a database. Its purpose is to enable efficient human interaction with the system.

Four common DML operations include:

- Retrieval of information from the database
- Insertion of new information
- Deletion of existing information
- Modification of existing information

Module Two

6.2.1 Unit 1: Answers to Self-Assessment Questions

Q1. Is there a difference between data modelling and conceptual modelling? Justify your answer.

Yes, there is a difference.

- Data modelling is a broad term that refers to the process of defining and analysing data requirements to support business processes.
- Conceptual modelling, on the other hand, is a specific type of data modelling that focuses on representing the high-level structure of organisational data using diagrams like the Entity-Relationship Diagram (ERD).
- While data modelling includes conceptual, logical, and physical levels, conceptual modelling is limited to the initial phase where entities, attributes, and relationships are identified without concern for physical implementation.

Q2. What determines how data are handled?

Data handling is determined by business rules, data modelling decisions, and system design choices.

- Business rules define how data is created, updated, and deleted.
- Data models represent the organisation's rules, policies, and data relationships.
- Conceptual models like the ER diagram ensure that entities and relationships reflect real-world business logic, including constraints such as cardinality, attribute requirements, and data integrity..

Q3. Relationship instance and Associative entity are similar but different.

Similarities:

- Both represent associations between entity instances in a database system.
- Both are essential in modelling many-to-many relationships.

Differences:

- A relationship instance is a specific occurrence of a relationship between two or more entities (e.g., an EMPLOYEE completing a COURSE).
- An associative entity is an entity type that exists to represent a many-to-many relationship and may contain attributes specific to that relationship (e.g., CERTIFICATE with DateCompleted).
- Associative entities are used when relationships have their own properties, while relationship instances do not contain additional attributes.

6.2.2 Unit 2: Answers to Self-Assessment Questions

Q1. The two most important types of constraints in Supertype/Subtype Relationships are?

The two most important types of constraints in Supertype/Subtype Relationships are:

- Completeness constraints
- Disjointness constraint

Q2. What does the partial specialisation rule specify? How is it different from the total specialisation rule?

- The partial specialisation rule specifies that an entity instance of the super-type is allowed not to belong to any subtype.
- In contrast, the total specialisation rule specifies that each entity instance of the supertype must be a member of some subtype in the relationship.

Q3. What are the processes that serve as the mental models in developing supertype/subtype relationships?

The processes that serve as mental models in developing supertype/subtype relationships are:

- Generalization
- Specialization

6.2.3 Unit 3: Answers to Self-Assessment Questions

Q1. Is XML the same as HTML? Justify your answer.

No, XML is not the same as HTML.

- XML (eXtensible Markup Language) is used to describe and store data with a focus on structure and meaning. It allows users to define their own custom tags and is both machine-readable and human-readable.
- HTML (HyperText Markup Language) is used to display data on web pages. It has a predefined set of tags and focuses on content presentation.
- XML is used for data transport and representation, while HTML is for formatting and displaying content in browsers.
- What mass will remain after 10 hours?

Q2. Explain the Semi-Structured Data types.

Semi-structured data types include:

- **Flexible Schema:** Each record can have different fields or attributes, allowing easy schema evolution. Examples include wide-column or sparse-column representations.
- **Multivalued Data Types:** Attributes can hold collections like sets or arrays. For example: {basketball, La Liga, cooking}.
- **Nested Data Types:** Attributes can themselves be composite, containing sub-elements or sub-attributes. This allows hierarchical data representation, such as in JSON or XML.

These types support non-tabular, hierarchical, and self-describing data models, ideal for modern data exchange and web applications.

Module Three

6.3.1 Unit 1: Answers to Self-Assessment Questions

Q1. What is the purpose of the relational keys?

Relational keys are used to ensure logical consistency and uniqueness within database tables.

- A primary key is an attribute or combination of attributes that uniquely identifies each row in a relation.
- A composite key is a primary key made up of more than one attribute.
- A foreign key is an attribute in one relation that serves as the primary key of another relation, enabling relationships between tables.
- Their purpose is to maintain connections between tables, prevent duplicate rows, and ensure that each record can be accurately retrieved and referenced.

Q2. What are integrity constraints? Explain them.

Integrity constraints are rules applied to ensure the accuracy, consistency, and validity of data in a relational database. There are three major types:

- **Domain Constraint:** specifies the permissible values for an attribute, including data type, length, and value range.
- **Entity Integrity:** ensures that every table has a primary key and that none of its values is null. This guarantees that each record can be uniquely identified.
- **Referential Integrity:** ensures that foreign key values in one table must either match a primary key in another table or be null, maintaining consistency across related tables

6.3.2 Unit 2: Answers to Self-Assessment Questions

Q1. What is the difference between a conceptual model and a logical model in database design?

- A conceptual model focuses on high-level data requirements using ER or EER diagrams, representing entities, relationships, and attributes without concern for how data will be stored.
- A logical model, on the other hand, translates the conceptual model into relational tables, defining primary keys, foreign keys, and constraints to ensure consistency and implementation feasibility in a relational DBMS.

Q2. Explain how a regular entity with a composite attribute is transformed into a relational table

Only the simple components of the composite attribute are included as columns in the relation. For example, a composite attribute like Address with components Street, City, and State is transformed into separate attributes: Street, City, and State in the relational table.

Q3. How are multivalued attributes handled during the transformation process?

- Two separate relations are created:
 - One for the entity excluding the multivalued attribute.

- Another that includes the multivalued attribute and the primary key of the first relation (as a foreign key).
- The combination of these two attributes forms the composite primary key of the second relation

Q4. What is the purpose of using a surrogate key in complex relationships?

- A surrogate key is a system-generated unique identifier used to simplify primary key structures, especially when natural keys are long or composite.
- It improves efficiency, ensures uniqueness, and reduces complexity in many-to-many or ternary relationships.

Q5. Describe the steps involved in transforming a binary many-to-many (M:N) relationship.

- Create a new relation for the M:N relationship.
- Include the primary keys from both participating entities as foreign keys in this new relation.
- These foreign keys together form a composite primary key.
- Add any attributes associated with the relationship to this new relation.

Q6. How is a unary one-to-many relationship represented in a relational schema?

- Add a recursive foreign key to the relation that references its own primary key.
- This foreign key links a row in the table to another row in the same table, modelling the one-to-many relationship within a single entity.

Q7. Explain how to transform a supertype/subtype relationship into relational tables.

- Create a separate relation for the supertype with common attributes and a unique primary key.

- For each subtype, create a relation that includes:
- The primary key from the supertype (as both a primary and foreign key).
- Any attributes unique to that subtype.
- Optionally, include a discriminator attribute in the supertype to indicate the subtype classification.

Module Four

6.4.1 Unit 1: Answers to Self-Assessment Questions

Q1. What is normalisation, and why is it important in relational database design?

- Normalisation is the process of decomposing relations with anomalies into smaller, well-structured relations.
- It ensures that data are logically structured, reduces data redundancy, and eliminates update, insertion, and deletion anomalies.
- It simplifies the enforcement of referential integrity constraints, ensures better data integrity, and provides a stronger foundation for future data-base growth.
- Normalisation is a key part of logical database design and helps represent real-world entities more accurately.

Q2 Differentiate between First Normal Form (1NF) and Third Normal Form (3NF).

- A relation is in First Normal Form (1NF) if:
 - All repeating groups have been removed.
 - Each column contains atomic values (single facts).
 - A primary key uniquely identifies each row.
- A relation is in Third Normal Form (3NF) if:
 - It is already in Second Normal Form (2NF).

- It contains no transitive dependencies—i.e., non-key attributes are only dependent on the primary key, not on other non-key attributes.
- 3NF ensures a higher level of data integrity than 1NF by removing both partial and transitive dependencies. Integrating both sides of the equation gives.

Q2 Explain the concept of functional dependency and how it is used in the normalisation process

- A functional dependency is a constraint between two attributes such that the value of one attribute (A) uniquely determines the value of another attribute (B), written as $A \rightarrow B$.
- Functional dependencies help identify how attributes relate to one another in a table and are the foundation for defining normal forms.
- During normalisation, identifying functional dependencies allows us to detect and eliminate partial and transitive dependencies, leading to better-structured and anomaly-free tables.

Module Five

6.5.1 Unit 1: Answers to Self-Assessment Questions

Explain the concept of functional dependency and how it is used in the normalisation process

- The select operation (σ) retrieves tuples (rows) from a relation that satisfy a specified condition or predicate.
 - Example: To select instructors with a salary below 50,000 from the instructor relation: $\sigma_{\text{salary} < 50000}(\text{instructor})$
- The project operation (π) retrieves specific attributes (columns) from a relation, effectively filtering out unwanted columns.
 - Example: To display only courseid and credits from a course relation: $\pi_{\text{courseid}, \text{credits}}(\text{course})$
- While select filters rows, project filters columns.

Q2. What is the purpose of the Cartesian product and join operations, and how do they differ in terms of output and use cases?

- The Cartesian product ($\times$) combines every tuple of one relation with every tuple of another, generating all possible combinations. It is used to merge relations when no explicit condition is given, typically as a precursor to a join.
- The JOIN operation ($\bowtie$) combines related tuples from two relations based on a common attribute or condition. The natural JOIN merges relations using attributes with the same name and eliminates duplicate columns.

Key Differences:

- Output: Cartesian product returns all combinations (may be large and irrelevant), while join returns only meaningful combinations based on matching attributes.
- Use Case: Cartesian product is rarely used directly; join is preferred when combining logically related data from multiple tables.

